

Structural Exhaustion

A Reference Manual for AI-Assisted Theorem Proving via Structural
Bookkeeping and Textbook Combinatorial Methods

Guillem Duran-Ballester

July 7, 2026

Abstract

Long AI-assisted proofs are difficult to inspect when case choices, residual obligations, and uses of quantitative slack remain implicit. This document specifies *structural exhaustion*, a reference manual for human-supervised, LLM-assisted development and review of structural-exhaustion and minimal-counterexample arguments. It is intended for extremal and combinatorial proofs that can be organized through explicit case splits, local reductions, finite-state reasoning, and quantitative budgets. The manual supplies three coordinated components: a typed library of proof tactics, a strategy manual for selecting and parameterizing them, and an assurance layer for auditing their inputs, outputs, and dependencies. A candidate proof is represented by finitely recorded branch states containing its standing hypotheses, certified exclusions on the current branch path, residual conditions, and target-relevant interface data. An instantiation is complete only when every terminal branch returns one of the five declared certificate classes and every nonterminal residual has a declared downstream consumer. Capacity closures combine local accounting inequalities. The remaining terminal classes cover direct target realization, minimality or compression, routed defect accounts, and finite enumeration. These are protocol requirements rather than a guarantee of automated discovery: the LLM proposes steps within the declared state system, while human reviewers verify the local mathematics, routing contracts, and terminal certificates. The Erdős–Gyárfás power-of-two-cycle instantiation provides the full-scale proof model motivating this architecture.

AI-use disclosure. AI tools were used as drafting and proof-engineering aids. The author reviewed all outputs and is responsible for all mathematical and methodological claims; model output is not used as evidence of

correctness.

Contents

1	Motivation and design rationale	7
1.1	The core pattern	8
1.2	Literature positioning	8
1.3	Design rationale: AI-assisted theorem proving	10
1.4	The formalization bridge	12
2	The tactic library	14
2.1	Scope and philosophy	15
2.2	Branch states and residuals	15
2.3	Pieces, interfaces, and outside contexts	23
2.4	The target predicate and local test families	24
2.5	External types	25
2.6	Labels	26
2.7	Ledgers: demands, payers, capacities	26
2.8	Budgets and currencies	27
2.9	Payloads	28
2.10	The certificate alphabet C1–C5	28
2.11	Lemma schemas	29
2.12	Notation and diagram legend	31
2.13	Terminological conventions	32
2.14	The master interlock graph	36
2.15	CT1 — Target-test alignment and first closure	47
2.16	CT2 — Minimality and replacement	51
2.17	CT3 — External-type compression	56
2.18	CT4 — Charging schemes	60
2.19	CT5 — Local-to-global bookkeeping	63
2.20	CT6 — Active/dormant dichotomy	67
2.21	CT7 — Exchange trichotomy	70
2.22	CT8 — Finite-state pumping	73
2.23	CT9 — Overload exhaustion	76
2.24	CT10 — Default refinement	79
2.25	CT11 — Localization	82
2.26	CT12 — Peeling loop	85
2.27	CT13 — Tiered charging	88
2.28	CT14 — Aggregate closure	90

2.29	CT15 — Rank forcing	93
2.30	CT16 — Whole-object exact types	95
2.31	CT17 — Target thickening	97
3	The strategy manual	100
3.1	Runtime model	100
3.2	Branch states	102
3.3	Closure criteria and the branch table	102
3.4	Branch taxonomy	104
3.5	The general branch-closure algorithm	106
3.6	Admission	107
3.7	Bookkeeping versus new theorems	108
3.8	Structural invariants of the method	109
3.9	Template for a new problem	110
3.10	Audit checklist for branch design	111
3.11	Choosing invariants and building budgets	111
3.12	Budget synthesis and rate checks	114
3.13	Monotonicity and design checklists	116
3.14	Selection policy	117
3.15	Nondeterminism inventory	118
3.16	Parameter synthesis	121
3.17	Combinators	126
3.18	Advanced closure patterns	126
3.19	Opening script and strategy narratives	129
3.20	Filling the templates	129
4	Assurance	131
4.1	Decoupling conformance checks	133
4.2	LLM and formalization bridge	134
4.3	Constant interlock and computational certificates	135
4.4	Proof-engineering artifacts	135
4.5	Redundant closures and graceful degradation	136
4.6	The defect register: recording discovered failures	144
4.7	Repair taxonomy and common failure shapes	146
4.8	Portability	149
4.9	Limitations	150
A	Glossary	153
B	Instantiation map: the Erdős–Gyárfás power-of-two-cycle	

instantiation	163
B.1 Section-by-section correspondence	163
B.2 Closure-tactic instantiation map	174
B.3 EG proof-flow charts	174
B.4 Advanced patterns in EG	175
B.5 Reading order	178
C A small worked instantiation	186
C.1 Target predicate and local tests	188
C.2 Minimal counterexample and resource inventory	188
C.3 A failed peeling proposal and the recorded invariant	189
C.4 The branch invariant by hypothesis extraction	190
C.5 A failed global-estimate shortcut	190
C.6 The leaf: overload, monochromatic pair, direct hit	191
C.7 The closing artifacts	191
C.8 Scope of the example	192
D Library index and quick-reference card	192

Introduction

In 1977, Appel and Haken, with Koch, proved the Four-Colour Theorem using reducibility, discharging, and computational verification [??]. Their proof remains a useful reference point for methodologies that replace a compact global argument by an explicit family of local obligations. The present manual studies a related proof-engineering trade-off in AI-assisted combinatorial theorem proving.

The cost profile of proof engineering has changed with contemporary language-model tools. This manual adopts a conservative operating assumption: model output is not treated as an autonomous source of global mathematical conclusions. Within a constrained interface, a model can instead maintain structured state, expand local cases, enforce fixed formats, and draft repetitive derivations for human review. Structural exhaustion formalizes that interface as a transition system over branch states, typed residuals, local certificates, finite labels, ledgers, budgets, and dependency records.

The architecture is designed against a declared threat model for LLM-assisted proof development. A generated draft may omit the step that carries the difficulty, use a global estimate with an unrecorded independence assumption or double count, restate the original problem as an auxiliary lemma, supply

unsupported proof-shaped prose, or accept an incorrect proposed step. These are design risks that the protocol is intended to expose, not empirical claims that every model or proof task exhibits them uniformly. The manual therefore requires local obligations, typed residuals, and checkable certificates at the proof interface.

Stronger models, formal verification, formal-first development, interactive theorem proving, retrieval, and human review can all improve reliability. Structural exhaustion addresses the complementary question of how a proof is developed before final verification: it decomposes proof search into constrained local obligations whose outputs are typed, routed, and audited.

The framework is based on the following organizing principle. A putative counterexample can avoid the target only by spending some scarce resource, violating an accumulated exclusion, or creating a repeatable local operation. Scarce resources are charged by ledgers and budgets. Exclusions are recorded in the branch state and spent by dichotomies. Repetitions are consumed by pumping, exchange, exact typing, or compression. The proof does not ask for an unstructured global insight; it maintains these accounts until a branch produces a certificate or a typed residual routed to a downstream closure tactic.

The machinery converts a minimal-counterexample argument into the maintenance of explicit data structures: branch states that accumulate every standing hypothesis and every prior exclusion, external types recording what a bounded piece can exhibit to its surroundings, ledgers routing demands to scarce payers, budgets in a fixed normal form, and typed payloads passing between proof steps. These data evolve under a fixed transition system of seventeen **closure tactics** whose moves are textbook: minimality and replacement, charging, exchange, pigeonhole, finite-state repetition, compression, and finite enumeration. A completed instantiation is leaf-total. Every terminal branch must end in one of the five certificate classes C1–C5. A residual is not a final state; it is a typed payload with a declared consumer in the interlock graph. If a branch emits a residual with no consumer, the proof has not reached an open mathematical leaf — it has failed to compile. Every step either closes a branch or emits a strictly smaller, explicitly typed residual, so an omitted step is not a stylistic lapse but a syntactic violation, visible before any mathematical judgment is required. Global estimates are admitted only after their hypotheses and accounting steps have been decomposed into named, auditable premises. Capacity inequalities are one recurring output of that decomposition.

The manual separates mechanism, policy, and verification. Section 1 motivates the architecture; Section 2 defines the proof language, tactic interfaces, and local obligations; Section 3 governs admission, parameter synthesis, selection, and sequencing; and Section 4 supplies the audit artifacts and failure semantics. The interfaces make the obligations explicit and checkable; proving an instance-specific obligation may still require substantial mathematical judgment.

Two properties distinguish the resulting proofs from conventional long arguments, and both are consequences of the no-unnamed-residuals discipline. The first is graceful degradation. Because every branch failure produces a named, typed residual class, the retraction of any closing lemma does not collapse the proof; it relocates the conclusion to an explicit reduction theorem, with a computable blast radius. Partial verification is therefore not partial confidence: a reader who has checked some of the branches has unconditionally established the corresponding reduction theorem, and further checking only moves the result up a lattice whose top is the unconditional statement. The second is a three-label audit taxonomy. This taxonomy is an operational policy, not a theorem that every objection falls immediately and uniquely into one class. An apparent non-closure is recorded as an execution error only when a required obligation is absent or invalid, as a design failure only when a typed obstruction has a declared repair route, and as a scope obstruction only after the language-extension audit has established nonexpressibility in the current proof language. The base language is designed for targets represented by finite local tests and finite type data under bounded-interface gluing. The spectral-gap discussion in the limitations section is a proposed example of extending that language with cut and energy descriptors, not a claim that every spectral argument is thereby admitted.

The primary case study is the companion candidate manuscript *A Structural Exhaustion Proof of the Erdős–Gyárfás Power-of-Two Cycle Problem* [?], hereafter [EG]. The release package contains its source as `../erdos_64_proof.tex` and its live review dossier as `../erdos_64_proof_review.md`. The dossier records known defects, repairs, and open review obligations; accordingly, [EG] is presented as a candidate proof under public review rather than as an independently verified result. The appendix maps this manual to [EG], and Appendix C gives a smaller worked example. Readers are invited to check the companion’s named interfaces, computations, and unresolved routes and to contribute corrections or repairs.

Table 1: Rosetta table for the main structural-exhaustion vocabulary.

Framework term	Classical neighbour	What the framework adds
Branch state	Case with hypotheses	The full accumulated context: baseline assumptions, minimality, certified exclusions accumulated along the current branch path, invariants, residual condition, and queued payloads.
Local test	Local witness or obstruction	A bounded family whose realization is equivalent to the target inside the declared scope.
External type	Myhill–Nerode-style context equivalence; logical type	A live proof coordinate deciding exits, neutral exchanges, replacement, and typed handoffs.
Ledger	Charging or discharging account	Canonical demands, payers, capacities, tie-breaking, and bounded multiplicity as auditable data.
Budget	Potential function	A normal-form quantitative account with recorded currencies, additivity, monotonicity, and slack.
Payload	Intermediate lemma conclusion	A typed tuple whose producer and consumer are both recorded, so every non-closing exit has a declared route.
Closure tactic	Standard proof move	A fixed mechanism interface returning only a certificate C1–C5 or a named residual payload.
Residual	Remaining subcase	A fully typed branch state that can stand alone as a reduction theorem if its closure later fails.

1 Motivation and design rationale

The method targets theorems of the shape

every object satisfying baseline hypotheses
contains a specified target configuration.

One assumes a minimal counterexample and converts the proof into a con-

trolled elimination of every way it could avoid the target.

1.1 The core pattern

The strategy has five recurring layers:

minimal counterexample $\Rightarrow$ local state space $\Rightarrow$ branch split
 $\Rightarrow$ charging / compression / finite routing $\Rightarrow$ contradiction.

The essential discipline is that **every branch is an explicit state**. The logical core of the formal branch-state tuple records the accumulated context of a case: to say “we are in branch B ” includes

B = baseline hypotheses + minimality
 + certified exclusions on the current branch path
 + current residual condition.

This prevents a pervasive error in case-based arguments: treating a residual as if it were unconstrained. In this method, every residual carries the full history of constraints under which it arose. The full eight-component state is defined in Section 2.2.

1.2 Literature positioning

The method packages standard proof moves into an explicit branch-closure architecture. Its primitive moves are inherited from minimal-counterexample arguments, discharging, finite-state repetition, and replacement. The closest predecessors are the discharging and reducibility tradition around the four-colour theorem, especially the work of Appel–Haken and the later Robertson–Sanders–Seymour–Thomas proof [????]. Cranston–West serves as a general survey of discharging as a reusable method [?]. Those sources are the right comparison point whenever the present document speaks about local reducibility, finite certified leaves, unavoidable local configurations, or charging/discharging arguments. In the classical proofs, the reducible configurations, discharging rules, and potential-like quantities enter through problem-specific insight. In the present framework, the same kinds of objects become branch-state data generated from the resource inventory, the both-sides test, and the budget checklist, so several interacting charges and invariants can be maintained simultaneously.

The second comparison point is the finite-state and context-equivalence literature. The external-type language below is deliberately Myhill–Nerode-like: two pieces have the same type when no compatible context can separate them with respect to the target [??]. In graph and logic settings this viewpoint is aligned with Courcelle’s monadic second-order treatment of graph properties and with Courcelle–Engelfriet’s language-theoretic account of graph structure [??]. In those settings, types commonly support recognition, automata, or dynamic programming. Here they also function as live proof coordinates: an external type records which exits remain available, when an exchange is neutral, and when replacement/minimality can close a branch.

In parameterized complexity, the closest technical relatives are finite integer index, boundaried graphs, protrusion replacement, and explicit representative replacement. The relevant line includes Bodlaender–Fomin–Lokshtanov–Penninkx–Saurabh–Thilikos and subsequent work of Kim and coauthors, Garnero and coauthors, and Jansen–Wulms [????]. That literature supplies representative sets and replacement relations for algorithmic reduction. The present framework uses the same finite-interface discipline as a routing language inside a contradiction proof: representatives, carrier quotients, and exact types are generated as needed by the branch state and then consumed by compression, defect, or handoff routes.

The Erdős–Gyárfás power-of-two-cycle instantiation places the method in the cycle-length forcing literature around the Erdős–Gyárfás power-of-two cycle problem. The relevant background includes the classical even-cycle extremal theorem of Bondy–Simonovits, the sparse cycle-length results of Sudakov–Verstraete, and the recent class-restricted progress for P_8 -free, P_{10} -free, and P_{13} -free graphs by Gao–Shan, Hu–Shen, and Hegde–Sandeep–Shashank, respectively [????]. Carr’s 2026 structural result is especially relevant because it also studies minimal counterexamples to the Erdős–Gyárfás conjecture [?]. The comparison here is methodological: the cycle-length literature provides problem-specific structural insight, while the branch-closure framework specifies how local tests, first-failure witnesses, target thickenings, and budget rates are produced and sequenced inside a single auditable state system.

Finally, the machine-assistance design discipline belongs beside the recent literature on autoformalization and proof-search scaffolding. The closest anchors are Wu et al. on autoformalization, Jiang et al. on Draft–Sketch–Prove, and Yang et al. on LeanDojo [???]. The present document functions as a proof-architecture discipline: its point of contact with that literature is the shared need to decompose long arguments into explicit local obligations

with recorded dependencies. The distinction is that the present framework organizes the informal combinatorial construction before formalization: it generates ledger entries, invariant registers, handoff payloads, and closure certificates that can later be checked by a human or a formal system.

The contribution is a reference architecture rather than a new primitive combinatorial move. Its concrete components are a typed proof-language specification, seventeen tactic contracts, a registered payload-routing system, strategy and repair procedures, an [EG] crosswalk, and a complete small worked example. Together these components record which hypotheses, invariants, quantitative accounts, and residual routes support each closure step.

1.3 Design rationale: AI-assisted theorem proving

Beyond its mathematical content, structural exhaustion is engineered as a framework for AI-assisted theorem proving. It assigns language models structured tasks such as maintaining state, proposing case splits, filling ledgers, generating local tests, and drafting named obligations inside a fixed audit format. It shares with autoformalization, proof-sketch-guided proving, and retrieval-augmented formal proving the premise that long arguments become tractable when decomposed into explicit local obligations with visible dependencies [???]. The following seven items are operational risks observed while developing [EG] and are treated here as design assumptions. They are not claims about the frequency of these behaviours across all models or proof tasks. The method turns each risk into a structural check.

Failure mode 1: omitted difficult steps. A generated draft may read like a proof while omitting the step that carries the difficulty. The countermeasure is the closure discipline of Section 3.3 and Section 3.8. A branch closes when every leaf terminates in one of a fixed, finite list of certificate types, and every residual has a name. Phrases such as “this structure should be impossible” or “this cannot happen generically” are classified as *prompts for further stratification*. The enumerated terminal states expose an omitted step syntactically, before any mathematical judgment is required.

Failure mode 2: unsupported global estimates. An unconstrained draft may gravitate toward a single global inequality — a counting argument, an entropy bound, a spectral estimate — that appears to resolve everything at once. Such estimates concentrate the common errors: an unjustified independence assumption, a double count, or a boundary term absorbed

without justification.

In the simplest unweighted case, the template admits a statement of the form $|\mathcal{D}| \leq C |\mathcal{P}|$, obtained from a canonical charging scheme and a bounded-multiplicity lemma (Section 2.18). The weighted and payer-specific form is the ledger inequality of Section 2.7. Each ingredient is a named local claim, and the global conclusion is the output of that bookkeeping.

Failure mode 3: re-encoding the difficulty. A model that stalls on a step may request “one more lemma” whose statement restates the original problem at another level. The progress invariant of Section 3.3 detects this relocation: every step outputs either a closed branch or a *strictly smaller* named residual — smaller in size, in the number of live exits, or in an explicitly monitored complexity parameter.

A proposed lemma qualifies as progress when its residual is smaller, finite, charged, or routed. Section 3.7 supplies the complementary diagnostic: a bookkeeping step factors through overload, monochromatic extraction, and the exchange trichotomy; an unconditional statement about all large objects with some property is a new global theorem requiring further branch refinement.

Failure mode 4: extrapolation beyond standard material. The protocol assumes that model-generated steps require closer scrutiny when an argument leaves well-represented material. The base library therefore organizes a small vocabulary of standard mechanisms: minimal-counterexample replacement, charging arguments with bounded multiplicity, pigeonhole and finite-state repetition, matching and star extraction, exchange arguments, and exhaustive finite enumeration. Instance-specific lemmas, imported results, and proof-language extensions remain separate audited obligations; membership in the library does not make their proofs automatic.

Failure mode 5: untyped residuals. A model may declare that the adversarial object has none of the structures required by the available tools and then treat the remaining case as a reason to stop. The countermeasure is the branch-state ledger. The absence of a feature is recorded as a positive fact and routed: it feeds a budget, strengthens an exclusion, creates a repetition payload, or triggers a level-4 language extension under Section 4.5. An untyped residual is an invalid worksheet row, not an open mathematical case.

Failure mode 6: deferential agreement with erroneous steps. A model in an assistant role may accept a user-supplied step whose hypotheses,

quantifiers, branch state, or output type do not match the current proof obligation. The failure is not mathematical creativity; it is loss of interface discipline. The countermeasure is that agreement has no formal status. A proposed step enters the system only as a worksheet row with declared inputs, quantitative data, schema obligations, and a typed output. The schema alphabet is defined in Section 2.11; the row is accepted only after the relevant S-Def, S-Dich, S-Comp, S-Rout, S-Trig, or S-Meas obligation is discharged. When the row fails, the required response is a typed reject, a named residual, or an alternative routed payload, not conversational assent.

Failure mode 7: status-cue audit drift. A model may replace a local audit by an appeal to a problem’s published status. Status is useful context but is neither a certificate nor a defect. Positive steps and red-team objections use the same interface: each objection must name a failed schema, missing import, stale branch state, unconsumed payload, unregistered global theorem, or candidate scope issue. Inventing a new terminal such as “open problem” or “untyped residual” is return-alphabet drift. Certification remains a property of the documented mathematical, computational, and review artifacts.

1.4 The formalization bridge

The artifacts of this framework are designed to admit possible formal encodings. The table describes intended correspondences rather than a claim that the encodings have already been implemented. The framework is **pre-formal**: it organizes the informal combinatorial construction into local obligations with recorded dependencies, the same shape consumed by sketch-guided and retrieval-augmented formal proving [???].

Table 2: Intended formal encodings of the manual’s audit artifacts.

Artifact in this document	Formal counterpart	Consequence
Branch state	The local hypothesis context in a proof assistant; entering a branch extends the context	Cumulative constraints are enforced by the kernel rather than by reader memory

Table 2: Intended formal encodings of the manual’s audit artifacts.

Artifact in this document	Formal counterpart	Consequence
Invariant register	A named-lemma ledger, one declaration per invariant	“Established at stage X , consumed at stage Y ” becomes import order
Typed handoff	A lemma signature, with the payload as the explicit argument list	A format mismatch is a type error before proof search begins
Branch table	An exhaustive case split with no wildcard arm	“No unnamed residuals” becomes checked exhaustiveness
Closure certificates	Terminal tactics: contradiction, minimality, and verified computation for finite leaves	The certificate list of Section 3.3 becomes a fixed tactic vocabulary
Finite enumerations and published code	Decision procedures or verified computation	The code appendix becomes a decidable instance rather than trusted output
Peeling loop (Section 2.26)	Well-founded recursion with an explicit termination measure	The per-pass peel lemma is the decreasing-measure obligation
Exact external type (Section 2.30)	An explicit data structure, with identification only through a proved equivalence	Quotienting through the empty context requires a stated equality proof
Reverse index (Section 4.4)	Machine-assisted premise/dependency extraction over the development	The blast radius of Section 4.5 is a graph query

Table 2: Intended formal encodings of the manual’s audit artifacts.

Artifact in this document	Formal counterpart	Consequence
Slack table (Section 3.13)	Numeric side conditions discharged by interval arithmetic, each with recorded margin	The binding constraint is a named, re-checkable obligation
Black-box register (Section 3.18)	The axiom or assumption list of the development	The trust base is printable on demand

Two intended workflow consequences follow. The recorded dependencies permit branch-by-branch formalization in an order compatible with the invariant ladder. A reduction theorem is formally verified only when that theorem and every dependency it invokes have been encoded and checked by the proof assistant’s kernel; unchecked branches remain outside the verified statement.

2 The tactic library

Section 2 is the tactic reference manual. It records the base CT1–CT17 library in numerical order for this dated release and treats the diagrams as part of the tactic entries. Each tactic consumes a recorded branch state and returns either a canonical closure certificate or a named residual whose payload is routed to another tactic. Entries are mechanism-only: they describe what the tactic needs, what it does, which obligation schemas it discharges, and what it emits. They do not contain selection advice, parameter-synthesis recipes, problem-specific choices, or comparisons between alternative tactics. A CT entry may contain local implementation instructions once its input data have been supplied: branch order, comparison order, restoration checks, payload construction, and certificate verification. Rules for choosing the next CT, delaying a CT, or selecting among competing routes belong to Section 3. An extension uses a distinct tactic identifier and declares which base signatures it preserves or supersedes.

Status of the entries. CT1–CT17 are conditional protocol specifications, not assertions that their hypotheses hold for every proof problem. An invocation is admissible only after the current proof supplies the definitions, lemmas, and schema obligations named in its entry. The diagrams record required obligations and permitted outputs; an instantiated claim becomes a proved result only through the cited problem-specific lemmas.

2.1 Scope and philosophy

The base proof language applies to minimal-counterexample arguments whose target can be expressed through bounded local data; extensions follow the protocol of Section 4.5. The governing separation is mechanism, policy, and verification. **Mechanism** is a rigid tactic interface: an input payload, parameter slots, obligation schemas, and certificate or payload outputs. **Policy** is the choice of which trigger to propose, how to synthesize parameters, and how to sequence payloads. **Verification** is the audit layer: worksheets, branch-table rows, invariant registers, dependency graphs, slack tables, defect records, and reduction-lattice statements. The vocabulary block defines proof data, CT entries define tactic operations, and Section 3 governs tactic choice and sequencing.

2.2 Branch states and residuals

Definition 2.1 (Ambient class). An **ambient class** $\mathcal{U}$ is the class of finite objects on which the theorem is stated. It is part of the proof instance, not a background convention. It specifies the following data:

- the notion of subobject;
- the notion of isomorphism;
- the rule for gluing along a declared interface;
- the size function;
- the ambient operations used by the proof.

Every object quantified over by a branch state is an element of $\mathcal{U}$ or a typed object constructed from one.

Examples include:

- finite simple graphs for graph problems;

- colourings, hypergraphs, matroids, words, labelled complexes, or finite relational structures for other problem classes.

Definition 2.2 (Target predicate). A **target predicate** is a map

$$T : \mathcal{U} \longrightarrow \{0, 1\}.$$

The theorem has the form

$$G \in \mathcal{U}, \quad G \models \mathcal{H}_{\text{thm}} \implies T(G) = 1.$$

A counterexample branch works under $T(G) = 0$. The local test family of Section 2.4 is a representation of this predicate by bounded witnesses; it is not a replacement for the predicate itself.

Definition 2.3 (Proof language). A **proof language** $\mathfrak{L}$ is the declared typed language available to the proof at a given stage of the construction. It consists of the following seven typed components.

$$\mathfrak{L} = (\text{Sorts}, \text{Constructors}, \text{Predicates}, \text{Measures}, \\ \text{Certificates}, \text{Payloads}, \text{BlackBoxes}).$$

The seven coordinates are interpreted relative to a branch state $\mathfrak{B}$. They determine which objects may be named, how new objects may be constructed, which assertions are well typed, which quantities may be compared, which terminal certificates are legal, which residual payloads may be emitted, and which external theorems may be called. A level-4 repair extends $\mathfrak{L}$ by adding new typed coordinates of the same form; it does not license untyped residuals.

Definition 2.4 (Sort system). The component Sorts is a finite typed alphabet

$$\text{Sorts} = \{S_1, \dots, S_m\}.$$

Each sort S has, for every well-formed branch state $\mathfrak{B}$, an interpretation $\llbracket S \rrbracket_{\mathfrak{B}}$: the class of objects of that type available in the branch. The interpretation must be invariant under isomorphism of the ambient object and stable under branch-state extension: if $\mathfrak{B}' \succeq \mathfrak{B}$, then every object of sort S carried from $\mathfrak{B}$ to $\mathfrak{B}'$ keeps the same type. Typical sorts are ambient objects, pieces, boundary interfaces, outside contexts, local tests, witness spaces, external types, finite labels, ledgers, budgets, ranks, residual classes, payload tuples, certificates, and audit rows.

Definition 2.5 (Constructor system). The component Constructors is a finite set of typed partial operations

$$c : S_1 \times \cdots \times S_k \rightarrow S_0.$$

A constructor entry records its input sorts, output sort, domain condition, defining formula or algorithm, and well-formedness certificate. The value $c(x_1, \dots, x_k)$ may enter the vocabulary record $\mathcal{V}$ only when the inputs x_i are already recorded branch-state data and the domain condition has been proved. This is the object-level content of S-Def. Examples are forming a boundary profile from a piece, a charge map from a demand/payer rule, an external type from a context family, a rank quotient from exact response data, or a payload tuple from a residual condition.

Definition 2.6 (Predicate system). The component Predicates is a finite set of typed relations

$$P \subseteq S_1 \times \cdots \times S_k$$

with branch-state interpretations $\llbracket P \rrbracket_{\mathfrak{B}}$. A predicate entry records its input sorts, its admissible context, and the certificate schema required to use it. Predicate truth must be determined by recorded branch-state data or by a declared black-box handle. Typical predicates are admissibility of a piece, compatibility of a context, realization of a local test, equality of external response, boundedness of a fibre, availability of a payer, restoration of a loop state, and membership in a residual class.

Definition 2.7 (Measure system). The component Measures is a finite set of typed maps

$$M : S \rightarrow W_M,$$

where W_M is a declared ordered codomain. The codomain is well-founded when M is used for termination, ordered additively when M is used for a budget or ledger comparison, and finite when M is used as a label alphabet. Each measure entry records its domain, codomain, additivity or monotonicity rules, comparison lemmas, and slack or error terms. Measures include size orders, loop measures, finite labels, capacities, ranks, rates, budget functionals, and arithmetic offsets. S-Meas certifies well-founded descent; S-Comp certifies the comparisons made with these measures.

Definition 2.8 (Certificate component). The component Certificates is the set of terminal proof objects and their validation rules. In this document it contains the five terminal classes C1–C5 from Section 2.10, together with the required input sorts and audit rows for each class. A certificate entry specifies

the branch-state data it consumes, the schema obligations that validate it, and the theorem statement it discharges. A branch may terminate only by producing one of these certificates.

Definition 2.9 (Payload component). The component Payloads is the finite set of legal inter-tactic signatures

$$P_{i \rightarrow j} : S_1 \times \cdots \times S_k \longrightarrow \text{Payload}_{i \rightarrow j}.$$

Each payload entry records the producer CT*i*, the consumer CT*j*, the ordered tuple of component sorts, the branch-state extension it carries, and the trigger condition required by the consumer. Producing the payload requires S-Rout; consuming it requires S-Trig. A residual with no entry in Payloads is not a legal output of the proof language.

Definition 2.10 (Black-box component). The component BlackBoxes is a finite register of imported statements. A black-box entry records the theorem name, hypotheses, conclusion, constants, parameter ranges, source citation or proof location, and the typed interface through which the result may be used inside a branch state. It also records whether the black box supplies a predicate, constructor, measure comparison, or certificate. A later branch may call only the conclusion stated in the register and only when the branch state proves the listed hypotheses.

Definition 2.11 (Proof instance). A **proof instance** is a tuple

$$\mathfrak{P} = (\mathcal{U}, T, \mathcal{L}, \mathcal{H}_{\text{thm}}, \preceq_0),$$

where $\mathcal{U}$ is the ambient class, T is the target predicate, $\mathcal{L}$ is the initial proof language, $\mathcal{H}_{\text{thm}}$ is the theorem’s baseline hypothesis scheme, and $\preceq_0$ is the root well-founded order used to choose a minimal counterexample. All branch states in the proof are formed over the current proof instance, possibly after explicitly recorded proof-language extensions.

Definition 2.12 (Baseline hypotheses $\mathcal{H}_0$). The component $\mathcal{H}_0$ is a finite list of formulas over the current proof language. It contains the theorem hypotheses inherited by every branch and the root counterexample assumptions currently in force: membership in $\mathcal{U}$, the structural hypotheses $\mathcal{H}_{\text{thm}}$, the choice of the object under the root order $\preceq_0$, and target avoidance when the proof is inside a counterexample branch. A later branch may strengthen $\mathcal{H}_0$ only by a recorded lemma; it may not silently replace the theorem hypotheses or the minimality order.

Definition 2.13 (Progress record $\preceq$). The component $\preceq$ is the branch’s well-founded progress record. It contains the root minimality order and any additional loop, refinement, peeling, rank, or finite-enumeration measures active in the branch. Each entry specifies its domain, codomain, strict descent relation, and the lemma proving that the relevant tactic decreases it. Thus $\preceq$ may be a single lexicographic order, a tuple of well-founded measures, or a finite schedule of measures used by nested tactics.

Definition 2.14 (Exclusion register $\mathcal{E}$). The component $\mathcal{E}$ is the ordered register of exits already proved absent in the current branch. An entry has the form

$$(e, \text{condition}(e), \text{certificate}(e), \text{dependencies}(e), \text{consumers}(e)).$$

The condition is the event excluded from the residual class, the certificate is the lemma or computation proving the exclusion, the dependencies identify the branch-state data used, and the consumers are the later dichotomies or comparisons allowed to spend that exclusion. Examples include absent local tests, absent target-complete compressions, absent overloaded fibres, and absent branch exits in a finite table.

Definition 2.15 (Invariant register $\mathcal{I}$). The component $\mathcal{I}$ is the register of structural and quantitative facts available downstream. An invariant entry records its name, statement, proof handle, dependencies, consumers, and, when applicable, its budget effect or slack contribution. Invariants include minimality consequences, forbidden-response statements, density bounds, rank bounds, monotonicity facts, capacity estimates, and exact-type equalities. A tactic may use an invariant only through this register or through a payload that carries it.

Definition 2.16 (Residual condition $\mathcal{R}$). The component $\mathcal{R}$ is the typed condition defining the subcase currently under analysis after the baseline hypotheses, exclusions, and invariants have been imposed. It is a formula or finite conjunction of formulas over the current proof language, together with the data needed to route it: the active piece or class, its interface, its labels or types, its budget state, and its candidate consumer tactics. The residual condition is the part of the branch that changes when a dichotomy takes a non-closing side.

Definition 2.17 (Vocabulary record $\mathcal{V}$). The component $\mathcal{V}$ is the finite typed inventory currently in scope. Each entry records a symbol, its sort in $\mathfrak{L}$, its definition from earlier branch-state data, its well-formedness certificate,

and its permitted consumers. Typical entries are pieces $(X, \partial X, \iota_X)$, outside context classes, local tests $\mathcal{T}$, witness spaces, labels, external types, charge maps, budgets, rank quotients, offset sets, exact response profiles, and black-box theorem handles.

Definition 2.18 (Payload queue $\mathcal{Q}$). The component $\mathcal{Q}$ is a finite queue of typed residual payloads awaiting consumption. A payload entry has the form

$$(P_{i \rightarrow j}, \text{tuple}, \text{producer} = \text{CT}i, \text{consumer} = \text{CT}j, \text{status}).$$

The tuple must match the signature in the typed residual payload table. Its producer discharges S-Rout, and its consumer discharges S-Trig when the payload is consumed. Queue priority, deferral, and empty-queue conditions belong to the runtime policy of Section 3.

Definition 2.19 (Audit record $\mathcal{A}$). The component $\mathcal{A}$ is the proof-engineering record attached to the branch. It contains branch-table rows, dependency-graph entries, reverse-index entries, invariant-register links, slack-table rows, defect-register rows, finite-enumeration certificates, code hashes when computation is used, and resilience-register rows. Each audit entry names the mathematical statement or schema obligation it certifies. A proof step whose result is absent from $\mathcal{A}$ is unavailable to later branches.

Definition 2.20 (Branch state). A **branch state** over a proof instance $\mathfrak{P}$ is a tuple

$$\mathfrak{B} = (\mathcal{H}_0, \preceq, \mathcal{E}, \mathcal{I}, \mathcal{R}, \mathcal{V}, \mathcal{Q}, \mathcal{A}),$$

whose eight components are the objects of the preceding eight definitions. The tuple is the complete local context of a branch. The proof may read only data present in this tuple or imported through a black-box handle recorded in $\mathcal{V}$ and $\mathcal{A}$.

Definition 2.21 (Well-formed branch state). A branch state $\mathfrak{B}$ is **well formed** when every entry of $\mathcal{V}$ is defined from $\mathcal{H}_0, \mathcal{E}, \mathcal{I}$, and $\mathcal{R}$ using constructors admitted by $\mathfrak{L}$; every queued payload in $\mathcal{Q}$ has a declared producer, tuple type, and consumer; and every audit entry in $\mathcal{A}$ names the lemma or schema obligation it certifies. A phrase such as “the remaining case” is a branch state only after it has been converted into such a tuple.

Definition 2.22 (Residual class). The **residual class** of a well-formed branch state is

$$\mathcal{C}(\mathfrak{B}) = \{G \in \mathcal{U} : G \models \mathcal{H}_0, \mathcal{E}, \mathcal{I}, \mathcal{R} \text{ and realizes the recorded data in } \mathcal{V}\}.$$

Equivalently, $\mathcal{C}(\mathfrak{B})$ is the class of ambient objects still compatible with every recorded hypothesis, exclusion, invariant, residual condition, and typed datum of the branch. This class is the statement published by a reduction theorem if the branch has not yet closed.

Definition 2.23 (Branch-state extension). For well-formed branch states $\mathfrak{B}$ and $\mathfrak{B}'$, write $\mathfrak{B}' \succeq \mathfrak{B}$ when $\mathfrak{B}'$ is obtained from $\mathfrak{B}$ by adding certified exclusions, invariants, residual data, vocabulary entries, payloads, or audit rows, and by preserving every component of $\mathfrak{B}$ that remains in scope. Equivalently, $\mathcal{C}(\mathfrak{B}') \subseteq \mathcal{C}(\mathfrak{B})$, with the containment justified by recorded certificates rather than by prose.

Definition 2.24 (Legal residual exit). A non-closing tactic exit from $\text{CT}i$ is a **legal residual exit** when it produces a branch-state extension $\mathfrak{B}' \succeq \mathfrak{B}$ and a typed payload $P_{i \rightarrow j} \in \mathcal{Q}'$ whose tuple entries are objects of the current vocabulary and whose consumer $\text{CT}j$ is declared in the interlock graph. A tactic exit is legal precisely when it returns C1–C5 or emits such a payload; the term *terminal leaf* is reserved for a globally discharged C1–C5 leaf. **Consumed by:** all CT entries.

Example 2.1 (Erdős–Gyárfás instantiation). For the companion Erdős–Gyárfás proof, the root proof instance has the following coordinates.

Ambient class. $\mathcal{U}_{\text{EG}}$ is the class of finite simple graphs. The subobjects used by the proof are subgraphs, induced paths, induced P_{13} -windows, atoms, carrier cores, fan envelopes, and residual supports. Interfaces are boundary vertices, half-edges, degree profiles, stubs, and the declared attachment data through which a piece is glued to an outside context.

Target predicate. Let

$$\text{Pow} = \{2^k : k \geq 2\}, \quad \text{Mers} = \{2^k - 1 : k \geq 2\}.$$

The target predicate is

$$T_{\text{EG}}(G) = 1 \iff G \text{ contains a cycle of length in Pow.}$$

The CT1 target algebra represents this predicate by oriented-edge return tests: a directed edge e and a return path of length in Mers form a power-of-two cycle, and the absence of all such returns is recorded as target avoidance.

Proof language. $\mathfrak{L}_{\text{EG}}$ contains the graph sorts above; oriented edges; return sets $R_e(G)$; Mersenne labels; induced P_{13} -window packings; the P_{13} -free remainder; atoms with boundary profiles; target-response profiles;

exact response profiles; replacement families; Type A and Type B continuation classes; carrier data; fan data; charge ledgers; and the quantitative currencies

$$\begin{aligned} \beta &= m - n + 1, & \lambda &= 2n - 3 - m, & \sigma &= 2m - 3n, \\ \partial_+(X), & W_2(X), & r_\Omega(X), & \eta(R), & N(X). \end{aligned}$$

It also contains the finite labels, payload signatures, black-box theorem handles, interval-arithmetic constants, and code certificates used by the proof.

Baseline hypotheses. $\mathcal{H}_0^{\text{EG}}$ records that $G \in \mathcal{U}_{\text{EG}}$, $\delta(G) \geq 3$, $T_{\text{EG}}(G) = 0$, and G is chosen as a lexicographically minimal counterexample under the order described below. After CT1, $\mathcal{H}_0^{\text{EG}}$ is read together with the target-algebra exclusion that $R_e(G) \cap \text{Mers} = \emptyset$ for every oriented edge e .

Progress record. $\preceq_{\text{EG}}$ begins with lexicographic order on $(|V(G)|, |E(G)|)$. Later branch states append the well-founded measures required by their tactics: routed-load size for peeling, finite survivor-list size for pumping, tier or overlap counts for tiered charging, carrier-count measures in the Type A route-8 branch, and the descent measure for target-defective exit-(4) peeling.

Exclusion register. $\mathcal{E}_{\text{EG}}$ contains the exclusions accumulated along the current path. At the root target-avoiding state these include absence of Mersenne returns. Minimality then adds no proper minimum-degree-three subcounterexample and no target-complete proper replacement. Later branches add, for example, absence of closed Type A exits, absence of available exit-(4) peel data after it has been exhausted, absence of specified Type B fan closures, or absence of finite-table exits in a local germ analysis.

Invariant register. $\mathcal{I}_{\text{EG}}$ contains the target algebra, the no-proper-core minimality invariant, hereditary target-uncompressibility, the P_{13} -existence and P_{13} -density facts, the near-cubic spine when it has been derived, the cycle-rank and sparsity-rank identities, the surplus and deficiency budgets, the curvature target-rank facts, and the Type A/Type B bridge estimates. Each entry records the lemma that establishes it and the later branches that consume it.

Residual condition. $\mathcal{R}_{\text{EG}}$ names the surviving subcase. Near the root it is the target-avoiding minimal-counterexample state. Deeper in the proof

it may be the full-rank pressure branch, the saturated Type A branch, the true route-8 silent trace-basin residual, a Type B bridge residual, or the cold-window residual. In each case the residual condition includes the active support, the relevant interface, the available exits, and the budget or exact-type data needed by the next tactic.

Vocabulary record. $\mathcal{V}_{\text{EG}}$ contains the symbols actually available to the branch: Pow, Mers, return sets $R_e(G)$, the chosen P_{13} -packing and density θ , the remainder R , atoms and boundary profiles, exact response profiles, labels, ledgers, offset sets, rank quotients, budgets, slack constants, and black-box handles such as the P_{13} -free theorem and the finite enumeration/code certificates.

Payload queue. $\mathcal{Q}_{\text{EG}}$ consists of the active typed handoffs. Examples are $P_{1 \rightarrow 2}$ for a proper reducible piece, $P_{1 \rightarrow 17}$ for sparse target arithmetic, $P_{7 \rightarrow 12}$ for a peelable exchange-defect load, $P_{13 \rightarrow 14}$ for overlap mass, and $P_{17 \rightarrow 8}$ for a finite survivor list. A payload is present only after its producer has proved S-Rout; the receiving CT may execute only after S-Trig.

Audit record. $\mathcal{A}_{\text{EG}}$ is the collection of proof-dependency diagrams, branch tables, invariant ledgers, reverse-index rows, slack tables, defect-register rows, and computational-certificate records appearing in [EG] and in Appendix B. These artifacts specify which lemma established each component and which later branches are allowed to consume it.

Thus a typical EG branch state is not the phrase “the Type A residual” but the full tuple

$$\mathfrak{B}_{\text{EG}} = (\mathcal{H}_0^{\text{EG}}, \preceq_{\text{EG}}, \mathcal{E}_{\text{EG}}, \mathcal{I}_{\text{EG}}, \mathcal{R}_{\text{EG}}, \mathcal{V}_{\text{EG}}, \mathcal{Q}_{\text{EG}}, \mathcal{A}_{\text{EG}}),$$

with every displayed component instantiated by named lemmas, tables, or certificates.

2.3 Pieces, interfaces, and outside contexts

Definition 2.25 (Piece, interface, and context package). Fix a well-formed branch state $\mathfrak{B}$. A **piece package** in $\mathfrak{B}$ is a tuple

$$(X, \partial X, \iota_X, \text{Ctx}_{\mathfrak{B}}(\iota_X), \text{Adm}_{\mathfrak{B}}, \preceq_X).$$

Here X is a subobject of the ambient object G ; ∂X is the finite boundary through which X can interact with a declared compatible outside context K ;

ι_X is the typed interface record, including boundary order, ports, colours, orientations, degree data, or any other descriptor admitted by $\mathfrak{L}$; and $\text{Ctx}_{\mathfrak{B}}(\iota_X)$ is the declared class of outside contexts compatible with that interface. The predicate $\text{Adm}_{\mathfrak{B}}(X, \iota_X, K)$ states that a context K may be glued to X along ι_X , and the gluing constructor satisfies

$$\text{Adm}_{\mathfrak{B}}(X, \iota_X, K) \implies K \oplus_{\iota_X} X \in \mathcal{U}.$$

The order $\preceq_X$ is present only when the tactic invokes minimality, replacement, compression, or a well-founded search over pieces.

Definition 2.26 (Well-formed piece package). A piece package is **well formed** when the following data have been certified in the branch state:

- S-Def defines X , ∂X , ι_X , the context class, the admissibility predicate, and the gluing constructor from recorded data;
- the interface size and every interface descriptor lie in a declared bounded sort of $\mathfrak{L}$, unless the current proof language has explicitly admitted a larger interface language;
- compatibility of contexts is decidable from the interface record and the recorded branch state;
- S-Pers records which hypotheses, exclusions, and invariants survive after gluing, deletion, replacement, or compression;
- when $\preceq_X$ is used, S-Meas certifies that it is well founded on the admissible pieces under consideration.

Thus a phrase such as “a reducible subgraph” is not a piece package until its boundary, legal contexts, gluing rule, and persistence data are all recorded.

Consumed by: CT2, CT3, CT7, CT8, CT10, CT11, and CT16.

2.4 The target predicate and local test families

Definition 2.27 (Local test family). Let $T : \mathcal{U} \rightarrow \{0, 1\}$ be the target predicate of the proof instance. A **local test family** for T in a proof language $\mathfrak{L}$ is a declared typed family

$$\mathcal{T} = \{(T_i, W_i, r_i, b_i, \text{Real}_i)\}_{i \in I}.$$

The index object I is an admitted sort of the current proof language. In the base proof language I is finite; in an extended proof language it may be parameterized, provided the branch state records the finite type data used for enumeration, routing, or sparse-target arithmetic. For every $G \in \mathcal{U}$, $W_i(G)$

is the witness space for test T_i ; the quantities r_i and b_i record, respectively, the radius of the data inspected by a witness and the size of the interface exposed by that witness, either as fixed bounds or as declared functions of extension parameters; and $\text{Real}_i(G, w)$ is the predicate that the witness $w \in W_i(G)$ realizes test T_i in G .

Definition 2.28 (Target-test equivalence). A local test family $\mathcal{T}$ **represents** T when the branch state proves the equivalence

$$T(G) = 1 \iff \exists i \in I \exists w \in W_i(G) \text{Real}_i(G, w).$$

The proof of this equivalence is an S-Equiv obligation. Each witness space and realization predicate must be defined by S-Def, and every negative branch must record the absence of all realized tests as an exclusion in $\mathcal{E}$. When the index set, radius, or interface bound is produced by a proof-language extension, that extension is part of the proof instance and is recorded before CT1 executes.

Consumed by: CT1, CT6, CT9, CT11, and CT17.

2.5 External types

Definition 2.29 (External response and external type). Let X be a well-formed piece package with interface ι_X . An **external response map** is a typed map

$$\rho_X : \text{Ctx}_{\mathfrak{B}}(\iota_X) \longrightarrow \text{Resp}_{\mathfrak{B}}$$

whose value records the target-relevant outcome of gluing X to a compatible outside context. The response sort may contain target realization flags, exit flags, compression flags, carrier data, budget effects, or exact closed-type data, provided each coordinate is a sort of the current proof language.

An **external type** of X is a code $\tau(X)$ such that the recorded decoding map determines $\rho_X(K)$ for every compatible context K in the declared context class. A finite external type has values in a finite alphabet. An exact external type may use a larger descriptor admitted by CT16, but its equality predicate and response evaluation must still be computable from recorded branch-state data or a declared black-box handle.

Definition 2.30 (Type equivalence certificate). Two pieces X and Y with the same interface are **externally equivalent** in $\mathfrak{B}$ when

$$\rho_X(K) = \rho_Y(K) \quad \text{for every } K \in \text{Ctx}_{\mathfrak{B}}(\iota_X).$$

The equality $\tau(X) = \tau(Y)$ may be used as external equivalence only after S-Equiv proves that the type decides every target-relevant compatible context. A compression or replacement argument also requires S-Pers for the branch-state data preserved by replacing X with Y , and S-Meas when the replacement is justified by a smaller piece.

Consumed by: CT3, CT7, CT8, and CT16.

2.6 Labels

Definition 2.31 (Label system). A **label system** in a branch state $\mathfrak{B}$ is a tuple

$$(\Lambda, D, \lambda, \Pi_\Lambda).$$

The alphabet Λ is finite. The map $\lambda : D \rightarrow \Lambda$ assigns labels to the objects under consideration, and Π_Λ is the finite list of named projections that later tactics may read from a label. Typical projections record boundary role, local geometry, target offsets, charge role, exit availability, tier membership, rank coordinate, activity status, or a pointer to an exact type.

Definition 2.32 (Legal label). A label assignment is **legal** when S-Def defines the alphabet, domain, assignment map, and projections; S-Det proves that the label of each object is a function of recorded branch-state data; and S-Pers records which label coordinates persist under the transitions that will consume them. A later route may inspect only the projections in Π_Λ , together with any exact type or payload explicitly named by the label. Finite-state arguments additionally require the alphabet Λ to be finite in the sense recorded by the measure system of $\mathfrak{L}$.

Consumed by: CT8, CT9, CT10, CT13, and CT17.

2.7 Ledgers: demands, payers, capacities

Definition 2.33 (Ledger). A **ledger** in a branch state is a tuple

$$(\mathcal{D}, \mathcal{P}, \pi, w, \kappa, \ell).$$

The set $\mathcal{D}$ is the demand set, $\mathcal{P}$ is the payer set, $\pi : \mathcal{D} \rightarrow \mathcal{P}$ is the proposed canonical assignment rule, $w : \mathcal{D} \rightarrow W$ assigns demand weights, $\kappa : \mathcal{P} \rightarrow W$ assigns payer capacities, and $\ell \in W$ is the lower-bound target. The ordered codomain W is a measure codomain in $\mathfrak{L}$; in the unweighted case $W = \mathbb{N}$, $w \equiv 1$, and κ is the usual capacity bound.

Definition 2.34 (Well-formed ledger and ledger closure). A ledger is **well formed** when S-Def defines $\mathcal{D}$, $\mathcal{P}$, w , κ , and ℓ from recorded data; S-Det proves that π is canonical, including all tie-breaks, and total on $\mathcal{D}$; and S-Comp proves the multiplicity or capacity estimates that bound every fibre $\pi^{-1}(p)$. A ledger closes by C4 when the branch state proves

$$\sum_{d \in \mathcal{D}} w(d) \geq \ell \quad \text{and} \quad \ell > \sum_{p \in \mathcal{P}} \kappa(p),$$

or the corresponding finite-additive inequality in the declared codomain W . A non-closing ledger output must specify which demand, payer, tier, or capacity hypothesis failed and route that typed payload by S-Rout.

Consumed by: CT4, CT5, CT6, CT9, CT13, and CT15.

2.8 Budgets and currencies

Definition 2.35 (Currency and budget functional). A **currency** is a branch-state measurable map

$$\gamma : S \longrightarrow W_\gamma$$

from a declared sort S to an ordered codomain W_γ . A **budget functional** is a finite expression in declared currencies, error terms, rates, and slack variables. Its normal form is recorded in $\mathcal{V}$; typical examples have the shape

$$B_\rho(Y) = \text{supply}(Y) - \text{demand}(Y) - \rho \text{ size}(Y) - \text{error}(Y).$$

The parameter ρ may be fixed, interval-valued, or synthesized by the rate-check procedure, provided its admissible range is recorded.

Definition 2.36 (Well-formed budget). A budget is **well formed** when every currency is defined by S-Def; every comparison uses an ordered codomain admitted by $\mathfrak{L}$; every decomposition, gluing, peel, or exchange has a recorded additivity or subadditivity rule; every monotonicity claim is proved by S-Comp; and every slack term is entered in the slack table with its consumers. A budget closes by C4 when the recorded comparisons force an impossible sign, interval, rank, rate, or capacity inequality. A failed comparison is a legal residual only after the failed hypothesis is extracted as a typed payload.

Consumed by: CT4, CT5, CT11, CT14, and CT15.

2.9 Payloads

Definition 2.37 (Payload signature and payload instance). A **payload signature** is an entry

$$P_{i \rightarrow j} : S_1 \times \cdots \times S_m \longrightarrow \text{Payload}_{i \rightarrow j}$$

in the payload component of the proof language. A **payload instance** is a tuple

$$(P_{i \rightarrow j}, \langle x_1, \dots, x_m \rangle, \mathfrak{B}', \text{Trig}_j),$$

where $x_r \in \llbracket S_r \rrbracket_{\mathfrak{B}'}$, $\mathfrak{B}' \succeq \mathfrak{B}$ is the branch-state extension created by the producer, and Trig_j is the consumer trigger condition required by $\text{CT}j$.

Definition 2.38 (Legal payload exit). A non-certificate branch exit from $\text{CT}i$ is complete precisely when it produces a legal payload instance. The producer must prove S-Rout, including the tuple entries, their sorts, the branch-state extension, and the uniqueness of the declared consumer. The consumer may execute only after S-Trig verifies arity, component types, standing hypotheses, interface data, and any certificate handles required by $\text{CT}j$. A residual tuple with no declared consumer is a compile failure, not a terminal branch.

Consumed by: every routed output in Section 2.

2.10 The certificate alphabet C1–C5

Definition 2.39 (Certificate alphabet). The **certificate alphabet** is the terminal return type of the tactic library. A terminal branch is legal exactly when it returns one of the following five certificate classes, together with the audit rows and schema obligations named below.

- C1: direct target certificate.** The branch exhibits a declared local test or target witness and invokes the target-test equivalence from Section 2.4.
- C2: minimality or compression certificate.** The branch constructs an admissible smaller object, replacement, compression, or exact identification; proves preservation of the baseline hypotheses and target response; and invokes the relevant well-founded order.
- C3: routed defect certificate.** The branch enters a defect account already present in the interlock graph. The certificate names the defect payload, its consumer, and the ledger, peel, or repair routine that has accepted responsibility for it.

C4: bounded-capacity certificate. The branch proves an impossible inequality in a ledger, budget, rank, tier, rate, or aggregate comparison using S-Comp and the relevant capacity, additivity, monotonicity, or slack records.

C5: finite verified impossibility certificate. The branch proves that a finite labelled table, exact-type table, survivor list, or enumerated state space contains no admissible survivor. The certificate records the finite alphabet, enumeration procedure, verification theorem, and, when computation is used, the code or proof artifact referenced by the audit record.

Definition 2.40 (Leaf-totality). A completed instantiation is **leaf-total** when every terminal branch returns C1–C5. A residual is legal only as a payload accepted by a downstream tactic. Consequently, a branch ending in a prose description of a remaining case has no corresponding element in the reduction lattice.

Consumed by: every terminal leaf of the master graph.

2.11 Lemma schemas

The CT entries cite a fixed alphabet of lemma schemas. Each schema is a contract between the local proof text and the branch state: the proof may use an object, split, comparison, payload, or loop step only after the corresponding schema has been discharged.

Lemma schema 2.1 (S-Def: definability). **S-Def** $\langle X \rangle$ proves that the object X is defined from the recorded branch state using constructors admitted by the current proof language. The certificate names the source data in $\mathcal{H}_0, \mathcal{E}, \mathcal{I}, \mathcal{R}$, or $\mathcal{V}$, and records every parameter, tie-break, interface, context class, or finite alphabet used in the definition.

Lemma schema 2.2 (S-Equiv: response equivalence). **S-Equiv** $\langle X, Y; \mathcal{K} \rangle$ proves equality of target-relevant response for the declared context class $\mathcal{K}$. For a type $\tau(X)$, it states that the type decides every compatible target-relevant context named by the interface. For a local test family, it states that T holds precisely when one declared local test is realized. For an exchange pair, it states that the finite context generator decides all-context equivalence on the common interface.

Lemma schema 2.3 (S-Dich: exhaustive split). **S-Dich** $\langle \mathcal{P}_1, \dots, \mathcal{P}_m \rangle$ proves that the listed branch alternatives cover the current residual class $\mathcal{C}(\mathfrak{B})$.

Each alternative must be stated as a condition measurable from the branch state, and each surviving alternative must name the certificate or payload constructor it feeds.

Lemma schema 2.4 (S-Pers: persistence). **S-Pers** $\langle \mathfrak{B} \rightarrow \mathfrak{B}' \rangle$ proves that the hypotheses, exclusions, invariants, residual conditions, and vocabulary entries required downstream persist under the transition from $\mathfrak{B}$ to $\mathfrak{B}'$. It also records any strengthened invariant or newly absent exit added to $\mathfrak{B}'$.

Lemma schema 2.5 (S-Det: deterministic choice). **S-Det** $\langle f \rangle$ proves that a map, assignment, representative, payer, label, or tie-break is a function of recorded branch-state data. The schema is required whenever a later argument uses a choice as canonical, charges multiplicity through it, or compares objects selected by it.

Lemma schema 2.6 (S-Rout: payload construction). **S-Rout** $\langle P_{i \rightarrow j} \rangle$ proves that CT i constructs the advertised payload tuple from the current branch state. It records the tuple entries, verifies that each entry belongs to the declared vocabulary, and names CT j as the unique consumer in the interlock graph.

Lemma schema 2.7 (S-Trig: consumer trigger). **S-Trig** $\langle P_{i \rightarrow j}, \text{CT}j \rangle$ proves that the payload $P_{i \rightarrow j}$ satisfies the trigger signature of CT j . It checks arity, component types, standing hypotheses, interface data, and any required certificate handles before the consumer tactic begins.

Lemma schema 2.8 (S-Comp: comparison). **S-Comp** $\langle \Phi \rangle$ proves the advertised numerical, finite, order, capacity, rank, or arithmetic comparison Φ . Its conclusion must be one of the certificate inequalities or finite impossibility claims listed in C1–C5, or a comparison statement used to construct a declared residual payload.

Lemma schema 2.9 (S-Rest: loop restoration). **S-Rest** $\langle \mathfrak{B}_k \rightarrow \mathfrak{B}_{k+1} \rangle$ proves that a loop or peeling pass returns to the tactic’s required branch-state signature. All baseline hypotheses, admissibility conditions, invariants, ledgers, and payload handles needed for the next pass must be present in $\mathfrak{B}_{k+1}$.

Lemma schema 2.10 (S-Meas: well-founded descent). **S-Meas** $\langle M \rangle$ proves that the measure used by a loop, peel, refinement, or recursion is defined from recorded data, takes values in a well-founded ordered set, and decreases strictly on each non-closing pass. Together with S-Rest, it certifies termination of the iterative tactic.

An unfilled or invalid schema is an execution error and is written to the defect register. A failure becomes a design residual only after it has been constructed as a typed obstruction accepted by a declared consumer. It becomes a scope obstruction only after the language-extension audit of Section 4.5 establishes that the required schema cannot be expressed in the current proof language.

2.12 Notation and diagram legend

Solid arrows in a tactic diagram are intended intra-tactic proof steps, including the construction of an intended residual payload. Dotted arrows are checks or post-residual handoffs and require S-Rout at the source plus S-Trig at the target. Dashed arrows are repairable design-failure routes. Red-dashed arrows mark candidate scope obstructions referred to the global audit in Section 4.9. Green ovals are certificate sinks C1–C5, thick-bordered nodes are completed payload constructors, dotted-bordered nodes are checks or consumer tactics, and dashed-bordered nodes are recovery objects. Execution errors are audit rejects rather than diagram branches. Per-invocation records use $CTk\text{-}\#$: CT9-1 is one worksheet instance of CT9, while CT9 is the problem-independent tactic.

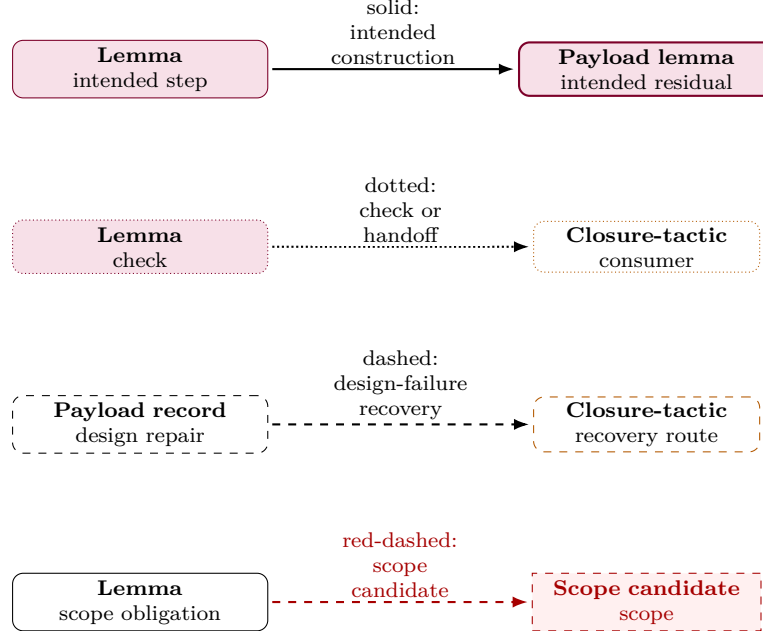


Figure 1: Line grammar legend for the closure-tactic diagrams.

2.13 Terminological conventions

The vocabulary of this document deliberately parallels standard technique names, so that a first-time reader can anchor each concept to something familiar:

- **charging schemes** generalize the charging and discharging arguments of extremal graph theory and computational geometry [???];
- **local exchanges** generalize the exchange and switching arguments of extremal combinatorics;
- **external types** are modeled on Myhill–Nerode-style context equivalence and on types in logic: two pieces have the same external type when no admissible surrounding context can tell them apart [????];
- the **finite-state pumping principle** is a finite-state pigeonhole argument built from context equivalence [??];
- **compression** and **replacement** are used in the same spirit as protrusion replacement in kernelization [????].

Matching, star, pigeonhole, fibre, minor, and immersion retain their standard meanings. Each method-specific term is defined in bold at first occurrence and repeated in the glossary (Appendix A).

Throughout, G denotes the ambient object under analysis (a graph, colouring, hypergraph, matroid, etc., depending on the problem). A **piece** is a subobject of G considered together with a controlled interface; a tactic may impose an additional cardinality bound. Formally, it is a **boundaried piece** $(X, \partial X, \text{interface data})$, where ∂X is the boundary along which X attaches to the rest of G . An **attachment point** is a designated vertex or element of that boundary. A piece is **proper** if it is not all of G . The distinct notions of residual condition, residual class, and payload instance are defined in Section 2.2 and Section 2.9.

Failure-classification convention. The entry boundary separates three audit outcomes. Execution errors (E), such as an unfilled S-Equiv or an unrecorded handoff, are audit rejects written to the defect register. Repairable design failures (D), such as a missing finite label or an admissibility gap, are recovery-output fields. A local nonexistence is a candidate scope issue for the current tactic, not a terminal proof leaf. It becomes a scope obstruction (S) only after the global scope audit has established that no admitted tactic or finite descriptor or proof-language extension can encode the residual. It is terminal only after the defect register has ruled out a level-4 language extension.

Certificate recap. The success leaves use the five certificate classes defined in Section 2.10.

Library closure claim. For every tactic CT_k , each non-certificate output is a named payload accepted by the trigger of another CT entry, and the master interlock graph contains the corresponding edge. The only terminal success classes are C1–C5. Candidate scope obstructions are not residual outputs or immediate terminals; they are referred to the global audit in Section 4.9.

Invocation contract. Given an admitted CT invocation whose trigger and parameters are recorded, execute its listed procedure, discharge its obligation schemas, and return exactly one declared certificate or typed residual payload on each resulting leaf. Proposal, admission, selection, repetition, and composition are governed by Section 3.

Typed residual payloads. Each intended or recovery inter-tactic edge below is a typed handoff. A label $P_{i \rightarrow j}$ means that CT_i constructs the displayed tuple and that the trigger of CT_j accepts exactly that tuple. Intended payloads are the ordinary forward residuals in the tactic graph; recovery payloads are dashed design-failure routes in the CT diagrams. Concatenating two diagrams is therefore a type check: the producer must discharge S-Rout for the tuple, and the receiving tactic must discharge S-Trig for the same tuple. The CT entries use the following payloads.

Payload	Route class	To	Typed tuple	Description
$P_{1 \rightarrow 2}$	Intended	CT2	$\langle \mathfrak{B}_{-T}, X, \partial X, \iota_X, \preceq_0 \rangle$	Target-avoiding branch state, proper admissible piece, certified interface, and root order.
$P_{1 \rightarrow 3}$	Intended	CT3	$\langle \mathfrak{B}_{-T}, X, \partial X, \iota_X, \tau\text{-slot, exit flags} \rangle$	Boundary-response datum ready for external typing.
$P_{1 \rightarrow 4}$	Intended	CT4	$\langle \mathfrak{B}_{-T}, \mathcal{D}, \mathcal{P}, \pi, C, \ell \rangle$	Demand set, payer set, canonical assignment rule, capacity bound, and lower-bound target.
$P_{1 \rightarrow 5}$	Intended	CT5	$\langle \mathfrak{B}_{-T}, \mathcal{S}, \lambda_S, w, \mathcal{L} \rangle$	Linear local-site family, site labels, witness extractor, and locality record.
$P_{1 \rightarrow 6}$	Intended	CT6	$\langle \mathfrak{B}_{-T}, \mathcal{A}, \theta, \text{activity threshold, first-failure extractor} \rangle$	Active/dormant setup generated after target avoidance.

Payload	Route class	To	Typed tuple	Description
$P_{1 \rightarrow 17}$	Intended	CT17	$\langle \mathfrak{B}_{-T}, S, \Delta, \Omega, \text{sparse target arithmetic} \rangle$	Sparse target set, bounded offset family, and compatible completion data.
$P_{2 \rightarrow 3}$	Recovery	CT3	$\langle \mathfrak{B}, X, \partial X, \iota_X, P_{\text{ctx}} \sqcup P_{\text{resp}} \rangle$	Separating context for the selected replacement, or a response defect exposed by the target-uncompressible survivor; the corresponding candidate or survivor state is retained in the payload.
$P_{2 \rightarrow 10}$	Recovery	CT10	$\langle \mathfrak{B}, X, S_{\text{surv}}, d_{\text{crit}} \rangle$	Target-uncompressible survivor state, including its deletion-critical and replacement-exhaustion proofs, together with the criticality datum required for refinement.
$P_{3 \rightarrow 7}$	Intended	CT7	$\langle \mathfrak{B}, X, X', \iota, C_{\text{sep}}, \text{response pair} \rangle$	Separating-context defect/exchange datum.
$P_{3 \rightarrow 12}$	Intended	CT12	$\langle \mathfrak{B}, L, \mu, \text{defect account} \rangle$	Routed load with well-founded peel measure.
$P_{3 \rightarrow 8}$	Intended	CT8	$\langle \mathfrak{B}, (X_0, \dots, X_m), \tau, \text{finite labels} \rangle$	Long repeated-type sequence.
$P_{4 \rightarrow 9}$	Intended	CT9	$\langle \mathfrak{B}, \mathcal{D}, \mathcal{P}, \pi, C, p, \pi^{-1}(p), \text{labels} \rangle$	Overcharged payer fibre.
$P_{4 \rightarrow 14}$	Intended	CT14	$\langle \mathfrak{B}, \mathcal{R}, \ell, \text{capacity record} \rangle$	Aggregate residual class with carried mass.
$P_{4 \rightarrow 13}$	Recovery	CT13	$\langle \mathfrak{B}, \mathcal{D}_{\text{tier}}, A, O, \text{tier-interaction record} \rangle$	Demand with no available payer, promoted to a tiered availability obstruction.
$P_{5 \rightarrow 4}$	Intended	CT4	$\langle \mathfrak{B}, \mathcal{D}_S, \mathcal{P}_S, \pi_S, C_S, \ell_S \rangle$	Charge map assembled from local witnesses.
$P_{5 \rightarrow 11}$	Intended	CT11	$\langle \mathfrak{B}, R, B(\cdot), \text{admissible decomposition} \rangle$	Additive global-deficit residual.
$P_{5 \rightarrow 14}$	Intended	CT14	$\langle \mathfrak{B}, \mathcal{R}, \ell_{\mathcal{R}}, \text{aggregate capacity record} \rangle$	Local-mass shortfall residual.
$P_{5 \rightarrow C4}$	Cert.	C4	C4 certificate	Summed lower bound, capacity upper bound, and positive slack.
$P_{6 \rightarrow 4}$	Intended	CT4	$\langle \mathfrak{B}, \mathcal{D}_{\text{act}}, \mathcal{P}_{\text{act}}, \pi_{\text{act}}, C_{\text{act}}, \ell_{\text{act}} \rangle$	Active-test demand ledger.
$P_{6 \rightarrow 7}$	Intended	CT7	$\langle \mathfrak{B}, A, A', \iota, \delta, \text{first-failure witness} \rangle$	Same-interface witness produced by dormancy.
$P_{6 \rightarrow 3}$	Intended	CT3	$\langle \mathfrak{B}, X, \partial X, \iota_X, \tau\text{-slot}, \text{witness flags} \rangle$	Boundary-response witness from the first failure.
$P_{6 \rightarrow 9}$	Intended	CT9	$\langle \mathfrak{B}, \mathcal{D}, \mathcal{P}, \pi, C, p, F_p, \text{saturation label} \rangle$	Saturated payer or overload witness.
$P_{6 \rightarrow 10}$	Recovery	CT10	$\langle \mathfrak{B}, \text{first-failure witness}, \text{missing refinement datum} \rangle$	Bounded first-failure witness that appears only after refinement.

Payload	Route class	To	Typed tuple	Description
$P_{7 \rightarrow 3}$	Intended	CT3	$\langle \mathfrak{B}, A, A', \iota, C_{\text{sep}}, \text{response pair} \rangle$	Distinguishing exchange as an external-type defect.
$P_{7 \rightarrow 12}$	Intended	CT12	$\langle \mathfrak{B}, L, \mu, \text{exchange-defect account} \rangle$	Distinguishing exchange already packaged as peelable load.
$P_{7 \rightarrow 10}$	Recovery	CT10	$\langle \mathfrak{B}, A, A', \iota, \text{neutral label} \rangle$	Neutral noncompressive exchange needing refinement.
$P_{7 \rightarrow 16}$	Recovery	CT16	$\langle \mathfrak{B}, Z = G, \tau_{\text{closed}}, \text{neutral closed data} \rangle$	Neutral datum whose support is the whole object.
$P_{8 \rightarrow 3}$	Intended	CT3	$\langle \mathfrak{B}, S_i, S_j, C_{\text{sep}}, \tau_i, \tau_j \rangle$	Separating context for repeated states.
$P_{8 \rightarrow 7}$	Intended	CT7	$\langle \mathfrak{B}, A, A', \iota, \delta, \text{chain exchange} \rangle$	Exchange extracted from a long chain.
$P_{8 \rightarrow 10}$	Recovery	CT10	$\langle \mathfrak{B}, (S_0, \dots, S_m)_i, \text{coarse finite labels, missing refinement datum} \rangle$	Finite label space requiring a certified refinement before pumping.
$P_{9 \rightarrow 4}$	Intended	CT4	$\langle \mathfrak{B}, \mathcal{D}, \mathcal{P}, \pi, C, \text{bounded-fibre proof} \rangle$	Bounded multiplicity returned to charging.
$P_{9 \rightarrow 7}$	Intended	CT7	$\langle \mathfrak{B}, A, A', \iota, \delta, \text{homogeneous fibre witness} \rangle$	Same-interface object extracted from overload.
$P_{9 \rightarrow 8}$	Intended	CT8	$\langle \mathfrak{B}, (S_0, \dots, S_m), \tau, \text{homogeneous chain labels} \rangle$	Long chain extracted from overload.
$P_{9 \rightarrow 10}$	Recovery	CT10	$\langle \mathfrak{B}, F_p, \text{failed finite label design, refinement datum} \rangle$	Label refinement needed for the overloaded fibre.
$P_{10 \rightarrow 7}$	Intended	CT7	$\langle \mathfrak{B}, C_\ell, A, A', \iota, \text{class exchange} \rangle$	Bounded exchange inside a refined label class.
$P_{10 \rightarrow 3}$	Intended	CT3	$\langle \mathfrak{B}, C_\ell, X, \partial X, \iota_X, \tau\text{-slot} \rangle$	Promoted response/type datum.
$P_{10 \rightarrow 15}$	Intended	CT15	$\langle \mathfrak{B}, C_\ell, r, \text{rank datum} \rangle$	Promoted rank datum for forcing.
$P_{11 \rightarrow 10}$	Recovery	CT10	$\langle \mathfrak{B}, X, \text{admissibility gap, finite local type slot} \rangle$	Localized piece outside the admissible toolbox.
$P_{11 \rightarrow 1}$	Intended	CT1	$\langle \mathfrak{B}, X, T, \mathcal{T}, \text{localized target witness} \rangle$	Localized deficit piece carrying target-test data.
$P_{11 \rightarrow 7}$	Intended	CT7	$\langle \mathfrak{B}, A, A', \iota, \delta, \text{localized exchange} \rangle$	Localized piece carrying a bounded exchange.
$P_{11 \rightarrow 14}$	Intended	CT14	$\langle \mathfrak{B}, \mathcal{R}, \ell_{\mathcal{R}}, \text{localized carried mass} \rangle$	Localized deficit converted to aggregate mass.
$P_{12 \rightarrow 4}$	Recovery	CT4	$\langle \mathfrak{B}, \mathcal{D}_{\text{new}}, \mathcal{P}_{\text{new}}, \pi_{\text{new}}, C_{\text{new}}, \ell_{\text{new}} \rangle$	Restored state with an ordinary new demand.
$P_{12 \rightarrow 13}$	Recovery	CT13	$\langle \mathfrak{B}, \mathcal{D}_{\text{tier}}, A, O, \text{tier-interaction record} \rangle$	Restored state whose new demand requires tiered charging.
$P_{13 \rightarrow 4}$	Intended	CT4	$\langle \mathfrak{B}, \mathcal{D}, \mathcal{P}_1, \pi_1, C_1, \ell_1 \rangle$	Tier-one charge account.
$P_{13 \rightarrow 9}$	Intended	CT9	$\langle \mathfrak{B}, p, F_p, \text{overlap obstruction labels} \rangle$	Overlap obstruction as a fibre overload.
$P_{13 \rightarrow 14}$	Intended	CT14	$\langle \mathfrak{B}, \mathcal{R}, \ell_{\mathcal{R}}, \text{overlap mass record} \rangle$	Overlap obstruction as aggregate mass.
$P_{14 \rightarrow 9}$	Recovery	CT9	$\langle \mathfrak{B}, u, \text{unbounded capacity member, multiplicity gap} \rangle$	Aggregate multiplicity failure routed to overload.

Payload	Route class	To	Typed tuple	Description
$P_{14 \rightarrow 10}$	Recovery	CT10	$\langle \mathfrak{B}, \mathcal{C}, \text{finite missing capacity datum, refinement slot} \rangle$	Aggregate capacity record requiring a finite pointwise refinement.
$P_{15 \rightarrow 3}$	Intended	CT3	$\langle \mathfrak{B}, X, \partial X, \iota_X, \tau\text{-slot, rank defect} \rangle$	Rank-drop dependence as type defect.
$P_{15 \rightarrow 7}$	Intended	CT7	$\langle \mathfrak{B}, A, A', \iota, \delta, \text{rank exchange} \rangle$	Rank-drop dependence as exchange or compression data.
$P_{15 \rightarrow 16}$	Intended	CT16	$\langle \mathfrak{B}, Z = G, \tau_{\text{closed}}, \text{rank delocalization} \rangle$	Rank-drop dependence that is whole-object.
$P_{15 \rightarrow 4}$	Intended	CT4	$\langle \mathfrak{B}, \mathcal{D}_{\text{rank}}, \mathcal{P}_{\text{rank}}, \pi_{\text{rank}}, C_{\text{rank}}, \ell_{\text{rank}} \rangle$	Full-rank demand ledger.
$P_{16 \rightarrow 3}$	Intended	CT3	$\langle \mathfrak{B}, X, \partial X, \iota_X, \tau(X), \text{proper subpiece from the whole-object split} \rangle$	Proper subpiece from the whole-object dichotomy.
$P_{16 \rightarrow 10}$	Intended	CT10	$\langle \mathfrak{B}, Z = G, \tau_{\text{closed}}^{(1)}, \tau_{\text{closed}}^{(2)}, \text{distinct data} \rangle$	Distinct closed-type residual.
$P_{17 \rightarrow 3}$	Recovery	CT3	$\langle \mathfrak{B}, \mathcal{K}, \iota, \tau\text{-slot, incompatible completion class} \rangle$	Completion class separated by type.
$P_{17 \rightarrow 10}$	Recovery	CT10	$\langle \mathfrak{B}, \mathcal{K}, \ell, \text{incompatible completion class} \rangle$	Completion class separated by label.
$P_{17 \rightarrow 8}$	Intended	CT8	$\langle \mathfrak{B}, (S_0, \dots, S_m), \tau, \text{finite survivor offsets} \rangle$	Finite survivor list for pumping/exhaustion.
$P_{17 \rightarrow 14}$	Intended	CT14	$\langle \mathfrak{B}, \mathcal{R}, \ell_{\mathcal{R}}, \text{arithmetic residual mass} \rangle$	Target-thickening residual mass.

2.14 The master interlock graph

The following figures are part of the tactic library in Section 2. The first is the expanded interlock map; the second suppresses closure sinks and scope boundaries to show only inter-tactic residual routing.

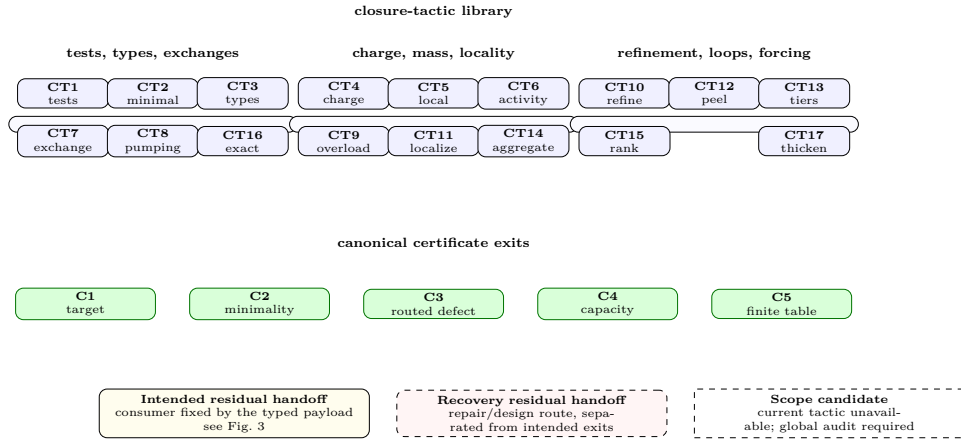


Figure 2: Closure-tactic interlock diagram. The blue chips are the seventeen closure tactics grouped by their role in the library. Every admitted CT execution exits through one of the displayed interfaces: a canonical certificate C1–C5, an intended residual handoff, a recovery residual handoff, or a candidate obstruction referred to the global scope audit in Section 4.9. The split inventory and Fig. 3 give the payload names and consumers; following a residual handoff requires S-Rout at the producer and S-Trig at the named consumer.

CT1-start typed-payload flow. The next figure records the tactics reachable from CT1 by declared typed payloads. Each CT appears once. An arrow $P_{i \rightarrow j}$ means that CT i has constructed the exact payload accepted by CT j . Each directed path carries its own cumulative branch-state extension; outgoing edges are alternative continuations, and a cycle returns with newly recorded data or progress. CT6 and CT17 are reachable from CT1 through the activity-setup payload $P_{1 \rightarrow 6}$ and the sparse-target offset payload $P_{1 \rightarrow 17}$.

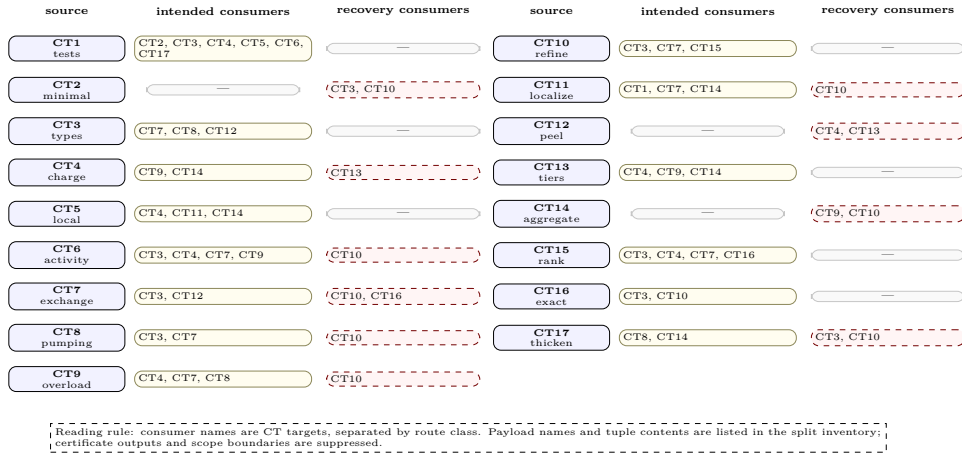


Figure 3: Closure-tactic routing graph. Each row lists one source tactic, its intended residual consumers, and its recovery-route consumers in separate columns. The typed payload names and tuple contents are listed in the split inventory before Fig. 2; following any row entry requires S-Rout at the source and S-Trig at the named target. Closure outputs and candidate obstructions referred to the global scope audit in Section 4.9 are suppressed here so that only inter-tactic dependencies remain.

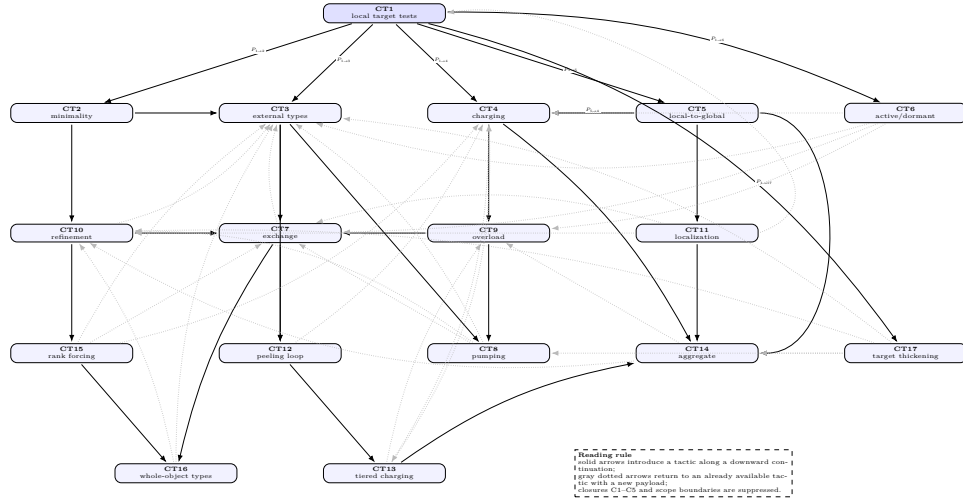


Figure 4: CT1-start typed-payload flow. The figure shows the CT tactics reachable from CT1 by declared payloads in the tactic-library table of Section 2, with each CT represented by a single node. Direct CT1→CT4 is present because $P_{1 \rightarrow 4}$ is already a complete charging payload; CT5→CT4 is the distinct route in which local witnesses assemble the charging payload $P_{5 \rightarrow 4}$. CT6 is reachable through $P_{1 \rightarrow 6}$, the activity-setup payload, and CT17 is reachable through $P_{1 \rightarrow 17}$, the sparse-target offset payload. Gray dotted arrows are re-entry edges to tactics already present in the cumulative flow; their payload names are listed in the typed-payload table.

Sequential inventory. The table below is the controlling description of the tactic library. The home column points to the detailed entry and its diagram; the CT entries are indexed here in numerical order, and each caption is part of the corresponding tactic specification.

ID	Tactic	Home and diagram	Certificates	Intended residuals	Recovery residuals
CT1	Target-test alignment	Section 2.15, Fig. 5	C1	$P_{1 \rightarrow 2}, P_{1 \rightarrow 3}, P_{1 \rightarrow 4}, P_{1 \rightarrow 5}, P_{1 \rightarrow 6}, P_{1 \rightarrow 17}$	—
CT2	Minimality and replacement	Section 2.16, Fig. 6	C2	—	$P_{2 \rightarrow 3}, P_{2 \rightarrow 10}$
CT3	External-type compression	Section 2.17, Fig. 7	C2, C3, C5	$P_{3 \rightarrow 7}, P_{3 \rightarrow 12}, P_{3 \rightarrow 8}$	—
CT4	Charging schemes	Section 2.18, Fig. 8	C4	$P_{4 \rightarrow 9}, P_{4 \rightarrow 14}$	$P_{4 \rightarrow 13}$
CT5	Local-to-global bookkeeping	Section 2.19, Fig. 9	C4	$P_{5 \rightarrow 4}, P_{5 \rightarrow 11}, P_{5 \rightarrow 14}$	—
CT6	Active/dormant dichotomy	Section 2.20, Fig. 10	C1	$P_{6 \rightarrow 4}, P_{6 \rightarrow 3}, P_{6 \rightarrow 7}, P_{6 \rightarrow 9}$	$P_{6 \rightarrow 10}$
CT7	Exchange trichotomy	Section 2.21, Fig. 11	C1, C2, C3	$P_{7 \rightarrow 3}, P_{7 \rightarrow 12}$	$P_{7 \rightarrow 10}, P_{7 \rightarrow 16}$
CT8	Finite-state pumping	Section 2.22, Fig. 12	C2, C5	$P_{8 \rightarrow 3}, P_{8 \rightarrow 7}$	$P_{8 \rightarrow 10}$
CT9	Overload exhaustion	Section 2.23, Fig. 13	C1	$P_{9 \rightarrow 4}, P_{9 \rightarrow 7}, P_{9 \rightarrow 8}$	$P_{9 \rightarrow 10}$
CT10	Default refinement	Section 2.24, Fig. 14	C5	$P_{10 \rightarrow 3}, P_{10 \rightarrow 7}, P_{10 \rightarrow 15}$	—
CT11	Localization	Section 2.25, Fig. 15	—	$P_{11 \rightarrow 1}, P_{11 \rightarrow 7}, P_{11 \rightarrow 14}$	$P_{11 \rightarrow 10}$
CT12	Peeling loop	Section 2.26, Fig. 16	C4	—	$P_{12 \rightarrow 4}, P_{12 \rightarrow 13}$
CT13	Tiered charging	Section 2.27, Fig. 17	C4	$P_{13 \rightarrow 4}, P_{13 \rightarrow 9}, P_{13 \rightarrow 14}$	—
CT14	Aggregate closure	Section 2.28, Fig. 18	C4	—	$P_{14 \rightarrow 9}, P_{14 \rightarrow 10}$
CT15	Rank forcing	Section 2.29, Fig. 19	C4	$P_{15 \rightarrow 3}, P_{15 \rightarrow 4}, P_{15 \rightarrow 7}, P_{15 \rightarrow 16}$	—
CT16	Whole-object exact types	Section 2.30, Fig. 20	C2	$P_{16 \rightarrow 3}, P_{16 \rightarrow 10}$	—
CT17	Target thickening	Section 2.31, Fig. 21	C1, C5	$P_{17 \rightarrow 8}, P_{17 \rightarrow 14}$	$P_{17 \rightarrow 3}, P_{17 \rightarrow 10}$

Mechanism-entry ledger. The next table is the per-entry audit ledger. It does not replace the figures; it makes the fields needed for format compliance explicit in a single lookup location. The procedure field is deliberately schematic: the numbered mathematical objects and arrows are the corresponding diagram.

ID	Purpose	Trigger	Parameters	Audit hooks	Candidate scope obstruction
CT1	Execute scope, equivalence, and realization, and payload nodes in baseline proof order.	Validated ambient object, branch state, and baseline proof.	Static target-test framework, four node plans, Port, and HandoffPlan.	Reject missing S-Equiv before admission; retain the exact avoiding state in every payload.	The declared test language is unavailable.

ID	Purpose	Trigger	Parameters	Audit hooks	Candidate scope obstruction
CT2	Execute the interface, deletion, candidate, context, and survivor nodes in order.	Proper admissible piece in a minimal counterexample.	Size order, admissibility class, response profile, replacement family, and five node plans.	Reject incomplete predecessor states; retain edge evidence and certify each terminal handoff.	No bounded interface exists.
CT3	Compare external types and route defects, tables, or peelable load.	Certified interface and type slot recorded.	Context universe, type evaluator, replacement table.	Reject unproved S-Equiv; write type-table or defect-account row.	No finite type index exists.
CT4	Turn a demand/payer ledger into C4 or a typed capacity residual.	Demand set, payer set, assignment rule, capacity, and lower bound recorded.	Canonical assignment rule, capacity proof, lower-bound estimate.	Reject non-deterministic assignment; write ledger, capacity, or missing-tier rows.	No locally generated demand exists.
CT5	Sum local witnesses into a global charge, budget, or localized residual.	Linear local-site family with witness extractor recorded.	Site labels, local lower bound, summation domain.	Reject missing locality lemma; write summation and slack rows.	No bounded local witness exists.
CT6	Split active tests from first-failure and dormant structure.	Activity threshold and first-failure extractor recorded.	Activity predicate, threshold, first-failure order.	Reject unrecorded threshold data; write active ledger or witness row.	No bounded first-failure witness exists.
CT7	Classify a bounded exchange as realizing, distinguishing, or neutral.	Unclassified exchange pair on a certified common interface.	Context generator, response evaluator, size order, table handle.	Reject neutral claim with S-Equiv unfilled; write exchange classification row.	No finite context generator exists.
CT8	Pump a bounded sequence through finite labels or exact types.	Bounded sequence with finite label/type slot recorded.	Label alphabet, equality test, finite threshold.	Reject informal periodicity; write repeat, table, or label-refinement row.	No finite type space exists.
CT9	Exhaust an overloaded payer fibre by homogeneous extraction.	Payer fibre and capacity bound recorded.	Label map, fibre capacity, extraction target.	Reject unlabelled fibre; write overload and extraction row.	No finite labelling exists.

ID	Purpose	Trigger	Parameters	Audit hooks	Candidate scope obstruction
CT10	Promote a missing finite datum to a refinement label.	Residual class exposing a finite missing datum.	Candidate datum, refined label alphabet, class table.	Reject datum not extracted from classification; write refinement row.	No finite refinement alphabet exists.
CT11	Localize an additive global deficit to an admissible piece.	Additive budget deficit and admissible decomposition recorded.	Budget, admissibility class, decomposition lemma.	Reject nonadditive budget; write localization or admissibility-gap row.	No additive bounded local type exists.
CT12	Peel routed load by a well-founded loop.	Routed load account and measure recorded.	Peel move, restoration map, measure.	Reject failed S-Rest or S-Meas; write per-pass and termination rows.	No measurable load exists.
CT13	Split a failed charge into tiered accounts or overlap residuals.	Tier-one account or obstruction record present.	Tier split, availability test, reconciliation rule.	Reject unrecorded tier overlap; write tier ledger and overlap row.	No measurable tier resource exists.
CT14	Close a residual class by linear mass versus sublinear capacity.	Residual class with lower mass and candidate capacity record.	Class mass lower bound, member capacity, aggregate upper bound.	Reject unbounded multiplicity; write aggregate, overload, or finite-refinement row.	No bounded capacity description exists.
CT15	Charge only target-independent rank coordinates.	Target-relative rank datum recorded.	Rank map, dependence classifier, full-rank ledger.	Reject incidental rank; write rank-drop or full-rank row.	No finite target-relative rank exists.
CT16	Resolve whole-object cases by exact closed types.	Delocalized whole-object datum and computable closed type recorded.	Closed-type evaluator, equality test, proper-piece test.	Reject empty-context equivalence; write exact-type row.	No finitely computable closed type exists.
CT17	Thicken sparse targets by bounded offsets and arithmetic residuals.	Sparse target set and bounded offset family recorded.	Offset set, compatibility classes, survivor list.	Reject incompatible offsets without class record; write offset and survivor rows.	No bounded offset family exists.

Signature and guarantee ledger. The complete entry for CT k is the union of the mechanism-entry row, the signature-and-guarantee row below,

and the diagram. Scheduler data are a separate policy view consumed by Section 3. This table records the template fields whose contents are easiest to audit as formal arrows: Input, Obligations, Certificate, intended outputs, and recovery outputs. The Procedure field is the corresponding diagram read from top to bottom. Every intended non-terminal arrow listed here must appear in that diagram and in the master interlock graph; every recovery arrow must appear as a dashed design-failure route in the corresponding diagram.

ID	Input	Obligations	Certificate	Intended outputs	Recovery outputs
CT1	$\text{Input}(F) = \langle G, \mathfrak{B}, \text{Baseline}(G) \rangle$	S-Def; S-Equiv($T, \mathcal{T}$); S-Dich (scope, realization, payload); S-Pers(T -avoidance); S-Rout; S-Trig.	C1, realized local test.	Indexed target-avoiding payloads to CT2, CT3, CT4, CT5, CT6, CT17.	Typed scope terminal.
CT2	$P_{1 \rightarrow 2}$	S-Def($X, \iota_X, \prec$); S-Dich (scope, delete, certify replacement, classify survivor); S-Comp($\prec$); S-Equiv; S-Rout; S-Trig.	C2, minimality contradiction.	—	Separating-context or response-defect residual to CT3; criticality-refinement datum to CT10.
CT3	$P_{* \rightarrow 3}$	S-Def(ι, τ); S-Equiv(τ -responses); S-Pers; S-Rout; S-Trig; S-Comp.	C2, C3, or C5.	Exchange defect to CT7; peelable load to CT12; long type sequence to CT8.	—
CT4	$P_{* \rightarrow 4}$	S-Def($\mathcal{D}, \mathcal{P}, \pi$); S-Det(π); S-Dich (gap, overload); S-Comp; S-Rout; S-Trig.	C4, bounded-capacity contradiction.	Overcharged fibre to CT9; aggregate mass residual to CT14.	Missing payer tier to CT13.
CT5	$P_{1 \rightarrow 5}$	S-Def(S, w); S-Det(site assignment); S-Comp(summation); S-Rout; S-Trig.	C4, local-to-global comparison.	Charge ledger to CT4; global deficit to CT11; aggregate residual to CT14.	—
CT6	$P_{1 \rightarrow 6}$	S-Def(activity threshold); S-Dich (active, dormant); S-Rout; S-Trig; S-Equiv(first failure).	C1, first-failure target hit.	Active ledger to CT4; boundary/type witness to CT3; exchange witness to CT7; saturation witness to CT9.	Refinement-gated witness to CT10.
CT7	$P_{* \rightarrow 7}$	S-Def(exchange pair); S-Equiv(context generator); S-Dich (realizing, distinguishing, neutral); S-Comp; S-Rout; S-Trig.	C1, C2, or C3.	External-type defect to CT3; peelable defect to CT12.	Neutral label to CT10; whole-object datum to CT16.
CT8	$P_{* \rightarrow 8}$	S-Def (bounded sequence, type); S-Dich (repeat, short); S-Equiv(type equality); S-Comp; S-Rout; S-Trig.	C2 or C5.	Separating type to CT3; chain exchange to CT7.	Coarse finite label space to CT10.
CT9	$P_{* \rightarrow 9}$	S-Def(F_p); S-Dich (bounded, overloaded); S-Comp(pigeonhole); S-Rout; S-Trig.	C1, target-hit overload.	Bounded multiplicity to CT4; homogeneous exchange to CT7; long chain to CT8.	Label-refinement residual to CT10.
CT10	$P_{* \rightarrow 10}$	S-Def(refined label); S-Dich(label classes); S-Comp(finite table); S-Rout; S-Trig.	C5, finite class exhaustion.	Promoted response to CT3; class exchange to CT7; rank datum to CT15.	—

ID	Input	Obligations	Certificate	Intended outputs	Recovery outputs
CT11	$P_5 \rightarrow_{11}$	S-Def (budget, decomposition); S-Comp(localization); S-Rout; S-Trig.	None; route-only tactic.	Local target witness to CT1; localized exchange to CT7; carried mass to CT14.	Admissibility gap to CT10.
CT12	$P_* \rightarrow_{12}$	S-Def(L, μ); S-Rest; S-Meas; S-Comp(termination); S-Rout; S-Trig.	C4, loop discharge.	—	Restored ordinary demand to CT4; tiered demand to CT13.
CT13	$P_* \rightarrow_{13}$	S-Def(tiers); S-Det(tier assignment); S-Dich (availability, obstruction); S-Comp; S-Rout; S-Trig.	C4, reconciled tier capacity.	Tier-one account to CT4; overlap fibre to CT9; overlap mass to CT14.	—
CT14	$P_* \rightarrow_{14}$	S-Def(C); S-Comp(lower mass); S-Comp(upper capacity); S-Rout; S-Trig.	C4, aggregate capacity contradiction.	—	Unbounded member or multiplicity gap to CT9; finite missing capacity datum to CT10.
CT15	$P_* \rightarrow_{15}$	S-Def(rank); S-Dich (rank drop, full rank); S-Comp(rank budget); S-Rout; S-Trig.	C4, rank-budget contradiction.	Rank defect to CT3; rank exchange to CT7; whole-object rank datum to CT16; rank ledger to CT4.	—
CT16	$P_* \rightarrow_{16}$	S-Def(closed type); S-Equiv(closed response); S-Dich (proper, whole); S-Comp; S-Rout; S-Trig.	C2, exact whole-object identification.	Proper subpiece to CT3; distinct closed data to CT10.	—
CT17	$P_1 \rightarrow_{17}$	S-Def(S, Δ, Ω); S-Dich(scale split); S-Comp(arithmetic); S-Rout; S-Trig.	C1 or C5.	Finite survivors to CT8; arithmetic mass to CT14.	Completion class to CT3 or CT10.

Scheduler data. The following data are published by the reference manual but are not rules of execution. Section 3 consumes these fields when it decides which tactic to admit or select. The cost classes are: **check** for steps whose parameters are already recorded and require no parameter synthesis, **synth** for tactics requiring manufactured parameters, and **loop** for tactics carrying a well-founded measure. Producer fields name CTs except for CT1’s unique root input.

ID	Cost	Intended producers	Recovery producers	Intended consumers	Recovery consumers	Formal counterpart / [EG] anchor
CT1	check	root input, CT11	—	CT2, CT3, CT4, CT5, CT6, CT17	—	four proof-producing node plans; [EG] target tests
CT2	check	CT1	—	—	CT3, CT10	minimality replacement lemma; [EG] criticality
CT3	synth	CT1, CT6, CT7, CT8, CT10, CT15, CT16	CT2, CT17	CT7, CT8, CT12	—	finite type comparison; [EG] response profiles
CT4	synth	CT1, CT5, CT6, CT9, CT13, CT15	CT12	CT9, CT14	CT13	charge-map capacity proof; [EG] ledgers
CT5	synth	CT1	—	CT4, CT11, CT14	—	local summation proof; [EG] window and stub accounts
CT6	synth	CT1	—	CT3, CT4, CT7, CT9	CT10	active/dormant classifier; [EG] hot/cold interface

ID	Cost	Intended producers	Recovery producers	Intended consumers	Recovery consumers	Formal counterpart / [EG] anchor
CT7	synth	CT3, CT6, CT8, CT9, CT10, CT11, CT15	—	CT3, CT12	CT10, CT16	decidable exchange classifier; [EG] germ routing
CT8	synth	CT3, CT9, CT17	—	CT3, CT7	CT10	finite-state/pumping table; [EG] trace basin
CT9	check	CT4, CT6, CT13	CT14	CT4, CT7, CT8	CT10	pigeonhole over finite fibres; [EG] overload extraction
CT10	synth	CT16	CT2, CT6, CT7, CT8, CT9, CT11, CT14, CT17	CT3, CT7, CT15	—	finite label refinement; [EG] label classes
CT11	synth	CT5	—	CT1, CT7, CT14	CT10	additive localization lemma; [EG] net-charge localization
CT12	loop	CT3, CT7	—	—	CT4, CT13	well-founded recursion; [EG] routed-load peeling
CT13	synth	—	CT4, CT12	CT4, CT9, CT14	—	tiered discharging proof; [EG] type-B ledgers
CT14	synth	CT4, CT5, CT11, CT13, CT17	—	—	CT9, CT10	aggregate capacity comparison; [EG] bridge residuals
CT15	synth	CT10	—	CT3, CT4, CT7, CT16	—	rank independence proof; [EG] curvature rank
CT16	synth	CT15	CT7	CT3, CT10	—	exact closed-type computation; [EG] whole-graph profiles
CT17	synth	CT1	—	CT8, CT14	CT3, CT10	bounded-offset arithmetic; [EG] sparse target thickening

Schema coverage by tactic. The detailed diagrams instantiate these schemas as named mathematical objects. This table records the obligation vocabulary that must be discharged when each tactic is used.

ID	Primary schemas
CT1	S-Def, S-Equiv, S-Dich, S-Pers, S-Rout, S-Trig.
CT2	S-Def, S-Dich, S-Comp, S-Equiv, S-Rout, S-Trig.
CT3	S-Def, S-Equiv, S-Pers, S-Rout, S-Trig.
CT4	S-Def, S-Det, S-Dich, S-Comp, S-Rout, S-Trig.
CT5	S-Def, S-Det, S-Comp, S-Rout, S-Trig.
CT6	S-Def, S-Dich, S-Rout, S-Trig.
CT7	S-Def, S-Equiv, S-Dich, S-Comp, S-Rout, S-Trig.
CT8	S-Def, S-Dich, S-Equiv, S-Comp, S-Rout, S-Trig.
CT9	S-Def, S-Dich, S-Comp, S-Rout, S-Trig.
CT10	S-Def, S-Dich, S-Rout, S-Trig.
CT11	S-Def, S-Comp, S-Rout, S-Trig.
CT12	S-Def, S-Dich, S-Pers, S-Rest, S-Meas, S-Rout, S-Trig.
CT13	S-Def, S-Det, S-Dich, S-Comp, S-Rout, S-Trig.
CT14	S-Def, S-Comp, S-Rout, S-Trig.
CT15	S-Def, S-Dich, S-Comp, S-Rout, S-Trig.
CT16	S-Def, S-Equiv, S-Dich, S-Rout, S-Trig.
CT17	S-Def, S-Dich, S-Comp, S-Rout, S-Trig.

2.15 CT1 — Target-test alignment and first closure

Signature. $\langle \mathfrak{B}, T, \mathcal{T}, W \rangle \rightarrow \text{Cert}\{C1\}; \text{Int}\{P_{1 \rightarrow 2}, P_{1 \rightarrow 3}, P_{1 \rightarrow 4}, P_{1 \rightarrow 5}, P_{1 \rightarrow 6}, P_{1 \rightarrow 17}\}; \text{Rec}\{\emptyset\}.$

CT1 is the entry machine for target detection. It consumes a validated ambient object and branch state, certifies the chosen target-test equivalence, and then returns exactly one indexed terminal. A realized test closes by C1. A negative realization produces a target-avoiding state that is retained by whichever downstream payload is selected.

Fires when. The invocation supplies **Input**: an ambient object G , its branch state, and the baseline proof for G . The static target predicate, test-index type, witness family, realization predicate, and scope predicate belong to **Framework** and remain outside the runtime graph. The CT2, CT3, CT4, and CT5 exits carry exact consumer inputs with alignment proofs; only the not-yet-formalized CT6 and CT17 exits retain abstract datum types.

You supply. Supply four independent node plans. The scope plan decides whether the declared test language is available. The equivalence plan proves both directions between $T(G)$ and realization of a declared test. The realization plan returns either a concrete realization or its exhaustive absence. The payload plan classifies the resulting target-avoiding state into one of six consumer payloads. Supply **Port** and **HandoffPlan** separately to prove downstream acceptance.

Execution. The machine is sequential. **Scope.Decision.exit** produces the typed scope terminal, while **Scope.Decision.ready** produces the only state admitted by the equivalence node. The next node constructs an **EquivalenceState**. This certification boundary has one successor. Admission of a **CorePlan** requires the certificate; an unsuccessful construction returns the target-test design to the pre-admission repair stage.

The realization node then returns **Decision.hit** with a **C1Certificate**, or **Decision.avoiding** with an **AvoidingState**. The latter stores the equivalence certificate, the refutation of every **TestRealization**, and the derived proof $\neg T(G)$. No later node reconstructs these facts. Finally, **Payload.Decision** has six constructors, one for each consumer. Thus payload construction and route selection form one typed decision, and every non-closing run ends immediately at its selected terminal.

Formal machine interface. `Framework` is parameterized by the exact `CT2.Framework` used at the $P_{1 \rightarrow 2}$ terminal. Consequently the ambient type and target predicate of `CT1` and `CT2` agree definitionally. The adapter `CT2Payload.toInput` fixes the downstream ambient object to `input.G`, combines the input baseline proof with `AvoidingState.targetAvoiding`, and requires precisely the remaining minimality, properness, and admissibility fields of `CT2.Input`. The `CT3`, `CT4`, and `CT5` payloads contain actual `Input` values for their embedded consumer frameworks. Their `toInput` adapters expose those values, and `CT3Aligned`, `CT4Aligned`, and `CT5Aligned` record the application-specific relation to the `CT1` ambient object and branch state. Thus no formalized consumer handoff is an untyped datum slot.

Each semantic node has its own module and contract. `CorePlan` contains exactly `Scope.Plan`, `Equivalence.Plan`, `Realization.Plan`, and `Payload.Plan`. Each constructor of `Graph.Edge` stores the evidence returned by its source node, and the indices of `Graph.Path` enforce exact edge composition. `runCore` returns a terminal-indexed `RawOutcome`; `certify` adds only the relevant consumer-acceptance proof; and `runTraced` returns an `ExecutionResult` whose path and `Outcome` share one terminal index. The term form `ct1_execute` invokes the interpreter. The tactics `ct1` and `ct1_total` apply `run_verified` and `run_total`. The totality theorem quantifies over the four supplied proof-producing plans and the handoff plan.

Soundness obligations.

- **S-Def** fixes the target, test indices, witness family, realization predicate, branch state, exact `CT2`–`CT5` consumer frameworks, alignment predicates, and the remaining `CT6` and `CT17` datum families in `Framework` and `Input`.
- **S-Equiv** is the pair of explicit maps $\text{TestRealization} \rightarrow T(G)$ and $T(G) \rightarrow \text{Nonempty}(\text{TestRealization})$.
- **S-Dich** is instantiated separately by scope admission, realization, and the six-way payload classifier; constructors and outgoing edges agree bijectively.
- **S-Pers** is enforced by indexing every routed payload by the exact `AvoidingState` returned by realization.
- **S-Rout** is instantiated by the six constructors of `HandoffPayload`; the consumer tag cannot disagree with the payload type.
- **S-Trig** is discharged only by `HandoffPlan.accept`, after the mathematical core run has selected its terminal.

Routing. The C1 terminal carries the equivalence state and realized test. Otherwise CT1 emits an indexed target-avoiding payload: a proper admissible piece and minimality eliminator to CT2, an aligned CT3 input, an aligned CT4 charge ledger, an aligned CT5 local-site input, an activity setup to CT6, or sparse-offset data to CT17. A run selects one constructor; any priority among several constructible plans is fixed outside the machine by Section 3.14.

Limit. Failure of scope admission is a typed terminal referred to the global audit in Section 4.9. A missing target-test equivalence prevents construction of the equivalence plan and records a CT1/S-Equiv design defect. The reverse implication of the equivalence certificate is the sole derivation of target avoidance.



Figure 5: Closure-tactic 1 as an exact sequential machine. The monospaced labels are the stable semantic identifiers shared by the JSON graph and Lean’s `Graph.NodeId`. Static target-test vocabulary is shown separately from the runtime path. Scope admission returns either a typed scope result or the state consumed by equivalence certification. Machine admission requires the equivalence certificate; incomplete certification returns to the pre-admission repair stage. Realization either supplies the C1 certificate or one target-avoiding state retained by the six-way payload classifier. Every non-closing edge ends at an indexed, consumer-certified terminal.

Local target tests in practice

Start with a **target predicate** T : the property whose existence the theorem asserts. Depending on the problem, the target may be the existence of a prescribed substructure, a monochromatic configuration, a flow, colouring, matching, or minor, or a moment-accessible spectral consequence such as a walk-count or girth condition. A spectral-gap target requires a cut or energy residual interface rather than only bounded-radius forbidden-pattern tests; see Section 4.9.

CT1 represents the target predicate by a family of **local tests**

$$\mathcal{T} = \{T_i\}_{i \in I} \quad \text{such that} \quad \text{target exists} \iff \exists i \in I \text{ with } T_i \text{ realized,}$$

where a test is **realized** if some configuration in G satisfies it. The tests must be local enough to interact with small pieces, yet complete enough that target avoidance is expressible as a conjunction of local exclusions.

The semantic machine for CT1 is Fig. 5; its typed states make the equivalence, closure, and six residual exits explicit.

2.16 CT2 — Minimality and replacement

Signature. $P_{1 \rightarrow 2} \rightarrow \text{Cert}\{C2\}; \text{Int}\{\emptyset\}; \text{Rec}\{P_{2 \rightarrow 3}, P_{2 \rightarrow 10}\}.$

You typically reach CT2 from CT1 after target avoidance exposes a proper piece that may be deleted or replaced. The tactic is the disciplined use of minimality: it closes legal reductions and routes the survivor when minimality leaves a target-uncompressible object.

Fires when. The entry state contains a minimal counterexample, a proper admissible piece with interface data, and a declared size order. These data form a single typed invocation. Interface boundedness is the first machine decision; it is not part of the entry hypothesis.

You supply. You supply the size order, the admissibility class, and the replacement family. These come from the resource inventory and from the interface vocabulary data; the replacement family must be compatible with the recorded outside contexts. You also supply five independent node interfaces: the interface audit, deletion analysis, replacement-candidate

search, candidate-specific context certification, and survivor classification. Consumer adapters certify the two outgoing payload types separately from these core decisions.

Execution. The machine is sequential. First audit the interface. An unbounded interface produces the scope exit; a bounded interface enables deletion analysis. A legal smaller deletion closes by C2, while its exhaustive refutation produces the deletion-critical state. Replacement-candidate search then returns either one concrete smaller compatible candidate or an exhaustive proof that no such candidate exists. Exhaustive absence produces the target-uncompressible state and enables the final survivor classifier. A concrete candidate instead enters its own context-certification node, which returns either the complete context-safe replacement certificate or an explicit separating context. The certified alternative closes by C2; the separating alternative produces the CT3 context-residual exit. The survivor classifier returns either the CT10 criticality-refinement exit or the CT3 response-defect exit.

Replacement certification is candidate-specific. The search result carries the concrete replacement, its local legality and interface conditions, and strict decrease. The following node alone is responsible for certifying the reverse response-profile implication and preservation of counterexample status. Its residual alternative records an actual compatible outside context and the response discrepancy that refutes context safety. Static definitions such as the response profile are resources of these interfaces rather than runtime states.

Formal machine interface. The Lean reference assigns one independently compiled module to each decision. `Interface.Decision` returns a `ScopeCandidate` or `BoundedState`; `Deletion.Decision` returns a `DeletionWitness` or `DeletionCriticalState`; `ReplacementCandidate.Decision` returns a `CandidateState` or `SurvivorState`; `Context.Decision` returns a `CandidateContextCertificate` or `ContextCT3Payload`; and `Survivor.Decision` returns a `CriticalityCT10Payload` or `ResponseCT3Payload`. The entry module has only the validated `Input` contract and its identity runner. A `CorePlan` contains exactly the five decision plans. The downstream `Port` and `HandoffPlan` are separate, so consumer acceptance cannot enter a core node contract.

Both CT3 variants contain an actual `CT3.Input` for the exact `Framework.ct3`

consumer. The context variant also carries `ContextCT3Aligned`; the survivor-response variant carries `ResponseCT3Aligned`. The shared `CT3Payload.toInput` adapter exposes either validated consumer input without erasing which residual produced it.

Each constructor of `Graph.Edge` stores the evidence returned by its source node, and the indices of `Graph.Path` enforce exact edge composition. `runCore` returns a terminal-indexed `RawOutcome`; `certify` adds only the appropriate handoff proof; and `runTraced` returns an `ExecutionResult` whose path and `Outcome` share the same terminal index. The two C2 origins are distinct constructors of `C2Certificate`. The term form `ct2_execute` invokes this interpreter, while the goal-closing tactics `ct2` and `ct2_total` apply `run_verified` and `run_total`. Here totality is relative to the supplied proof-producing node plans; it is not a claim that the required witnesses can always be synthesized automatically.

Soundness obligations.

- **S-Def** is instantiated with the proper piece X , its interface ι_X , the admissibility class, and the minimality order $\prec$.
- **S-Dich** is instantiated node by node: bounded versus scope, deletion witness versus deletion-critical state, candidate versus exhaustive absence, certified candidate versus separating context, and criticality datum versus response defect.
- **S-Comp** is instantiated with the proof that every legal deletion or replacement produces an object strictly smaller in the order $\prec$.
- **S-Equiv** is instantiated candidate by candidate: a replacement closes only after its interface compatibility and context safety have been certified; failure returns an explicit separating context.
- **S-Rout** is instantiated with the context-residual and survivor-response variants of $P_{2 \rightarrow 3}$, and with $P_{2 \rightarrow 10}$ for a survivor carrying criticality-refinement data.
- **S-Trig** is instantiated with the two corresponding CT3 triggers and the CT10 criticality trigger.

Routing. Legal deletion or replacement closes by C2. A context-dependent response emits $P_{2 \rightarrow 3}$ to CT3. A target-uncompressible survivor with criticality-refinement data emits $P_{2 \rightarrow 10}$ to CT10; a surviving response defect emits $P_{2 \rightarrow 3}$ to CT3. Each handoff is a normal typed terminal result and carries its consumer-admission proof. Each CT3 terminal additionally carries

an aligned, validated CT3 input.

Limit. CT2 produces a scope result when the required interface is unbounded. A replacement can reach C2 only through the candidate-specific context certificate. Missing interface data is a CT2/S-Def defect; a candidate without the required context certificate is a CT2/S-Equiv defect.

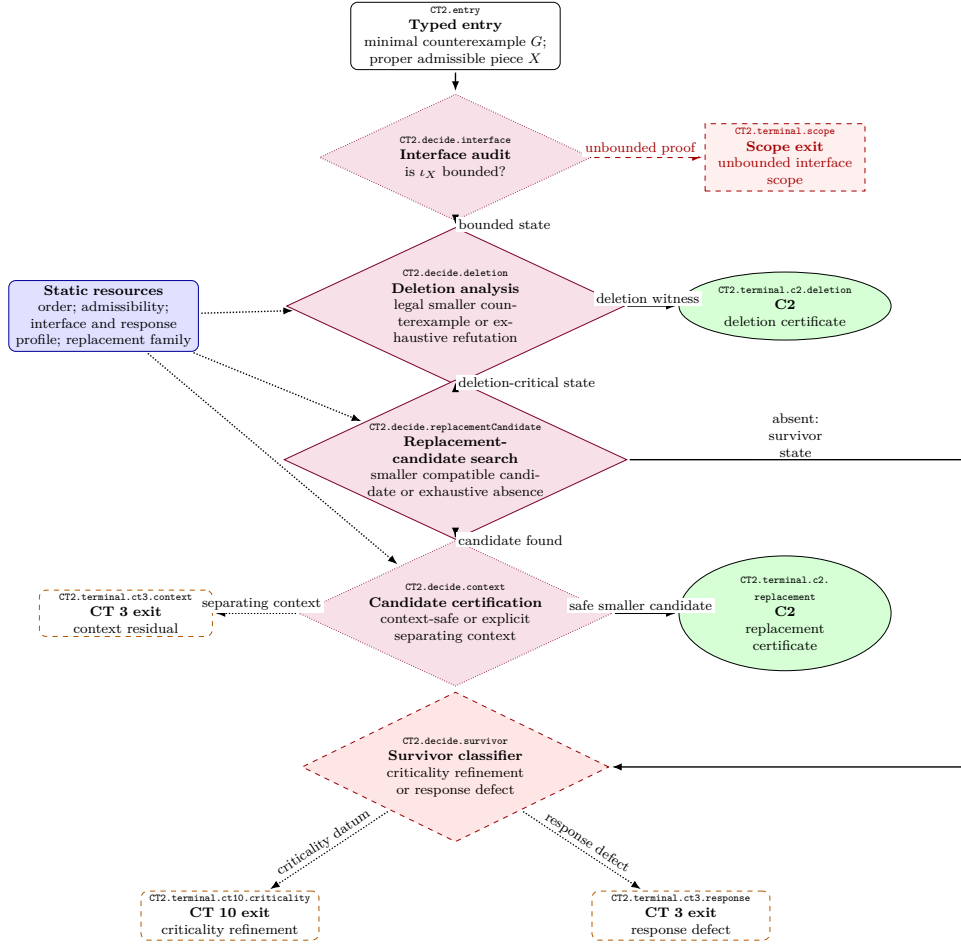


Figure 6: Closure-tactic 2 (minimality and replacement) as an exact sequential machine. The monospaced labels are the stable semantic identifiers shared by the JSON graph and Lean’s `Graph.NodeId`. The entry state packages the minimal counterexample and admissible piece. Each decision consumes one typed predecessor state and returns evidence for exactly one outgoing edge. Candidate-specific replacement certification either closes by minimality or produces an explicit separating context for CT3; exhaustive absence produces the target-uncompressible state consumed by the survivor classifier. The CT3 and CT10 handoffs are ordinary certified terminal results. Static definitions and proof resources are displayed separately because they are not runtime states. An unbounded interface produces the scope result referred to the global audit in Section 4.9.

Minimality and replacement in practice

Assume a **minimal counterexample** G , ordered by a lexicographic size parameter

$$(|G|, \text{secondary complexity}, \dots).$$

Minimality supplies two families of tools.

Deletion/criticality tools:

if deleting or simplifying X preserves the hypotheses
and avoids the target, contradiction.

Replacement/compression tools:

if a proper piece X admits a smaller
target-equivalent representative, contradiction.

The second family is the engine behind every “periodicity implies reducibility” step. A periodic structure contributes to the proof when it yields a smaller representative that is target-equivalent against *all* compatible outside contexts. Here an **outside context** is any completion of the rest of G that is compatible with the interface data of the piece under discussion. This is the same conceptual role played by representative replacement in the finite-integer-index and protrusion-replacement literature [????].

The low-level diagram for CT2 is Fig. 6; it records the deletion, replacement, context-validity, and residual routing obligations used by the minimal-counterexample argument.

2.17 CT3 — External-type compression

Signature. $P_{* \rightarrow 3} \rightarrow \text{Cert}\{C2, C3, C5\}; \text{Int}\{P_{3 \rightarrow 7}, P_{3 \rightarrow 12}, P_{3 \rightarrow 8}\}; \text{Rec}\{\emptyset\}.$

CT3 is an exact sequential machine for external-type compression. Its **Framework** fixes the ambient and branch-state types, boundaried pieces, interfaces, compatible contexts, response semantics, the external-type evaluator, type inclusion, replacement predicates, persistence, closure claims, and downstream datum types. A validated **Input** contains $\langle G, \mathfrak{B}, X \rangle$ and

a certified interface for X . Finiteness, semantic equivalence, replacement exhaustion, and persistence are produced by separate nodes.

The core plan has five independent fields: `scope`, `equivalence`, `compression`, `defect`, and `table`. Each field implements one node contract and depends only on its immediate predecessor state. The scope node returns either `ScopeCandidate` or `ScopedState`. The equivalence node has one successor and constructs `EquivalenceState`; its `EquivalenceCertificate` proves, for every compatible context, that the stored external type decides the response of X . Failure to construct this certificate is a pre-admission proof obligation, not a runtime alternative.

The compression node consumes that exact equivalence state. A `ReplacementWitness` contains an admissible smaller representative, type inclusion, persistence of the branch condition under replacement, and the resulting C2 claim. The positive constructor returns a `C2Certificate`; the negative constructor returns an `UncompressibleState` carrying an exhaustive proof that no such witness exists. The defect node then returns exactly one of four indexed results: a C3 certificate, a CT7 exchange payload, a CT12 peeling payload, or a `PersistentState`. Only the last result reaches the finite-table node, which returns a C5 certificate or a CT8 repeated-type payload.

Every payload is indexed by the exact uncompressible or persistent state that justified it. `Port` and `HandoffPlan` certify consumer acceptance after `runCore`; they cannot change the selected terminal or its evidence. The theorems `run_verified`, `run_total`, and `run_trace_valid` establish terminal soundness, totality relative to the five supplied plans, and validity of the evidence-carrying path.

The soundness contracts are assigned to machine boundaries as follows.

- **S-Def** fixes the interface, response evaluator, external type, replacement vocabulary, and downstream datum types in `Framework` and `Input`.
- **S-Equiv** is discharged by `EquivalenceCertificate` before replacement comparison.
- **S-Dich** is discharged by the constructors of the scope, compression, defect, and table decisions.
- **S-Pers** is carried by `PersistentState` and by every payload index.
- **S-Comp** is carried by replacement exhaustion and finite-table classification.

- **S-Rout** and **S-Trig** are discharged by the three handoff constructors and the separate consumer plan.

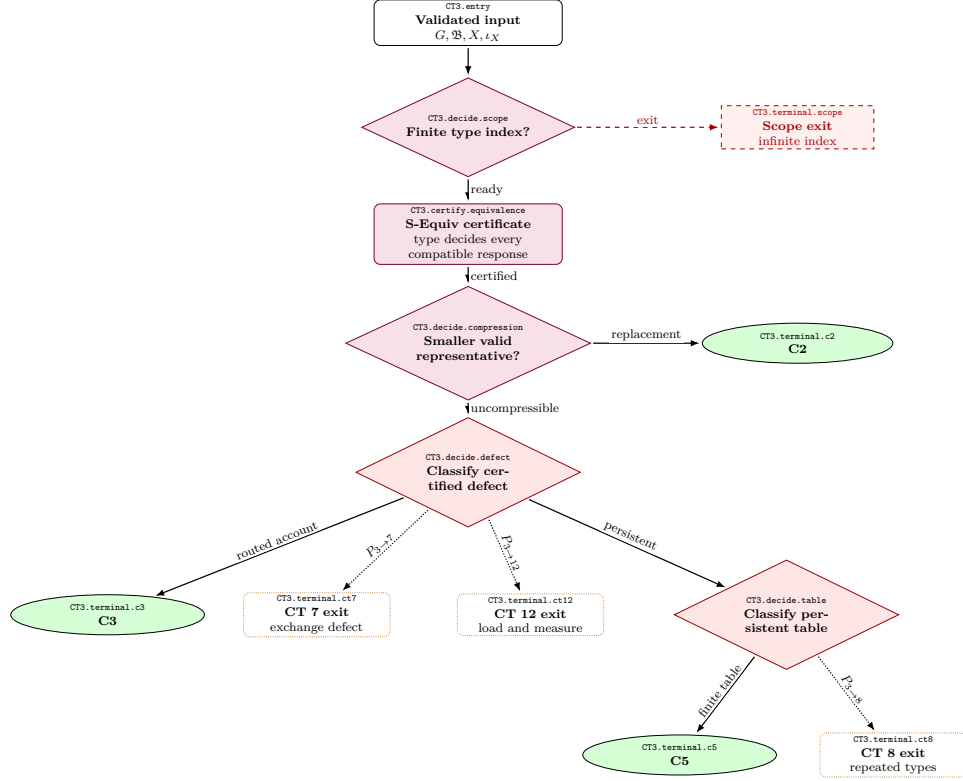


Figure 7: Closure-tactic 3 as an exact sequential machine. Monospaced labels are the stable identifiers shared by Lean’s `Graph.NodeId`, the JSON contract graph, and executable traces. Certification nodes have one successor and represent proof obligations required before later decisions. Every branching edge carries the indexed certificate, state, or payload named by its constructor. The three handoff terminals retain their complete provenance and are accepted separately by `HandoffPlan`.

External-type compression in practice

A piece X is analyzed as a boundaried piece $(X, \partial X, \text{interface data})$. To each piece one attaches its **external type**

$$\tau(X) = \text{all target-relevant answers that } X \\ \text{can exhibit to compatible outside contexts.}$$

Two pieces have the same external type when no admissible context can distinguish them with respect to the target. This is a Myhill–Nerode-style context equivalence, analogous to a type in logic and to the graph-theoretic analogue of the context-equivalence viewpoint in Courcelle-style recognizability and boundaried-graph replacement [?????].

An external type typically records boundary degrees, attachment order, local labels, and the responses of the local tests restricted to X . It also records **exit flags**, compression flags, and carrier data. An exit flag is a bit recording which named closure routes of Section 3.4 remain available in the current branch state. A **carrier** is an auxiliary bounded substructure, canonically associated to X , that carries interface and target data between distant pieces and to which charges may later be routed.

A smaller piece X' may replace X only if

$$\tau(X') \subseteq \tau(X)$$

(or with equality, depending on the replacement lemma in force). This inclusion is what converts a local reduction into a globally valid one.

The low-level diagram for CT3 is Fig. 7; it spells out the external-type definition, replacement test, separating-context branch, and typed exits.

2.18 CT4 — Charging schemes

Signature. $P_{* \rightarrow 4} \rightarrow \text{Cert}\{C4\}; \text{Int}\{P_{4 \rightarrow 9}, P_{4 \rightarrow 14}\}; \text{Rec}\{P_{4 \rightarrow 13}\}.$

CT4 is an exact sequential machine for a deterministic demand–payer ledger. Its **Framework** fixes dependent demand and payer types, eligibility, canonicity, fibre capacity, the global demand–capacity comparison, the C4 claim, and downstream datum types. A validated **Input** contains $\langle G, \mathfrak{B}, \pi, C, \ell \rangle$, where

π is an explicit partial assignment function. Scope readiness, canonicity, totality, bounded fibres, and the final comparison are distinct contracts.

The core plan has five independent fields: `scope`, `assignment`, `availability`, `fibres`, and `comparison`. The assignment node is a single-successor certification boundary. It constructs `AssignmentState`, whose `AssignmentCertificate` proves that the supplied function implements the declared deterministic tie-breaking rule. A missing proof of canonicity rejects the plan before execution; it does not create an untyped recovery edge.

The availability node consumes the exact assignment state. It either builds a `CT13Payload` containing a concrete unassigned demand and proof that no eligible payer exists, or constructs `TotalAssignmentState`, which proves that every demand is assigned to an eligible payer. The fibre node is indexed by this totality proof and returns either an overload payload for CT9 or `BoundedFibreState`. The comparison node is indexed by the bounded state and returns a C4 certificate or a CT14 aggregate residual. Thus determinism, payer availability, no-overcount, and the numerical comparison can be implemented and verified independently.

`Port` and `HandoffPlan` contain only consumer-side acceptance proofs for CT13, CT9, and CT14. They are applied after `runCore` and cannot alter the mathematical result. Theorems `run_verified`, `run_total`, and `run_trace_valid` establish soundness, relative totality, and exact trace validity.

The soundness contracts are assigned as follows.

- **S-Def** fixes the demand family, payer family, assignment, capacity datum, and lower-bound datum.
- **S-Det** is discharged only by `AssignmentCertificate`.
- **S-Dich** is discharged by the scope, availability, fibre, and comparison constructors.
- **S-Comp** is carried by `BoundedFibreState` and `C4Certificate`.
- **S-Rout** and **S-Trig** are discharged by the indexed CT13, CT9, and CT14 payloads and the separate handoff plan.

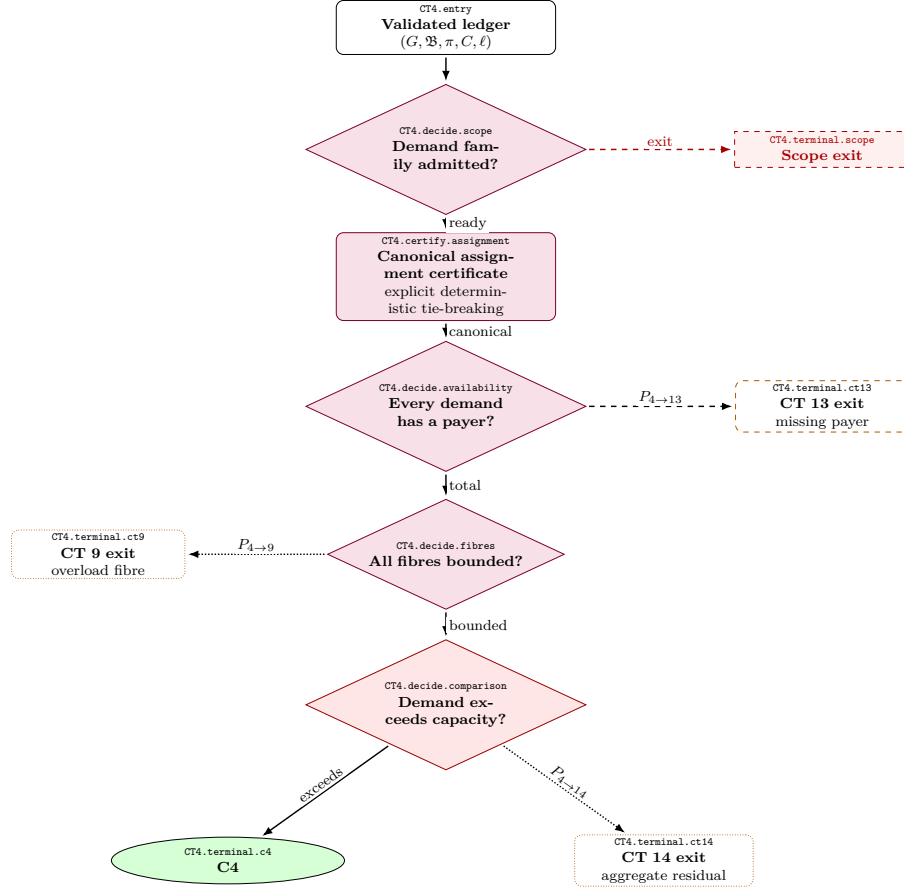


Figure 8: Closure-tactic 4 as an exact sequential machine. The assignment certificate, totality state, and bounded-fibre state are separate proof boundaries. Each later node is indexed by the exact earlier proof, so an overload, missing-payer witness, C4 certificate, or aggregate residual cannot be attached to another ledger. Monospaced identifiers are shared by Lean, JSON, and executable traces.

Charging schemes in practice

A **charging scheme** is an injective or bounded-multiplicity assignment from **demands** — the events that must be accounted for — to **payers** — the scarce structures that absorb the charges. A payer that has absorbed its full capacity is **saturated**. This is the same mechanism as the classical charging and discharging arguments, packaged with an explicit interface. The reference class is the discharging/reducibility architecture of the four-colour theorem and its later expositions [????].

A charging scheme has four parts.

1. **Demand definition.** This specifies what must be paid for. Typical demands are bad **windows**, blocked pairs, missing attachment points, **route entries**, saturated payers, and first-failure witnesses (Section 2.20). A *window* is a bounded-size region of G , fixed by the local structure of the branch, inside which the local tests are evaluated; it is *bad* if the required tests fail there. A route entry is a record deposited in the charging scheme of a previously named branch; see “routed” in Section 3.4.
2. **Canonical assignment.** Each demand selects one canonical payer under an explicit deterministic tie-break. This makes the ledger auditable. The bounded-multiplicity lemma, not determinism, controls how many demands may select the same payer.
3. **Bounded-multiplicity lemma** (the *no-overcount* lemma). Each payer is charged at most C times — or else an already-routed branch occurs (Section 2.23).
4. **Capacity inequality.**

$$|\mathcal{D}| \leq C |\mathcal{P}| + o(n).$$

When the proof requires a contradiction, the capacity inequality is combined with a demand lower bound $|\mathcal{D}| \geq cn - o(n)$; the contradiction $cn \leq C|\mathcal{P}| + o(n)$ closes the branch whenever the payers are known to number $o(n)$ or to lie below threshold.

The low-level diagram for CT4 is Fig. 8; it shows where the demand, payer, canonical map, bounded-fibre lemma, capacity comparison, and routed residual payloads enter.

2.19 CT5 — Local-to-global bookkeeping

Signature. $P_{1 \rightarrow 5} \rightarrow \text{Cert}\{C4\}; \text{Int}\{P_{5 \rightarrow 4}, P_{5 \rightarrow 11}, P_{5 \rightarrow 14}\}; \text{Rec}\{\emptyset\}.$

CT5 is an exact sequential machine for converting a local witness family into a global comparison. Its **Framework** contains the complete CT4 framework together with dependent site and local-witness types, the witness extractor predicates, locality, ledger readiness, summation, and the CT11 and CT14 datum types. A validated **Input** contains $\langle G, \mathfrak{B}, w \rangle$, where w is the explicit dependent witness extractor. The CT4 framework is embedded so that the CT5–CT4 handoff has a definitionally fixed consumer vocabulary.

The core plan has five independent fields: **scope**, **locality**, **deficit**, **summation**, and **comparison**. Scope admission returns either a typed obstruction or **ScopedState**. The locality node is a single-successor certification boundary: **LocalityCertificate** proves that the global estimates are supported by the declared local extractor. The deficit node then returns a CT11 additive-deficit payload or **LocalLedgerState**. Only a local ledger reaches the summation node, which certifies **SummationState** before the global comparison is classified.

The comparison node returns exactly one of a C4 certificate, a CT4 charge ledger, or a CT14 aggregate residual. **CT4Payload** contains only the assignment, capacity, and lower-bound fields that CT4 still needs to certify. Its **toInput** adapter fixes **G** and **branch** definitionally. The theorems **ct4_payload_same_ambient** and **ct4_payload_same_branch** therefore hold by reflexivity. This prevents the handoff from changing the object or branch state during ledger assembly.

Every terminal result retains the exact locality, ledger, and summation indices that justify it. **Port** and **HandoffPlan** certify only consumer acceptance. Theorems **run_verified**, **run_total**, and **run_trace_valid** establish terminal soundness, relative totality, and trace validity.

The soundness contracts are assigned as follows.

- **S-Def** fixes the sites, extractor, witness relation, and downstream vocabulary.
- **S-Equiv** is the locality certification that connects each global estimate to its local witness data.
- **S-Dich** is discharged by the scope, deficit, and comparison constructors.
- **S-Comp** is discharged by **SummationState** and the final comparison result.
- **S-Pers** is enforced by the dependent indices carried through the ledger, summation, and terminals.
- **S-Rout** and **S-Trig** are discharged by the CT4, CT11, and CT14

payload constructors and the separate handoff plan.

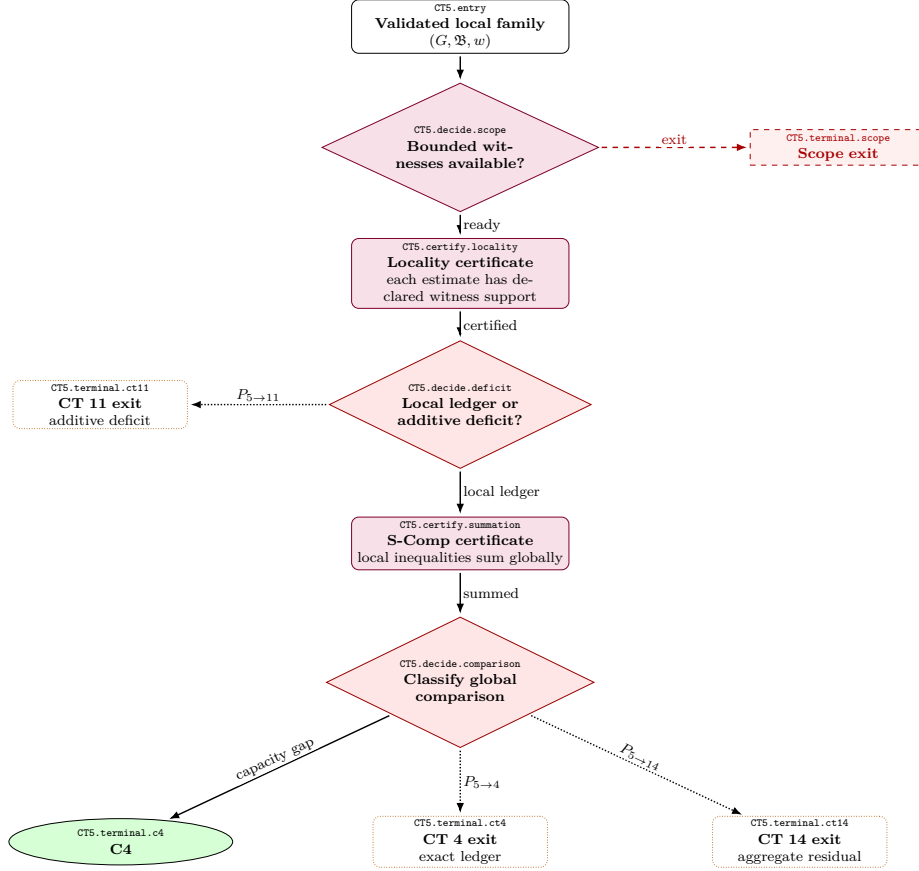


Figure 9: Closure-tactic 5 as an exact sequential machine. Locality and summation are one-successor certification boundaries. The comparison node can close at C4 or emit one indexed payload. The CT4 payload converts to the exact downstream **CT4.Input** while preserving the ambient object and branch state definitionally. Monospaced identifiers are shared by Lean, JSON, and executable traces.

Local-to-global bookkeeping in practice

The method derives global consequences from local bookkeeping:

$$\begin{aligned} \text{local tests} &\Rightarrow \text{local demands} \Rightarrow \text{canonical charging} \\ &\Rightarrow \text{global capacity bound.} \end{aligned}$$

The low-level diagram for CT5 is Fig. 9; it identifies the local witness payloads, the summation audit, the capacity comparison, and the exits to CT4, CT11, and CT14.

2.20 CT6 — Active/dormant dichotomy

Signature. $P_{1 \rightarrow 6} \rightarrow \text{Cert}\{C1\}; \text{Int}\{P_{6 \rightarrow 4}, P_{6 \rightarrow 3}, P_{6 \rightarrow 7}, P_{6 \rightarrow 9}\}; \text{Rec}\{P_{6 \rightarrow 10}\}.$

CT6 is an exact sequential machine for the active/dormant split. Its stable entry interface fixes the ambient object, branch state, monitored family, activity threshold, first-failure order, first-failure witness type, scope predicate, and C1 claim. A validated **Input** contains the monitor, threshold, and order. The executable **Framework** adds the predicates certified by the machine and the exact entry contracts of CT3, CT4, CT7, CT9, and CT10.

The entry contract is kept in **CT6.Interface**, independently of the machine implementation. This separation is necessary because the tactic graph contains cycles, while Lean imports must be acyclic. A producer may therefore construct an exact **CT6.Interface.Input** without importing CT6's nodes or any tactic to which CT6 routes. The same interface layer is used for CT7–CT10.

The core plan has five independent fields: **scope**, **definition**, **activity**, **activeLedger**, and **dormant**. The scope node returns a typed **ScopeCandidate** or **ScopedState**. The definition node has one successor and certifies the activity predicate, threshold, order, and first-failure extractor. The activity node returns either an indexed **ActiveState** or an indexed **DormantState**. A dormant state contains the actual first-failure witness and its structural proof.

The active branch has no further classification choice: the **activeLedger** certification node constructs the exact CT4 input and its alignment proof. The dormant node returns exactly one of C1, CT3, CT7, CT9, or CT10. Each payload is indexed by the dormant state that licensed it and contains the consumer's validated input. **Port** and **HandoffPlan** certify consumer

acceptance after `runCore`; they cannot change the route selected by a core node.

The soundness contracts occur at the following machine boundaries.

- **S-Def** fixes the monitor and first-failure vocabulary in the entry interface and is certified before the activity decision.
- **S-Dich** is the exhaustive active/dormant decision and the exhaustive dormant routing decision.
- **S-Equiv** is carried by the structural first-failure certificate used on the dormant branch.
- **S-Rout** and **S-Trig** are carried by the exact CT3, CT4, CT7, CT9, and CT10 inputs and their alignment proofs.

Failure to define activity or to prove the first-failure extractor structural is a pre-admission defect: no `DefinitionState`, and hence no admitted `CorePlan`, exists. Absence of any bounded first-failure interface is the typed scope terminal. It is not conflated with the CT10 recovery, which requires a concrete bounded witness and an exact refinement input.

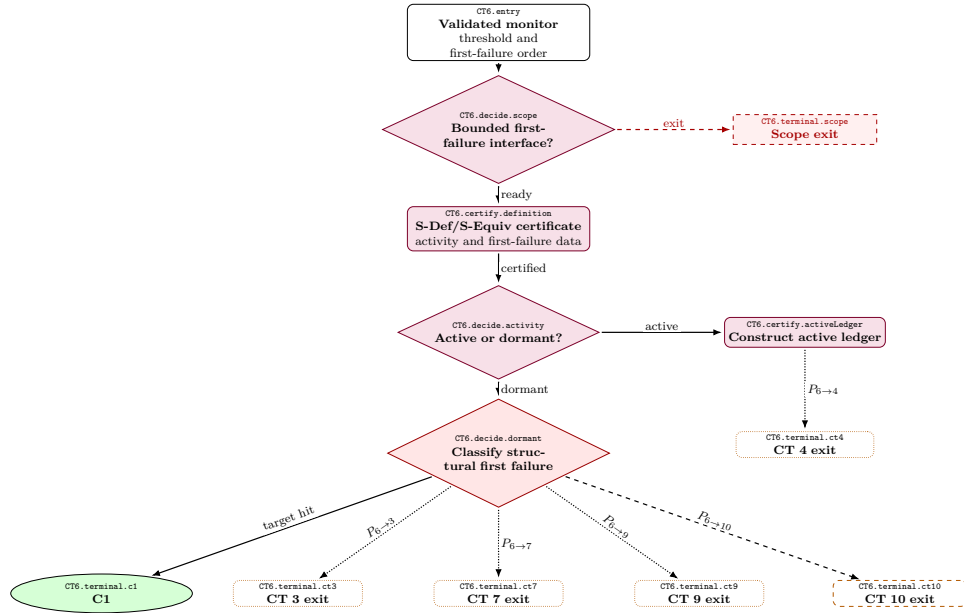


Figure 10: Closure-tactic 6 as an exact sequential machine. Monospaced labels are shared by Lean’s `Graph.NodeId`, the JSON graph, and executable traces. Certification nodes have one successor. The active ledger is an exact CT4 input; every dormant exit retains the first-failure state and the exact consumer input.

Active/dormant dichotomy in practice

Many difficult proofs contain a dichotomy of the following kind: the **active** or hot side is the branch on which the local test fires and pays an **explicit charge**, meaning a contribution to a demand count bounded from below; the **dormant** or cold side is the branch on which that local test is inactive.

Dormancy is handled by the **first-failure principle**: when a monitored property fails, the proof selects the minimal scale or index at which failure first occurs; the bounded local data witnessing that minimal failure is the **first-failure witness**. (The quantitative form of the principle is given in Section 3.8.) Schematically:

$$\begin{aligned} \text{dormant} &\Rightarrow \text{first failure} \Rightarrow \text{bounded structural object} \\ &\Rightarrow \text{finite trichotomy.} \end{aligned}$$

In the abstract: (1) define activity relative to the actual branch state; (2) if enough activity exists, count it; (3) if activity fails, select the *first* failure; (4) minimality of the first failure supplies extra structure; (5) that structure is a direct target, a defect, a compression, an overload, or a bounded exchange.

The quantitative form of the dichotomy deserves emphasis. When the active branch fails *linearly*, the entropy or counting cap that the active tests would supply falls short by a definite margin. The dormant family then has a *linear lower bound* computable from that margin. The dormant mass feeds the charging machinery of Section 3.5 and Section 2.23 with explicit constants. Thus the hot/cold split becomes a quantitative case ledger: a finite table of density regimes, each row carrying an exact numeric constraint and the name of the resource that pays it.

The low-level diagram for CT6 is Fig. 10; it gives the activity dichotomy, first-failure witness, and the typed routes from dormant mass to charging, types, exchange, or overload.

2.21 CT7 — Exchange trichotomy

Signature. $P_{* \rightarrow 7} \rightarrow \text{Cert}\{C1, C2, C3\};$
 $\text{Int}\{P_{7 \rightarrow 3}, P_{7 \rightarrow 12}\}; \text{Rec}\{P_{7 \rightarrow 10}, P_{7 \rightarrow 16}\}.$

CT7 is an exact machine for a bounded same-interface exchange. Its stable entry interface contains the ambient and branch states, the two local representatives, their common boundary, the exchange witness, the context

system, the finite-scope predicate, and the C1–C3 claims. The executable framework adds context completeness, realization, distinction, and neutrality predicates together with exact CT3, CT10, CT12, and CT16 entry contracts.

The core plan has six fields: **scope**, **context**, **realization**, **distinction**, **defect**, and **neutral**. The context node has one successor and constructs a **ContextState** containing the completeness proof for the finite generator. The realization node either returns a C1 certificate or an **UnrealizedState**. Only the latter reaches the distinction node, which returns a **DefectState** or a **NeutralState**.

Defect routing and neutral compression are separate APIs. The defect node returns C3, an exact CT3 input, or an exact CT12 peeling input. The neutral node returns C2, an exact CT10 input, or an exact CT16 whole-object input. Thus a node implementer proves only one local classification and may rely on every earlier certificate through the predecessor index. No string tag or unindexed residual can bypass these states.

- **S-Def** is fixed by the validated exchange entry.
- **S-Equiv** is the one-successor context-completeness certificate and is retained by both later branches.
- **S-Dich** is discharged separately by realization and distinction.
- **S-Comp** is discharged only at the neutral node, where a C2 certificate proves the smaller equivalent representative.
- **S-Rout** and **S-Trig** are discharged by the four indexed handoff families and the consumer plan.

No finite context generator yields the typed scope terminal. An assertion of neutrality without generator completeness is not an executable branch: it is an uninhabited node contract that must be repaired before admission.

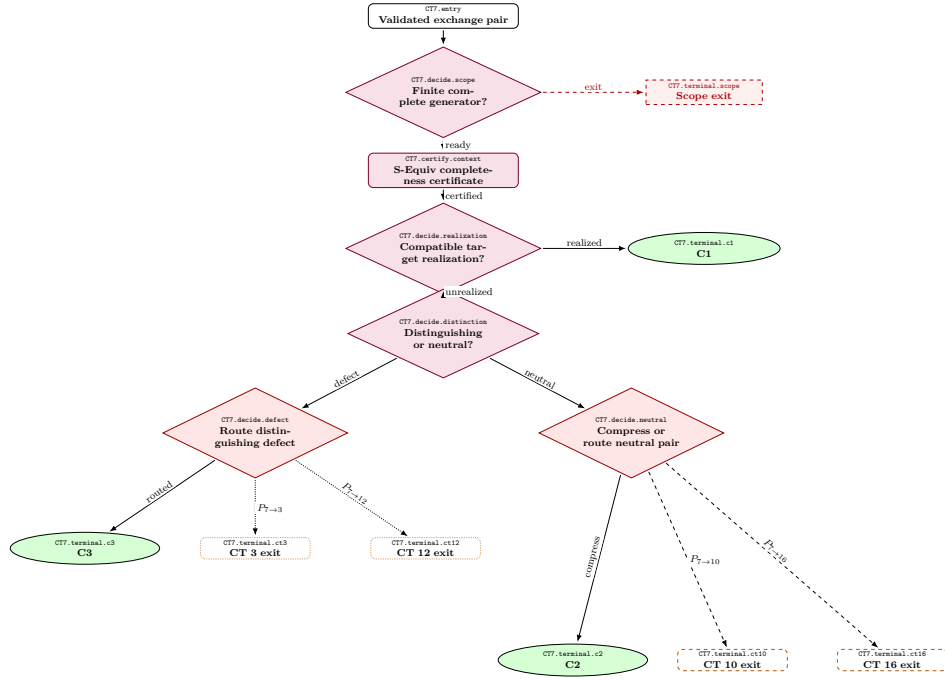


Figure 11: Closure-tactic 7 as an exact sequential machine. Context completeness is certified once. Realization, distinction, defect routing, and neutral compression are independent node contracts. Each edge carries the exact indexed state consumed by its successor.

Exchange trichotomy in practice

Recall from Section 3.4 that a **local exchange** is a bounded local operation that changes a target-relevant quantity by a bounded increment, presented as two local representatives A and A' with the same interface. This is the method's disciplined form of the classical exchange and switching arguments. Every local exchange is analyzed as follows.

Realizing. Some compatible context makes one representative realize the target: $T(A \cup C) = 1$. Close by target.

Distinguishing. Some context distinguishes the representatives: $T(A \cup C) \neq T(A' \cup C)$. Route to a target defect.

Neutral. No context distinguishes the representatives: $T(A \cup C) = T(A' \cup C)$ for all C . If one representative is smaller, compress.

For a CT7 invocation with a certified finite context generator, the declared outputs are

$$\boxed{C1 \mid C2 \mid C3 \mid P_{7 \rightarrow 3} \mid P_{7 \rightarrow 12} \mid P_{7 \rightarrow 10} \mid P_{7 \rightarrow 16}}.$$

The context-generator and S-Equiv completeness proofs are part of this contract. Exchanges of increment zero enter a designated finite same-interface table, where each entry is assigned one of the declared outputs.

The low-level diagram for CT7 is Fig. 11; it is the formal checklist for realizing, distinguishing, neutral, and whole-object exchange exits.

2.22 CT8 — Finite-state pumping

Signature. $P_{* \rightarrow 8} \rightarrow \text{Cert}\{C2, C5\}; \text{Int}\{P_{8 \rightarrow 3}, P_{8 \rightarrow 7}\}; \text{Rec}\{P_{8 \rightarrow 10}\}.$

CT8 is an exact sequential machine for finite-state pumping. Its entry interface fixes a bounded-width sequence, a type system, a numeric threshold, the finite-scope predicate, and the C2 and C5 claims. The executable framework adds exact-type equality, shortness, repetition, contextual equivalence, and separation predicates, together with exact CT3, CT7, and CT10 entry contracts.

The scope node has three proof-carrying constructors. **exit** proves that no finite type space is admitted. **refine** carries a proof that a finite label system is too coarse and an exact CT10 input. **ready** produces a `ScopedState` for

an exact finite type system. These cases are semantically different and cannot be represented by a shared failure tag.

The remaining core plan fields are **equivalence**, **repetition**, **response**, and **routing**. The equivalence node has one successor and certifies the exact equality test before pigeonhole reasoning. The repetition node returns a C5 finite-enumeration certificate or a **RepeatedState**. The response node returns a C2 certificate for contextually equal repeated types or a **SeparatingState**. Only that separating state reaches the final node, which constructs an exact CT3 or CT7 input.

- **S-Def** is fixed by the sequence, exact-type system, equality test, and explicit threshold.
- **S-Dich** is split between scope admission, repetition, and contextual response.
- **S-Equiv** is a one-successor certificate required before repeated states may be compared.
- **S-Comp** is carried by the short enumeration and repeated-type compression certificates.
- **S-Rout** and **S-Trig** are carried by exact CT3, CT7, and CT10 validated inputs.

Informal periodicity is not an executable outcome. If the exact response certificate cannot be constructed, the node plan is incomplete. Theorems `run_verified`, `run_total`, and `run_trace_valid` establish soundness, plan-relative totality, and validity of the evidence-indexed trace.

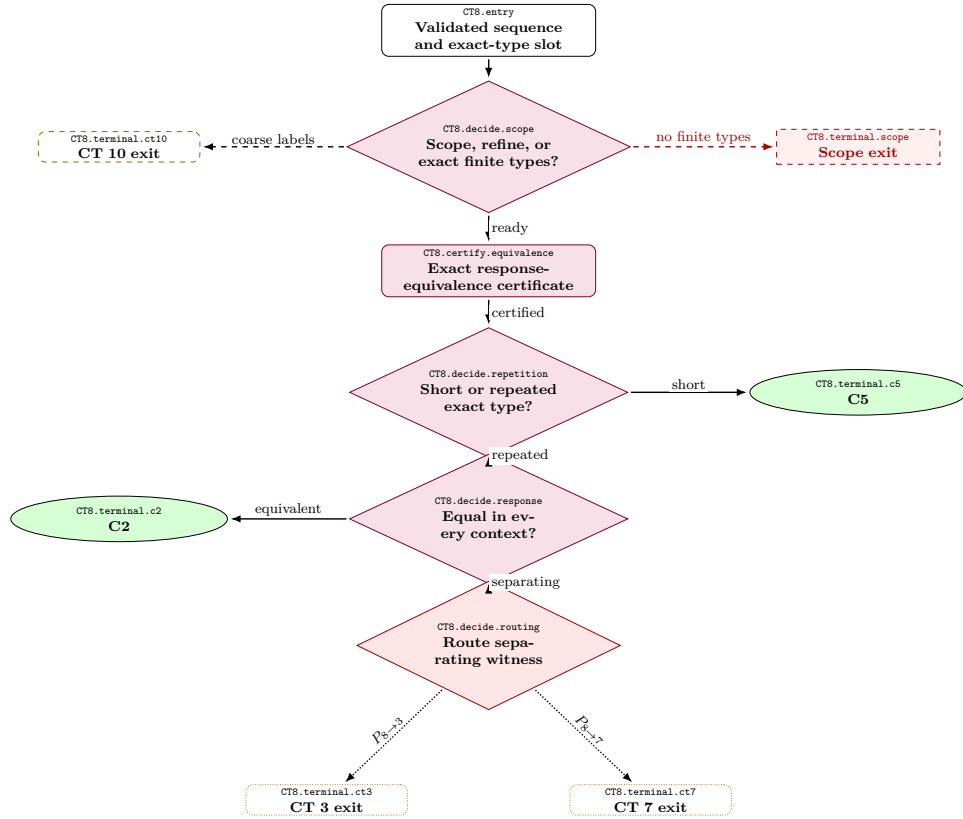


Figure 12: Closure-tactic 8 as an exact sequential machine. Scope failure and a finite-but-coarse label design are distinct typed terminals. Exact equivalence is certified before repetition, and every later branch retains that certificate through its state index.

Finite-state pumping in practice

The name deliberately echoes the pumping lemma of automata theory: a sequence with more elements than available states must repeat a state, and the repetition is exploitable. As in the Myhill–Nerode and Courcelle-style finite-state viewpoints, the relevant state is an interface-complete response profile [????].

Whenever a branch produces a long chain, path, or sequence of bounded-width states $S_0, S_1, \dots, S_m$, assign each state a finite label or an exact external type. If m exceeds the number of possible types, two types repeat.

If the repeated types are equal against all contexts, $\tau_i = \tau_j$, then the segment between them is removable or replaceable, and minimality kills it. If the types differ, there is a witness context, and that witness creates a target defect or a live local test. A long chain must therefore hit the target, produce a defect, or compress.

This is the abstract form of “periodicity implies reducibility.” In this document, *periodic* means equality of exact external types. Congruence modulo a number and equality of informal labels are insufficient.

The low-level diagram for CT8 is Fig. 12; it connects the finite label space, pigeonhole step, equality test, finite enumeration, and separating-context routes.

2.23 CT9 — Overload exhaustion

Signature. $P_{* \rightarrow 9} \rightarrow \text{Cert}\{C1\}; \text{Int}\{P_{9 \rightarrow 4}, P_{9 \rightarrow 7}, P_{9 \rightarrow 8}\}; \text{Rec}\{P_{9 \rightarrow 10}\}.$

CT9 is an exact machine for proving bounded multiplicity by exhausting an overloaded payer fibre. Its entry interface fixes the payer, fibre, finite labels, capacity datum, scope predicate, and C1 claim. The executable framework adds the fibre-definition certificate, bounded and overloaded predicates, the homogeneous-extraction theorem, and exact CT4, CT7, CT8, and CT10 entry contracts.

The scope node distinguishes three cases. Absence of any finite labelling is a typed scope terminal. A finite but inadequate labelling produces a proof of coarseness and an exact CT10 refinement input. A usable labelling produces a **ScopedState**. The **fibre** node then has one successor and certifies that the stored fibre is the declared inverse image with its capacity data.

The overload decision consumes the certified fibre. Its bounded constructor returns an exact CT4 input together with the bounded-multiplicity proof; its overloaded constructor returns an `OverloadedState`. The `extraction` node has one successor and certifies the homogeneous subfamily required by the instantiation theorem. Only then does the routing node choose C1, an exact CT7 exchange input, or an exact CT8 sequence input. Consequently CT9 does not assume that an arbitrary labelled family has one of these outputs: the extraction theorem is an explicit node contract.

- **S-Def** is fixed by the payer fibre, finite labels, and capacity datum and certified by `FibreState`.
- **S-Dich** is the bounded/overloaded decision and the exhaustive extraction result.
- **S-Comp** is the homogeneous-extraction certificate.
- **S-Rout** and **S-Trig** are carried by exact CT4, CT7, CT8, and CT10 validated inputs.

An unlabelled fibre and an undeclared tie-break cannot be represented by a successful plan. The scope terminal records failure of finite expressibility; the CT10 terminal records a concrete finite refinement problem.

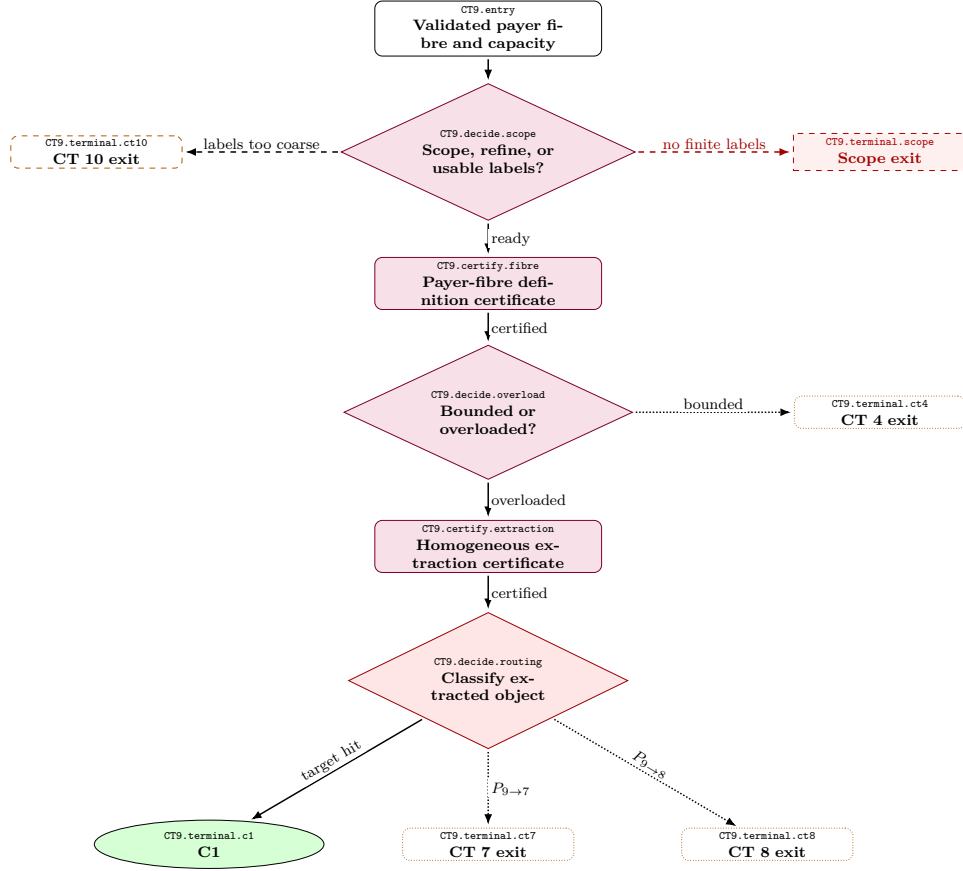


Figure 13: Closure-tactic 9 as an exact sequential machine. Fibre definition and homogeneous extraction are one-successor certification boundaries. Bounded multiplicity returns an exact CT4 input; overload routing occurs only after the extraction theorem has produced its indexed state.

Overload exhaustion in practice

Given demands $\mathcal{D}$ and payers $\mathcal{P}$, a charging scheme is a map $\pi : \mathcal{D} \rightarrow \mathcal{P}$, and the bounded-multiplicity lemma asserts $|\pi^{-1}(p)| \leq C$ for every p , whence $|\mathcal{D}| \leq C|\mathcal{P}|$.

The crucial methodological point is that **bounded multiplicity is proved by exhaustion of the overload branch**. If $|\pi^{-1}(p)| > C$ for some p , then that fibre is overloaded; one extracts a monochromatic subfamily, applies the instantiation-specific extraction theorem, and routes its target-hit, same-interface-exchange, or long-chain output. Equivalently,

$$\text{overcount} \Rightarrow \text{C1 or a typed residual for CT7, CT8, or CT10.}$$

The low-level diagram for CT9 is Fig. 13; it shows the payer-fibre dichotomy, homogeneous-overload extraction, target-hit branch, and typed returns to CT4, CT7, and CT8.

2.24 CT10 — Default refinement

Signature. $P_{* \rightarrow 10} \rightarrow \text{Cert}\{C5\}; \text{Int}\{P_{10 \rightarrow 3}, P_{10 \rightarrow 7}, P_{10 \rightarrow 15}\}; \text{Rec}\{\emptyset\}$.

CT10 is an exact machine for promoting a finite missing datum to the next branch invariant. Its entry interface fixes the residual class, extracted datum, finite alphabet, class table, finite-scope predicate, and C5 claim. The executable framework adds label-class exhaustion, finite exchange support, direct classification, missing-datum extraction, promotion certification, and the exact CT3, CT7, and CT15 entry contracts.

The core plan has six fields: `scope`, `labels`, `classification`, `direct`, `promotion`, and `promotedRouting`. The scope node produces a typed scope terminal or a `ScopedState`. The labels node has one successor and constructs a `LabelState` containing both the proof that label classes exhaust the residual class and the proof that supported exchanges are finite within each class.

The classification node returns a C5 certificate, a `DirectState`, or a `MissingState`. Direct routing constructs an exact CT3 response input or an exact CT7 class-exchange input. A missing state cannot route directly. It must first pass through the one-successor promotion node, which proves that the datum was extracted from the failed classification and records it as a new invariant. The promoted routing node then constructs a provenance-tagged CT3 input or an exact CT15 rank input.

CT10 has two semantically distinct routes to the same CT3 terminal. The inductive **CT3Payload** records whether the input came directly from a classified label class or only after promotion. Each constructor retains the corresponding predecessor state, so the consumer cannot erase this provenance. The dependent path type permits both incoming edges while keeping every individual execution linear.

- **S-Def** fixes the residual, promoted datum, alphabet, and class table; promotion is separately certified.
- **S-Dich** is the exhaustive class-level classification.
- **S-Comp** is carried by class exhaustion and the C5 certificate.
- **S-Rout** and **S-Trig** are carried by exact CT3 and CT7 inputs and the indexed CT15 datum.

An infinite adequate refinement alphabet is the typed scope terminal. Failure to extract the promoted datum is not a runtime loop or unnamed residual: no **PromotedState** can be constructed, so the tactic must be redesigned at that node boundary.

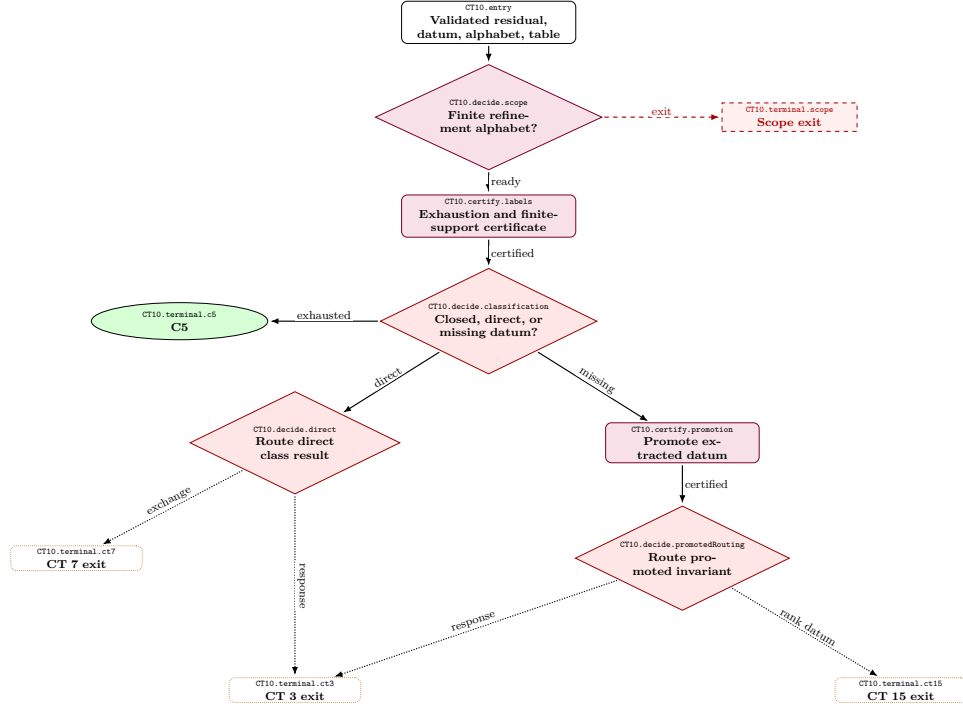


Figure 14: Closure-tactic 10 as an exact sequential machine. Label exhaustion and datum promotion are separate one-successor certificates. The two CT3 edges converge at one terminal but carry different constructors of `CT3Payload`, preserving direct versus promoted provenance.

Default refinement in practice

Given CT10’s admitted residual class and finite label data, the **default refinement** stratifies the residual by that label. Inside each label class, run the exchange trichotomy (Section 2.21) on the bounded exchanges the class supports. One of three things happens: the class closes; an exchange or response-type datum appears and routes; or the analysis reveals precisely which datum the label is missing. That datum, promoted to the label or to the type, is the next invariant.

The low-level diagram for CT10 is Fig. 14; it displays the finite label exhaustion, class-level classification, and the payloads routed to CT3, CT7, and CT15.

2.25 CT11 — Localization

Signature. $P_{5 \rightarrow 11} \rightarrow \text{Cert}\{\emptyset\}; \text{Int}\{P_{11 \rightarrow 1}, P_{11 \rightarrow 7}, P_{11 \rightarrow 14}\}; \text{Rec}\{P_{11 \rightarrow 10}\}.$

CT11 is a route-only localization machine. Its entry contract records the global deficit, admissible decomposition, admissibility class, and bounded local type. The executable framework adds four independent propositions: decomposition certification, closure of admissibility under local moves, summation, and existence of a localized deficit. It also fixes the exact validated entry interfaces of CT1, CT7, CT10, and CT14.

The core plan has five fields: **scope**, **decomposition**, **admissibility**, **localization**, and **routing**. The decomposition node and localization node each have one successor and therefore produce certificates rather than choices. The admissibility decision either constructs an exact CT10 input or proves that the localized object remains in the admissible class. Only the latter state may enter localization. The final routing decision constructs an exact CT1 target-test input, CT7 exchange input, or CT14 aggregate-mass input. CT11 has no closure constructor.

This separation makes two common mistakes unrepresentable. An uncertified decomposition cannot reach the admissibility decision, and an admissibility gap cannot be silently carried into the averaging lemma. Each handoff structure contains a downstream input and an alignment proof tying it to the CT11 entry.

- **S-Def** is discharged by the validated entry and **DecompositionState**.
- **S-Comp** is discharged by **LocalizationState**, which stores both summation and localized-deficit proofs.

- **S-Rout** and **S-Trig** are discharged by the four exact consumer payloads.

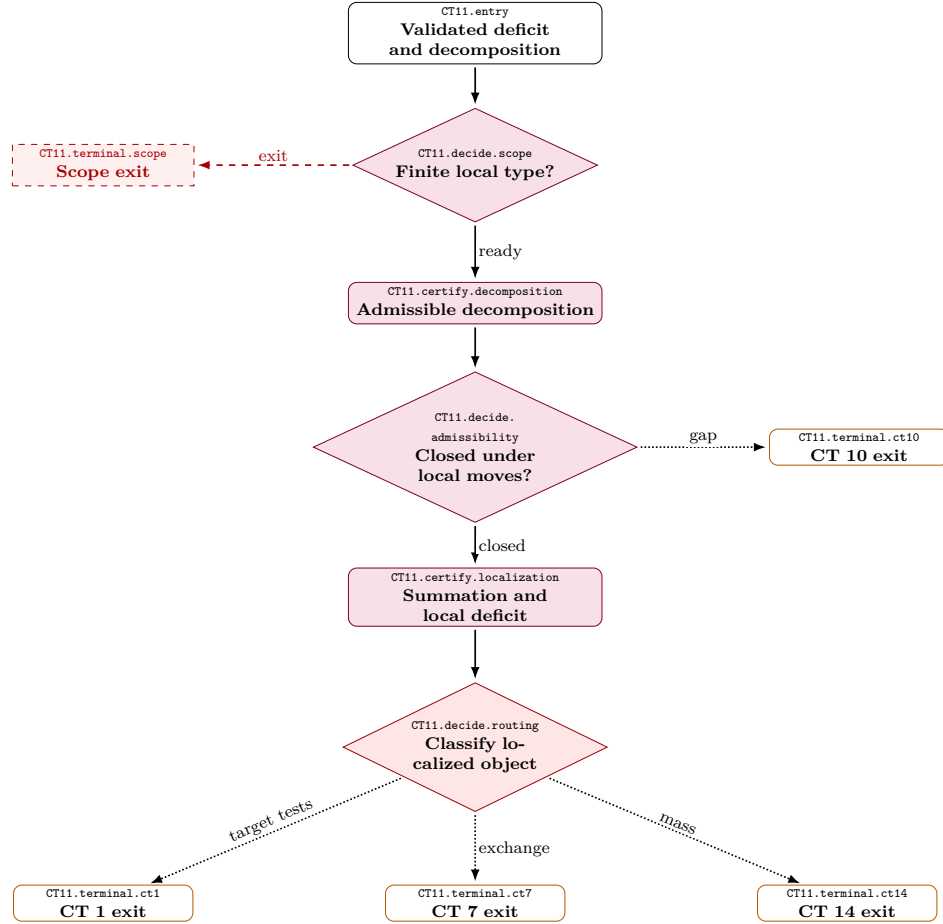


Figure 15: CT11 as an exact route-only machine. Certification of the decomposition, the admissibility decision, and certification of localization are distinct interfaces.

Localization in practice

Require every budget to be a sum of local contributions,

$$B(X) = \sum_{v \in X} b(v) + \text{boundary corrections, bounded per attachment point.}$$

Two consequences are then generic and are stated once, as lemmas, per admissibility class:

1. *Summation*: global bounds are sums of local bounds, so the capacity inequality of the branch is the sum of per-cell inequalities.
2. **Localization**: if the aggregate budget is in deficit, $B(R) < 0$, then some *connected admissible piece* $X \subseteq R$ satisfies $B(X) < 0$ — by decomposing R into connected admissible pieces and averaging.

The localization lemma is the standard bridge from the counting layer to the local-exclusion layer: it converts a global numerical deficit into a bounded local object, which is where the taxonomy of Section 3.4 takes over. Its proof is uniform; the admissibility class is the variable input. The design obligation is that **admissibility be closed under the moves of the local analysis** — deletion, peeling, quotient — so that the localized piece remains inside the toolbox.

The low-level diagram for CT11 is Fig. 15; it makes the additive decomposition, admissibility audit, localization lemma, and exits to CT1, CT7, CT10, and CT14 explicit.

2.26 CT12 — Peeling loop

Signature. $\langle \mathfrak{B}, L, \mu, \text{routed load} \rangle \rightarrow \text{Cert}\{C4\};$
 $\text{Int}\{\emptyset\}; \text{Rec}\{P_{12 \rightarrow 4}, P_{12 \rightarrow 13}\}.$

CT12 is the unique cyclic tactic in the library. Its entry interface records an ambient branch, load account, peel move, and natural-valued load. The executable framework supplies an invariant indexed by the current load, a single-peel certificate, a restoration relation, and exact CT4 and CT13 entry interfaces.

The machine separates the three facts needed for a legal iteration. A `PeeledState` proves that the chosen unit was peeled canonically. A `RestoredState` proves that the branch invariant was restored at a

named next load. A `DecreasedState` separately proves that this next load is strictly smaller. Only `DecreasedState` licenses the edge from `CT12.certify.decrease` back to `CT12.decide.saturation`. Restoration without decrease cannot form a machine path.

The executor `runLoop` is defined by well-founded recursion on the current natural-valued load. Its recursive call consumes the loop state stored in `DecreasedState`; Lean accepts the definition using the field `decreases`. The theorem `Graph.Edge.loop_decreases` extracts a strict inequality from every back edge. Removing this single edge leaves an acyclic graph.

At the saturation decision, an unsaturated state closes by C4. A peelable state passes through peel certification and restoration. Restoration may produce ordinary demand for CT4, tier interaction for CT13, or a restored state that proceeds to the decrease certificate. The node plan therefore exposes six independent fields: `scope`, `measure`, `saturation`, `peel`, `restoration`, and `decrease`.

- **S-Det** is localized at the peel certificate.
- **S-Rest** is localized at the restoration decision.
- **S-Meas** is localized at initial measurement and strict decrease.
- **S-Comp** is carried by the unsaturated C4 certificate.
- **S-Rout** and **S-Trig** are carried by exact CT4 and CT13 inputs.

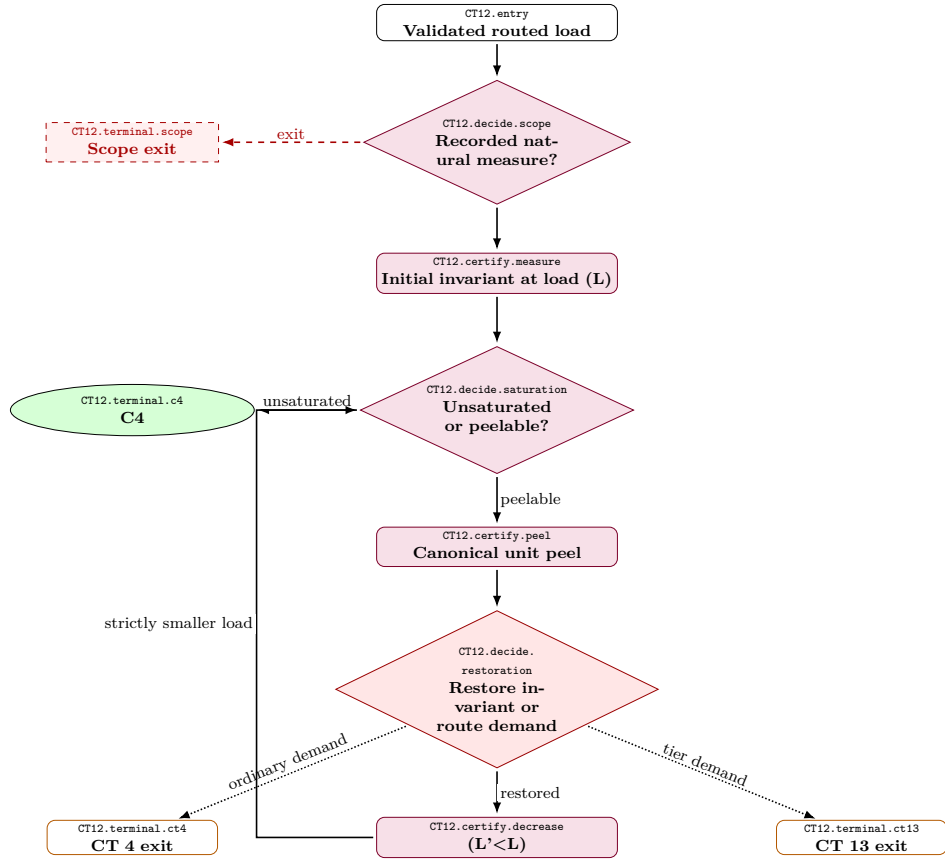


Figure 16: The exact CT12 loop. The back edge exists only after invariant restoration and a separate strict-decrease certificate.

2.27 CT13 — Tiered charging

Signature. $P_{*\rightarrow 13} \rightarrow \text{Cert}\{C4\}; \text{Int}\{P_{13\rightarrow 4}, P_{13\rightarrow 9}, P_{13\rightarrow 14}\}; \text{Rec}\{\emptyset\}.$

CT13 is an exact two-tier accounting machine. Its entry fixes the tier account, availability datum, measurable resource, branch state, and C4 claim. The executable framework separates tier-one availability, canonical payment, minimal fallback obstruction, canonical tier-two payment, resource reconciliation, and the combined capacity comparison.

The availability node produces either an **AvailableState** or an **UnavailableState**. The available branch must pass through tier-one payment certification and a separate CT4 payload-construction node. The unavailable branch must pass through fallback certification before the reconciliation decision. A reconciled state proceeds through the one-successor combined comparison and closes by C4. An overlap state proceeds to a separate routing decision and constructs an exact CT9 fibre input or CT14 mass input.

This flow prevents an availability failure from being treated as a free fallback choice. The fallback state contains both minimality and canonicity, and no overlap route exists until the reconciliation node has produced an **OverlapState**.

- **S-Det** is split between tier-one payment and minimal canonical fallback.
- **S-Dich** is the availability decision and the reconciliation decision.
- **S-Comp** is the resource-reconciliation and combined-capacity certificate.
- **S-Rout** and **S-Trig** are exact CT4, CT9, and CT14 inputs.

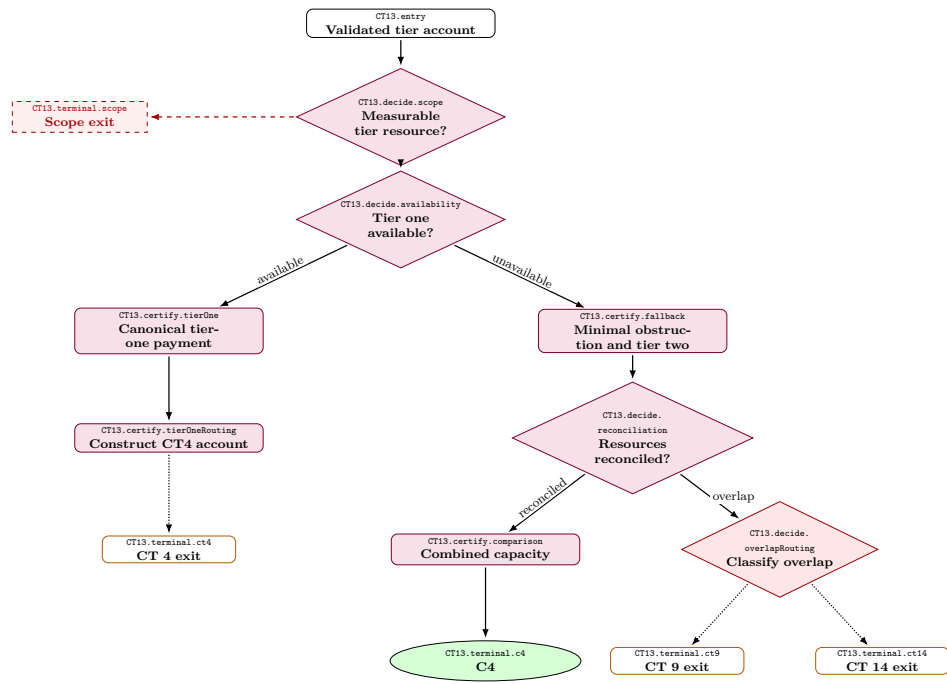


Figure 17: CT13 separates availability, payment, fallback construction, reconciliation, comparison, and overlap routing into independent contracts.

Tiered charging in practice

The bounded-multiplicity principle of Section 2.23 has a second-order form. Some charging schemes are *conditionally* available: they require a disjointness, independence, or compatibility hypothesis that may fail. The correct design is **tiered charging**. The availability hypothesis becomes a dichotomy, and its failure produces a *canonical minimal obstruction*, meaning a smallest witness of the incompatibility. That obstruction becomes a demand in a designated fallback scheme with its own payers.

The pattern is:

$$\begin{aligned} \text{scheme available} &\Rightarrow \text{pay in tier 1;} \\ \text{scheme fails} &\Rightarrow \text{minimal obstruction} \Rightarrow \text{pay in tier 2.} \end{aligned}$$

The closure certificate is the pair of capacity inequalities, one per tier, together with the minimality of the obstruction (which is what makes the tier-2 assignment canonical). The reconciliation audit requires the tier-2 payers to be disjoint from, or explicitly reconciled with, the tier-1 payers; otherwise the two tiers double-charge the same resource.

The low-level diagram for CT13 is Fig. 17; it shows the availability dichotomy, minimal obstruction, tier-two account, reconciliation audit, and typed overlap exits.

2.28 CT14 — Aggregate closure

Signature. $P_{* \rightarrow 14} \rightarrow \text{Cert}\{C4\}; \text{Int}\{\emptyset\}; \text{Rec}\{P_{14 \rightarrow 9}, P_{14 \rightarrow 10}\}.$

CT14 compares a forced-mass lower bound with a class-capacity upper bound. Its entry contract records the residual class, lower mass, capacity record, carried-charge multiplicity, branch state, and C4 claim. The core plan has four nodes: scope, joint bounds certification, multiplicity classification, and comparison certification.

BoundsState contains the lower and upper estimates but does not claim that their units agree. The multiplicity node either constructs an exact CT9 input for an unbounded member, an exact CT10 input for a missing finite label, or a **MultiplicityState**. Only the last state reaches the comparison certificate and C4. Thus a sublinear count cannot masquerade as a sublinear carried capacity without the multiplicity proof.

- **S-Def** fixes the residual class and its branch provenance.
- **S-Comp** is split between bounds, multiplicity, and final comparison.
- **S-Rout** and **S-Trig** are exact CT9 and CT10 inputs.

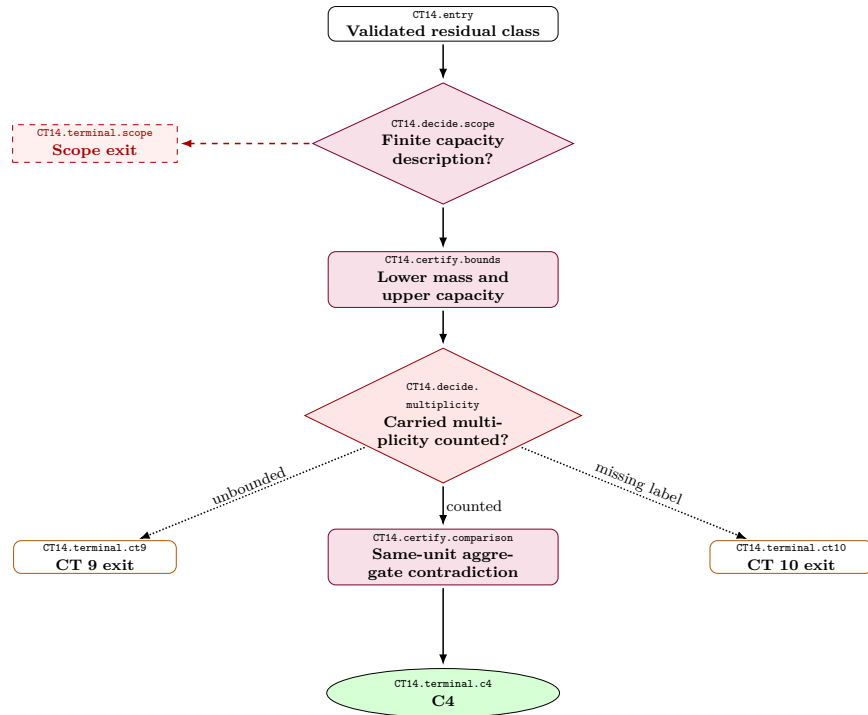


Figure 18: CT14 keeps bound certification, multiplicity accounting, and the final comparison as separate proof boundaries.

Aggregate closure in practice

A residual class $\mathcal{C}$ that is required, by the branch state, to carry a *linear* deficit can be closed by proving that its total **deficit-carrying capacity** is *sublinear*. This capacity is the aggregate mass of charge all members of $\mathcal{C}$ can absorb together. The certificate has two halves: a lower bound, where the surviving branch forces $\mathcal{C}$ to carry $\geq cn$ deficit, and an upper bound, where the members of $\mathcal{C}$ are charged to a global resource of size $o(n)$. Thus their total capacity is $o(n)$, and the class is empty on the surviving branch.

The aggregate comparison applies to a residual class with a recorded linear deficit, sublinear aggregate capacity, and per-member multiplicity. The upper bound counts members *with multiplicity by carried charge*; a sublinear number of members can still carry linear mass if individual capacities are unbounded, so the per-member capacity bound is part of the certificate.

The low-level diagram for CT14 is Fig. 18; it links the lower-bound lemma, upper-capacity lemma, multiplicity audit, comparison theorem, and overload route.

2.29 CT15 — Rank forcing

Signature. $P_{*\rightarrow 15} \rightarrow \text{Cert}\{C4\}; \text{Int}\{P_{15\rightarrow 3}, P_{15\rightarrow 7}, P_{15\rightarrow 16}, P_{15\rightarrow 4}\}; \text{Rec}\{\emptyset\}.$

CT15 is a target-relative rank firewall. Its entry contract fixes the test family, rank map, target datum, capacity datum, branch state, and C4 claim. The executable framework separately states that the rank is target-relative, that a drop yields structural dependence, that full rank yields independent coordinates, that the ledger is certified, and that rank demand exceeds capacity.

The rank certificate has one successor. The rank-drop decision then produces either a **DependenceState** or a **FullRankState**. Structural dependence enters a dedicated routing node and constructs an exact CT3, CT7, or CT16 input. Full rank enters a dedicated ledger certificate; the comparison node then closes by C4 or constructs an exact CT4 input. No ledger can be constructed on the dependence branch, so incidental rank cannot consume capacity.

- **S-Def** is the target-relative rank certificate.
- **S-Dich** is the rank-drop decision.
- **S-Comp** is the independent-coordinate ledger and comparison.
- **S-Rout** and **S-Trig** are exact CT3, CT4, CT7, and CT16 inputs.

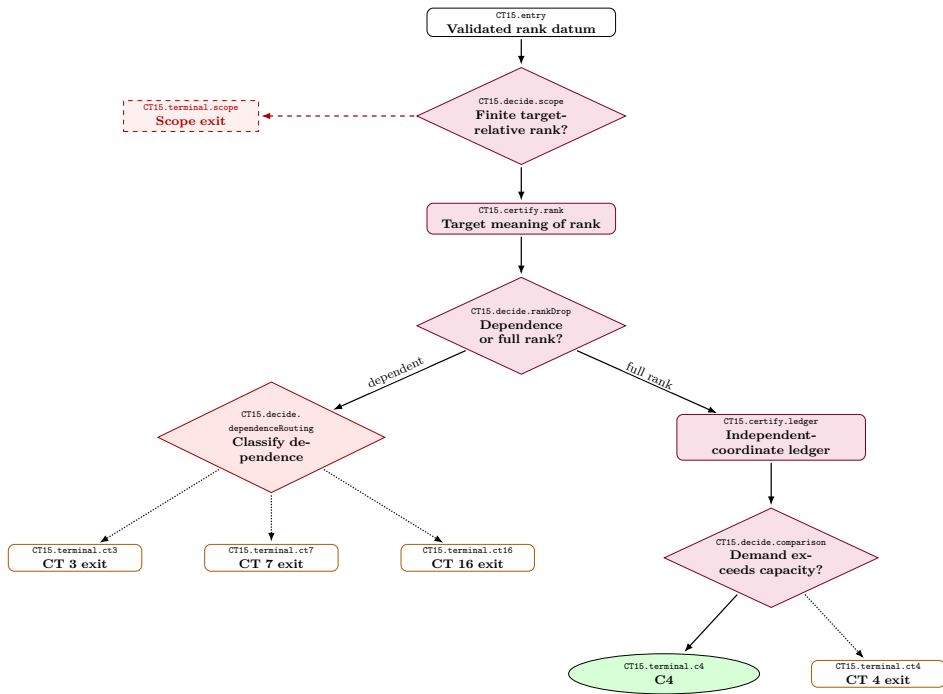


Figure 19: CT15 makes the dependence and full-rank branches disjoint at the type level; only full rank can reach ledger certification.

Rank forcing in practice

Raw local tests enter a counting or entropy bound after a rank-forcing layer has removed dependencies. Define a target-relevant rank on the family of tests, and split on rank deficiency. A rank drop is a *structural event* — some dependence among tests — and every dependence is routed through the standard taxonomy: it is target-defective, or it identifies pieces (compression), or it delocalizes. On the complementary branch, the tests have full rank, and then the forced cost

$$(\text{cost per test}) \times (\text{number of tests})$$

enters the budget as a recorded charge. Rank forcing is the accounting firewall between structure and entropy: dependence is structure and is routed; independence is entropy and is charged. The rank must be defined relative to the *target*, so that a rank drop carries routable structural meaning.

The low-level diagram for CT15 is Fig. 19; it displays the target-relative rank audit, rank-drop routing theorem, full-rank charge, and ledger route.

2.30 CT16 — Whole-object exact types

Signature. $P_{* \rightarrow 16} \rightarrow \text{Cert}\{C2\}; \text{Int}\{P_{16 \rightarrow 3}, P_{16 \rightarrow 10}\}; \text{Rec}\{\emptyset\}.$

CT16 handles the degenerate outside context of a whole-object datum. Its first runtime node tests support before asking for a finite closed type. A proper piece therefore routes directly to an exact CT3 input and does not inherit a whole-object computability requirement.

On the whole-support branch, the scope node either records that no finite closed type is available or produces a scoped whole-object state. The one-successor closed-type node proves both computation and the no-silent-identification rule. Literal equality then closes by C2; distinct closed data constructs an exact CT10 input. In particular, absence of a nonempty outside context is never treated as evidence of equality.

- **S-Dich** is split between proper/whole support and literal equality.
- **S-Equiv** is the exact closed-type computation and no-silent-identification certificate.
- **S-Rout** and **S-Trig** are exact CT3 and CT10 inputs.

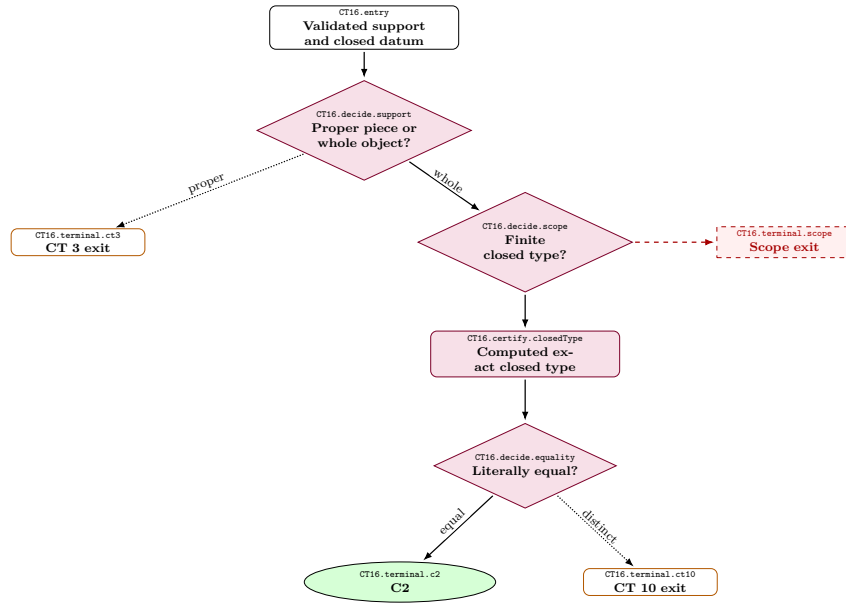


Figure 20: The proper-support route precedes whole-object scope and closed-type computation.

Whole-object exact types in practice

When delocalization reaches $Z = G$, the outside-context class contains only the empty completion. Outside-extension comparisons therefore provide no nontrivial distinguishing contexts. CT16 records a separate **exact closed type**, computed as an explicit finite data structure, and permits identification only when those closed types are literally equal. The certificate is the closed-form computation itself, together with a lemma stating that the degenerate empty context induces no additional identifications. An argument at $Z = G$ must exhibit the closed type explicitly; equality on an empty context supplies no equivalence beyond the recorded closed data.

The low-level diagram for CT16 is Fig. 20; it separates the proper-piece route, the whole-object closed-type computation, literal equality closure, and default-refinement exit.

2.31 CT17 — Target thickening

$P_{1 \rightarrow 17} \rightarrow \text{Cert}\{C1, C5\};$
Signature. $\text{Int}\{P_{17 \rightarrow 8}, P_{17 \rightarrow 14}\};$
 $\text{Rec}\{P_{17 \rightarrow 3}, P_{17 \rightarrow 10}\}.$

CT17 thickens a sparse target by offsets realized in one branch state. Its entry fixes the target set, offset family, completion classes, arithmetic datum, branch state, and C1 and C5 claims. Scope certifies that a bounded offset family with finite arithmetic leverage is available.

Compatibility is tested before block construction. Incompatible offsets enter a separation node and construct an exact CT3 type input or CT10 label input; they cannot be pooled in a target block. Compatible offsets pass through the one-successor block certificate and then the scale decision. A finite-range state enters the survivor decision, which closes by C5 or constructs an exact CT8 input. A repeated-increment state enters the arithmetic decision, which closes by C1 or constructs an exact CT14 input.

The two scale branches have different indexed states and different consumer payloads. Consequently a finite survivor list cannot be used as aggregate mass, and a large-scale arithmetic residual cannot be presented to the finite-state pumping tactic.

- **S-Def** fixes target, offsets, completion classes, and arithmetic data.
- **S-Dich** is split among compatibility, scale, survivor, and arithmetic decisions.

- **S-Comp** is carried by block construction and the C1/C5 certificates.
- **S-Rout** and **S-Trig** are exact CT3, CT8, CT10, and CT14 inputs.

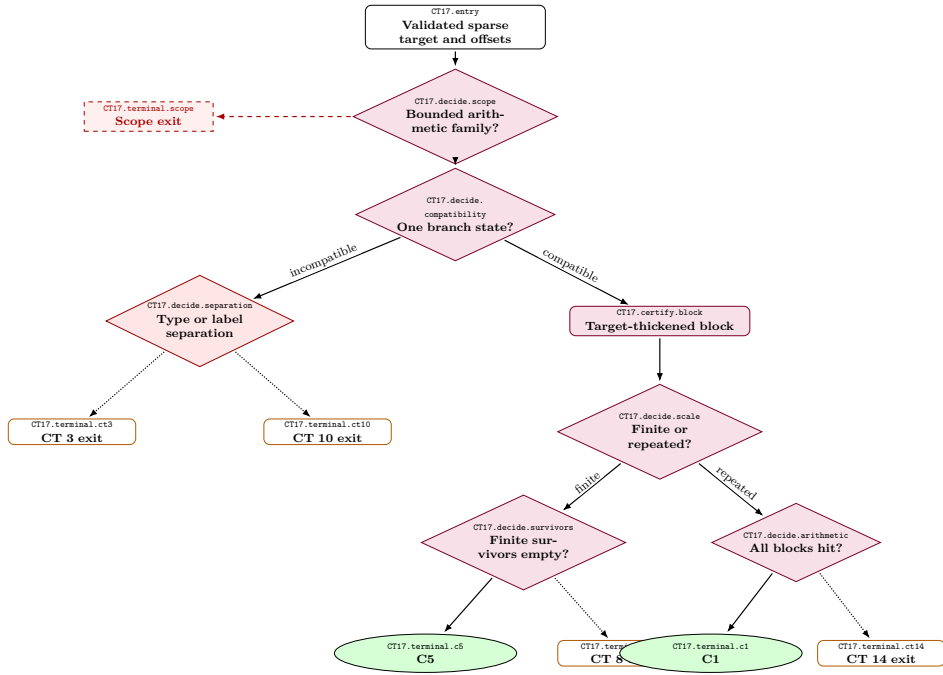


Figure 21: Compatibility precedes block construction; finite and repeated scales retain distinct states and exits.

Target thickening in practice

When the target lengths or values form a sparse set (powers, primes, a lacunary sequence), **target thickening** supplies the arithmetic leverage. This is the local branch-state analogue of the broader cycle-length forcing background around even cycles and sparse cycle lengths [??]. Identify a bounded family of local offsets, i.e. shifts realizable inside a single bounded piece. For a base value x , define its *offset block* by $x + \Delta = \{x + d : d \in \Delta\}$. No consecutiveness is assumed unless it is part of the supplied arithmetic lemma.

Avoidance of the sparse target becomes avoidance of thickened blocks, which is a strong arithmetic constraint. At small scales, only finitely many block positions dodge the target, and those positions become finite label classes. At large scales, repeated bounded increments make the blocks sweep intervals or residue classes, where orbit and order arguments apply. The certificate is the explicit finite list of surviving block positions plus the arithmetic criterion at large scales. The compatibility audit requires all offsets to be realizable within the *same* branch state; offsets requiring incompatible completions must be named and routed rather than pooled into one block.

The low-level diagram for CT17 is Fig. 21; it records the compatibility audit, block construction, scale split, finite-survivor route, arithmetic-mass route, and completion-class exits.

3 The strategy manual

Section 3 contains policy: the rules for proposing, admitting, selecting, parameterizing, and composing tactic invocations. It uses the Section 2 entry data, especially triggers, parameters, cost classes, and producer/consumer fields, but it does not redefine the internal procedure of any tactic. General branch design and tactic-choice discipline live here; CT-local implementation instructions live in the corresponding entries of Section 2.

3.1 Runtime model

The proof state is a triple

(branch tree, payload queue, registers).

The branch tree records the current residuals and closed leaves. The payload queue contains unconsumed typed messages $P_{i \rightarrow j}$. The registers are the

invariant register, ledger deposits, slack table, defect register, resilience register, and worksheet list.

The runtime loop is:

state+queue $\rightarrow$ propose $\rightarrow$ admit $\rightarrow$ select $\rightarrow$ execute $\rightarrow$ route $\rightarrow$ record.

Three phases are deliberately separated. **Before** a tactic invocation, the strategy layer proposes a trigger, checks admission, and synthesizes parameters. **Between** invocations, routed payloads are queued, consumed, or deliberately composed. **Alongside** invocations, worksheets and registers are written. Nothing strategic enters **during** a tactic: once an admitted CTk-# invocation starts, execution is the mechanism of Section 2.

Pattern	Phase	Role
Both-sides test	before	Admission criterion for a proposed invariant or trigger.
Bookkeeping-vs-theorem test	before	Rejects a proposed step that imports a new global theorem instead of spending local resources.
Invariant ladder	before	Orders branch predicates so later estimates never assume an unproved normalization.
Parameter synthesis (Section 3.16)	before	Manufactures labels, generators, ledgers, budgets, rates, and measures.
Typed payload graph	between	Forced routing of emitted residual payloads to consumer triggers.
Sequencing and spines	between	Deliberate composition of admitted invocations, with S-Meas on cycles.
Black-box isolation	between	Imports external theorems through declared interfaces.
Certificates C1–C5	halt	Terminal success returns.
Checklists, worksheets, registers	alongside	Verification artifacts that audit both mechanism and policy.
Resilience and defect registers	alongside	Failure semantics and repair history.

The queue discipline is strict: consume a queued payload before synthesizing a fresh one unless the selection policy below records a reason to defer it. This prevents the proof from accumulating orphan residuals whose routing is no longer visible. The queue may be empty only when the branch is closed or the active tactic is operating directly on the current residual condition.

Figure 22 displays the runtime workflow. In every strategy figure below, yellow nodes mark genuine policy choices, dotted diamonds mark checks whose outcome is determined by recorded data, solid arrows mark ordinary execution, dotted arrows mark typed or artifact handoffs, and dashed arrows mark repair or re-entry.

3.2 Branch states

A branch is created whenever the proof uses a dichotomy: property P holds or fails. The branch state records the cumulative context:

$$\mathfrak{B}_P = \mathfrak{B}_{\text{prev}} + P \quad \text{or} \quad \mathfrak{B}_{\neg P} = \mathfrak{B}_{\text{prev}} + \neg P.$$

The failure branch must be *meaningful*. If P is an accounting statement, $\neg P$ must force an overload (Section 3.4). If P is a structural statement, $\neg P$ must force a residual shape. If P is a target hit, $\neg P$ asserts the absence of all target-hitting local tests. This is what gives each branch a finite or measurable state space.

3.3 Closure criteria and the branch table

What counts as a finished branch

A branch is finished only when every leaf is one of:

1. **C1.** a direct target contradiction;
2. **C2.** a minimality contradiction;
3. **C3.** an entry in an already-routed defect account;
4. **C4.** a bounded-capacity contradiction;
5. **C5.** a finite, verified impossibility.

These five certificates are the fixed closure alphabet for the whole document. Every branch-level success must name one of C1–C5 and must do so by a route that the branch table can record.

A branch remains open whenever its final line gives only an informal plausibility judgment; Appendix D records the canonical prompt-lint examples.

The branch table

Every complete branch closure terminates in a table of the following form. The table records every residual explicitly.

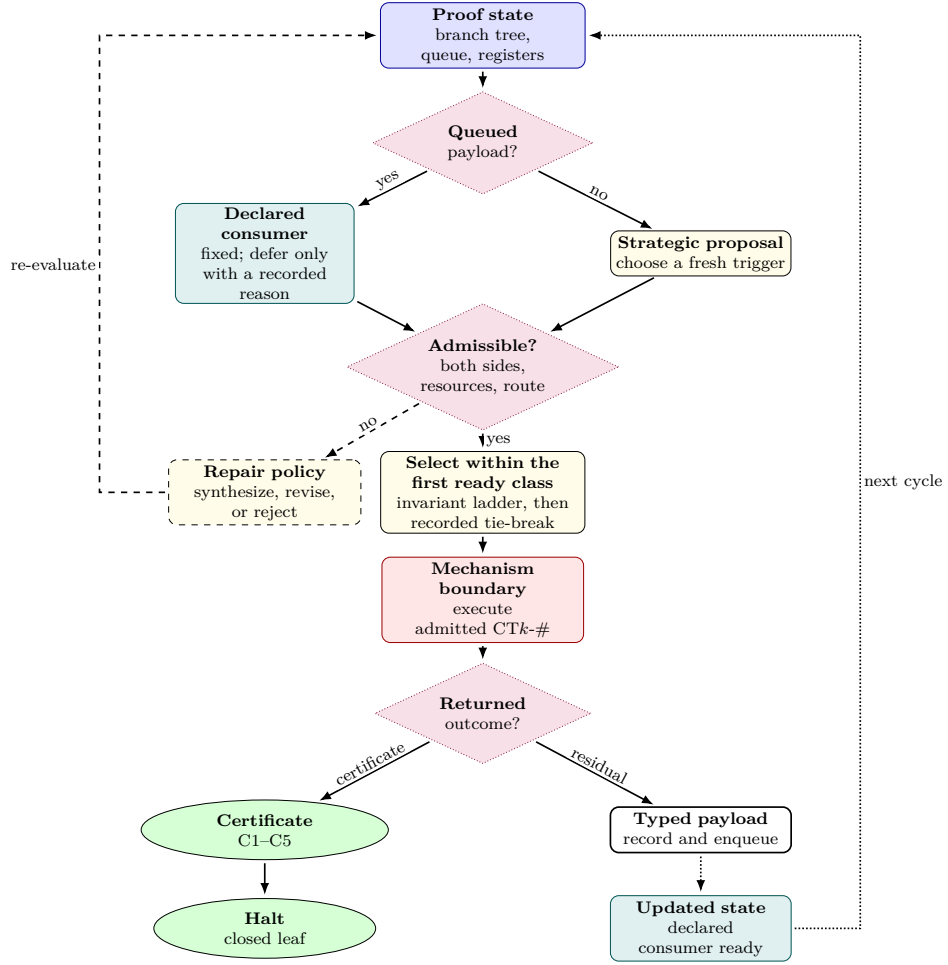


Figure 22: Strategy runtime loop. Queued payloads retain their declared consumers; fresh triggers and recorded tie-breaks are policy choices. Failed admission returns to repair, and every nonterminal output is requeued as a typed payload.

Branch	Residual condition	Demand	Payer	Bounded multiplicity	Closure
Direct hit	local test realized	none	none	n/a	contradiction
Defect	outside context	defective	defect account	canonical	routed
Compression	distinguishes pieces	quotient	minimality	piece	contradiction
	no context	piece size		n/a	
Overload	distinguishes smaller piece				
	demands exceed capacity	bad local objects	payer capacity	matching/star	bounded exchange
Dormant / low entropy	local tests absent	first failures	obstructions/chains	first-failure map	exchange trichotomy
Long chain	many bounded states	type sequence	exact types	pigeonhole	compression/defect
Bounded residual	finite local package	finite states	enumeration	exhaustive table	no leaf survives

3.4 Branch taxonomy

A new branch is admissible when the proof possesses a reliable routing principle for it. Two recurring verbs first: a residual is **routed** when it is mapped, by a canonical rule, into a previously named branch that has its own closure (so membership is the remaining obligation); and a demand is **charged** when the charging scheme assigns it a payer.

The admissible branch types are the following.

Direct-hit branch. A local configuration realizes the target: $X \Rightarrow T$. The branch closes immediately.

Defect branch. Two pieces that were supposed to be equivalent are distinguished by some outside context:

$$\exists C_{\text{out}} : \quad T(X \cup C_{\text{out}}) \neq T(X' \cup C_{\text{out}}).$$

The piece is then **target-defective**. A defect enters the **defect account** (a designated charging scheme whose demands are the defective pieces themselves) or a previously named route. A defect may also be consumed iteratively rather than all at once; see the peeling loops of Section 2.26.

Compression branch. Two pieces are indistinguishable against all outside contexts and one is smaller:

$$\tau(X') = \tau(X), \quad |X'| < |X|.$$

Minimality kills the branch. This move is called a **compression**.

Delocalization branch. The distinguishing structure spreads beyond the local **cell** to a larger piece $X \subsetneq Z$. A cell is an inclusion-minimal piece of the decomposition induced by the current branch state: the smallest unit to which finite labels are attached.

If Z is proper, replacement/minimality applies. If $Z = G$, a global type or **barrier argument** must be invoked explicitly. A barrier argument shows that a separating structure in G controls all target tests crossing it, so that the global external type itself excludes the target. Any such invocation must be audited as a global step in the sense of Section 3.7. The $Z = G$ case requires special care because its context is degenerate; see Section 2.30.

Overload branch. A demand count exceeds a payer capacity: $|\mathcal{D}| > C|\mathcal{P}|$. This situation is an **overload**. High load creates a homogeneous substructure via pigeonhole or matching/star extraction.

Monochromatic-family branch. Each demand carries a finite **label** describing its combinatorial role in the charging scheme. When a charging scheme is overloaded, pigeonhole over the finitely many labels extracts a large **monochromatic subfamily** — a subfamily of demands all bearing the same label:

$$\mathcal{D}_\lambda = \{d \in \mathcal{D} : \text{label}(d) = \lambda\}.$$

If $|\mathcal{D}_\lambda|$ is large, extraction inside a single label class produces a matching, a star, a long **chain**, a repeated external type, or a bounded family of **local exchanges**. A chain is a maximal sequence of bounded-width pieces glued consecutively along their boundaries, i.e. a long path of cells.

A local exchange is a bounded local operation, presented as two representatives with the same interface, that changes a target-relevant quantity by a bounded increment; its full analysis is the exchange trichotomy of Section 2.21. Each extracted structure is routed by direct hit, defect, compression, **auxiliary routing** (delegation to a designated side branch maintained for this purpose), or finite exhaustion.

Terminal-residual branch. A branch becomes terminal after every other exit has been formally excluded, where an **exit** is any of the named closure routes above. Terminal residuals are finite, bounded, or equipped with a

strict global deficit that the available charging schemes cannot carry. A terminal residual carries exact data:

$$\begin{aligned} \text{terminal residual} = & \text{local piece} + \text{branch state} \\ & + \text{absence of exits} + \text{minimality} / \text{carrier} / \text{type data}. \end{aligned}$$

The exactness requirement is load-bearing: the recorded data is frequently what kills the terminal residual, because the data a residual is forced to carry can contradict the exit absences that define it.

3.5 The general branch-closure algorithm

Three remaining terms are used below. The **local degrees of freedom** of a piece are the independent bounded modifications available at it; few degrees of freedom mean rigidity, hence structure. A local exchange is **neutral** if no compatible outside context distinguishes its two representatives (Section 2.21). A **carrier quotient** identifies pieces that share the same carrier data, collapsing the state space to finitely many classes. A **charge transfer** canonically relocates a demand from a saturated or ineligible payer to a designated high-degree neighbour, exactly as in a discharging rule [?].

Algorithm 1 (General branch-closure procedure).

Input. A minimal counterexample G , a target predicate T with its local-test family, and a branch state $\mathfrak{B}$.

Output. Either a closed branch, or a named residual with exact data that is smaller, finite, charged, or routed.

1. Define the local state space: list the bounded objects visible in $\mathfrak{B}$ (windows, cells, chains, payers, exchanges, and carriers) and assign finite labels to their local configurations.
2. State the branch dichotomy: either a local test is live, the local structure is rich, or the relevant capacity comparison closes; otherwise the failed alternative creates a recorded residual condition.
3. On a target-rich branch, close by the available counting, entropy, density, or direct target-realization argument.
4. On a target-poor branch, convert target-poorness into recorded structure: few local degrees of freedom, a repeated external type, a bounded chain, a neutral exchange, a carrier quotient, a charge transfer, or a routed residual.

5. Charge the recorded structure: define the demands, assign canonical payers, and prove the required bounded-multiplicity statement.
6. When a payer is overloaded, extract a labelled homogeneous subfamily and route the resulting matching, star, chain, or exchange by finite case analysis.
7. When a labelled state repeats, compare exact external types: a repetition visible to some context gives a target defect, while a repetition invisible to every context gives compression.
8. When the remaining state space is finite, enumerate it. Every leaf must be a target hit, a defect, a compression, a delocalization, an auxiliary branch, or a charging contradiction.
9. Update the branch table: every failure route becomes a named branch and every residual receives a typed name.

In Step 5, a **local obstruction** is a bounded structure that explains why a local test fails to fire at a given site. It is a candidate payer only after a canonical assignment and a bounded-multiplicity or capacity estimate have been proved.

The algorithm is a design discipline. A step closes a branch when it produces a closed leaf or a residual that is smaller, finite, charged, or routed.

3.6 Admission

A proposed tactic invocation enters the queue only after two admission tests.

Both-sides test. A proposed branch predicate or invariant must fill four blanks:

positive side:	what does it pay, and into which account?
negative side:	what bounded residual does it force?
measurability:	is the predicate recorded in $\mathfrak{B}$?
route:	which CT trigger receives the residual payload?

If any blank is empty, the proposal is not an admitted trigger; it is a parameter-synthesis task or a defect-register item.

Bookkeeping-vs-theorem test. A proposed step is admitted as a library instantiation only when it spends existing local resources: finite labels, bounded interfaces, local exchanges, ledgers, or recorded budgets. If the step requires an unconditional theorem about all large objects with a global property, it is not a tactic invocation. It must be replaced by a first failure,

finite state, compression, black-box input, or an explicit level-3 pattern addition.

Figure 23 displays the two admission gates.

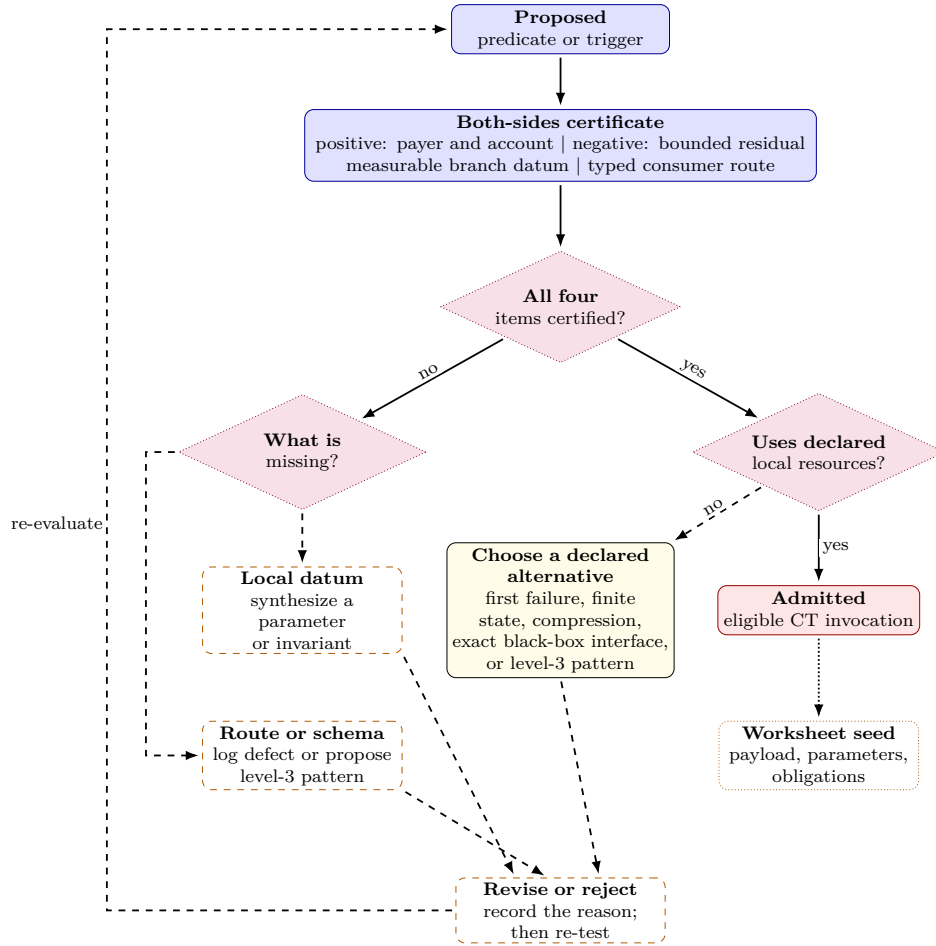


Figure 23: Strategy admission gates. The first gate checks the four both-sides fields; its failure identifies whether synthesis or pattern repair is needed. The second gate admits local bookkeeping and otherwise requires an explicit alternative rather than an undeclared global theorem.

3.7 Bookkeeping versus new theorems

A step is *bookkeeping* if it has the form

many local objects $\Rightarrow$ some label class is overloaded
 $\Rightarrow$ bounded exchange $\Rightarrow$ hit / defect / compression.

A step is a *new global theorem* if it requires an unconditional statement about all large objects with a given property — statements involving arbitrarily long spectra, global expansion, all substructures of a given kind, all possible lengths, density of special elements, or random-like distribution. Such a statement is not an existing tactic invocation. Record it as a cited black-box input with an exact interface, a new mathematical obligation, or a candidate for refinement into a first failure, bounded exchange, finite state, or compression. The last route is available only when its hypotheses can be proved.

This distinction is also the primary audit tool for detecting *displaced* difficulty (cf. Section 1.3, failure mode 3): a proposed auxiliary lemma in the second category restates the problem at a higher level. The self-test for a difficult branch is: “this branch must spend the existing resources of the proof.”

3.8 Structural invariants of the method

Design of finite labels. A finite label must include exactly what later routing reads. An admissible label records the boundary degree profile, ordered attachment type, local target offsets, short test-response flags, exit availability flags, the demand’s label in the charging scheme, and bounded-radius local geometry.

Absolute scale belongs in the label only for finite early cases; full path lengths enter only after reduction to a bounded increment or an exact external type; global object identity stays outside the label. If unbounded information appears, either truncate it and prove that routing uses only the truncated data, or promote it to an exact external type and use replacement/minimality.

The first-failure principle (quantitative form). When a branch fails across many scales, count the first failure. If the desired property is $h(k) \geq \rho k - C$ for all k , define $k_* = \min\{k : h(k) < \rho k - C\}$. Before k_* , the object has already paid its explicit charge or satisfied the desired condition; at k_* , minimality of the failure supplies a bounded local witness.

After k_* , further failures are counted only when they create new first-failure witnesses. This convention charges persistent obstructions once and handles multi-scale branches by their first local witness.

Compression is mandatory. The method rests on the invariant: *if a branch produces a smaller target-equivalent object, the branch is dead*. Every neutrality condition must therefore be tested against compression. A local modification that preserves the target response in every context and reduces size is a compression:

no hit $\Rightarrow$ target-complete $\Rightarrow$ replacement $\Rightarrow$ minimality contradiction.

This is frequently the decisive move.

3.9 Template for a new problem

Step A — Define local tests. Find a family of local tests equivalent to the target: $T \iff \exists$ local test realized.

Step B — Fix the minimal counterexample. Derive its deletion and replacement criticality.

Step C — Define cells and types. Equip every proper piece with an interface and an exact external type.

Step D — Build a supply–demand charging scheme. Identify what the counterexample must produce in bulk to avoid the target.

Step E — Split rich from poor. Rich branch: many active tests, closed by counting, entropy, or density. Poor branch: few active tests, forcing a first failure and hence structure.

Step F — Localize poor structure. Convert absence into a bounded object: a first-failure witness, exchange, chain, or carrier.

Step G — Close exchanges by the trichotomy: hit | defect | compression.

Step H — Close overloads via label classes: overloaded charging scheme $\Rightarrow$ monochromatic subfamily $\Rightarrow$ matching/star/chain $\Rightarrow$ exchange.

Step I — Close long structures via exact external types: long sequence $\Rightarrow$ repeated type $\Rightarrow$ defect or compression.

Step J — Enumerate finite leaves. Every finite leaf must carry an explicit certificate.

3.10 Audit checklist for branch design

For every proposed branch, answer all of the following. The branch is closed when every item has an explicit answer.

1. What is the branch state?
2. Which previous exits are absent?
3. Which local finite labels are retained?
4. What are the demands?
5. Who are the payers?
6. Is the charging assignment canonical?
7. What happens if one payer is overloaded?
8. Does overload produce a bounded exchange?
9. Does every exchange satisfy hit/defect/compression?
10. If there is a chain, what is its exact external type?
11. If a type repeats, what is the smaller replacement?
12. If a type changes, which witness is charged?
13. Are any unbounded parameters left?
14. Are the finite leaves explicitly enumerated?
15. Is every residual strictly smaller than before?

3.11 Choosing invariants and building budgets

This section is the generative core of the method. It turns existing proof resources into candidate invariants, budget functionals, branch orderings, and audit tables. Two design decisions in the method appear, from the outside, to require inspiration: the choice of the **stratifying invariants** — the predicates on which the state space is split into branches — and the construction of the **budget functions** — the quantitative accounts (deficiencies, surpluses, entropies, net charges) whose capacity inequalities close the counting branches. This section reduces both to procedure. The organizing principle is the same for both:

Design backwards from the resources. Start from “what can I already pay for, and which hypothesis would let me pay for it?”

Forward design invites the failure modes of Section 1.3: guessing the counterexample’s shape invites an unsupported global estimate, and inventing predicates ad hoc invites re-encoded difficulty. Backward design attaches a spending route to every invariant and every budget term. The resource inventory supplies admissible objects, the both-sides test turns them into

branch predicates, hypothesis extraction generates the invariant ladder, and backward rate computation turns the available supply and forced demand into budget coefficients. With those interfaces in place, the proof can carry many structural invariants at once because each invariant has a measurable payload, a position in the ladder, a positive-side account, and a negative-side route.

The resource inventory

Before any branch is created, compile the **resource inventory**: the list of everything the proof can already spend, each entry with its *price list* — what it pays for and at what rate. The standing categories are:

- *minimality tools*: deletion criticality and the replacement lemma with its uncompressibility corollary;
- *the target algebra*: the local tests, and any arithmetic structure of the target values (spacing, growth, residues) usable for target thickening (Section 2.31);
- *black boxes*, with their exact interfaces (Section 3.18);
- *existing charging schemes*, with their payers, current loads, and remaining capacities;
- *the finite label alphabet* and every finite enumeration already certified by code;
- *previously closed branches*, available as routes for residuals.

The inventory is a living artifact, maintained alongside the invariant register of Section 4.4. A proposed branch is admissible when every outcome can be paid from the inventory.

The both-sides test

A predicate P qualifies as a branch invariant precisely when **both** of its outcomes are spendable:

- the **positive side** must feed a quantitative account — an explicit charge, a density or entropy cap, a capacity inequality that moves closer to closing;
- the **negative side** must feed structure — a first-failure witness, a bounded exchange, a rigidity statement, or a residual with a named route.

The predicate P becomes a branch when both sides support progress. A

predicate that only names a distinction splits the state space without reducing it, which is the non-progress condition of Section 3.3. The test is applied *before* the branch is created, by writing one line for each side:

P pays ____ into account ____; $\neg P$ forces ____ routed to ____.

A branch proposal is ready when all four blanks have entries.

Candidate sources of invariants

One source of candidate invariants is **hypothesis extraction**:

Candidate invariants can be generated from the hypotheses of planned estimates and lemmas.

An invariant is extracted from a planned estimate or lemma. Take the estimate or lemma one wants to use, extract its hypothesis, and promote the hypothesis to a branch predicate. The positive branch applies the lemma. The failure branch is a new named branch *obligated in advance* to be closed from the inventory. Concretely, candidate invariants come from a short mechanical list, each item read off an object already present in the proof:

1. **Estimate hypotheses.** Every conditional estimate generates its own dichotomy: hypothesis holds (use the estimate) or fails (a named branch). This is one source of normalization dichotomies — density above/below a threshold, surplus above/below a bound — and it is what the sequencing discipline of Section 3.18 then orders.
2. **Ledger saturation.** Every charging scheme generates, at each payer, the split *saturated* (extract the overload structure of Section 2.23) versus *unsaturated* (apply the discharge inequality).
3. **Scheme availability.** Every conditional charging scheme generates the availability split of tiered charging (Section 2.27): the side condition holds, or its minimal failure obstruction exists.
4. **Rank.** Every family of tests destined to be charged generates the full-rank/rank-drop split of rank forcing (Section 2.29).
5. **Certificates.** Every finite labelling or certification generates *present/absent*.
6. **First failure.** Every monitored multi-scale inequality generates its first-failure index (Section 2.20 and Section 3.8).

7. **Label thresholds.** The finite alphabet itself generates size and degree thresholds: degree exceeding the largest labelled configuration, length exceeding the pumping bound of Section 2.22.

These operations generate candidates from recorded proof objects. A candidate becomes an admitted invariant only after its measurability, both outcome routes, and proof obligations have been checked.

Measurability, ordering, and granularity

Measurability. An invariant must be a function of data already recorded in the branch state: labels, external types, ledger loads, exit flags. If evaluating P would require unbounded new information, either truncate P to the recorded data (and prove routing reads only the truncation) or promote the missing datum to an exact external type first. This is the label-design rule of Section 3.8 applied to predicates.

Ordering. Invariants are arranged in a **ladder**: an invariant may be tested when everything both of its outcomes need is already standing. The practical order is nearly always: minimality consequences first (they are free), then normalization dichotomies (Section 3.18), then rank splits, then saturation splits, then certificate splits, and terminal packaging last. The ladder is recorded in the invariant register. The circularity rule of Section 3.18 applies: the failure branch of a normalization uses only estimates available before that normalization.

Granularity. Introduce a branch when its failure side requires a routing record distinct from every existing branch. Merge branches only when their payload type, consumer, inherited state, proof obligations, and progress record agree; otherwise retain them as separate branches. Split a branch when its outcomes require different routing records.

3.12 Budget synthesis and rate checks

Budgets: currencies and the normal form

Budget functions are assembled from a standard parts list of **currencies** — local quantities with known behaviour. For a baseline degree norm k (the hypothesis “minimum degree at least k ” or its analogue), the standing currencies are:

- *size*: $|V(X)|, |E(X)|$;

- *deficiency*: $\partial_+(X) = \sum_v \max(0, k - d_X(v))$, the total distance *below* the norm;
- *surplus*: $\sigma(X) = 2|E(X)| - k|V(X)|$, the total excess *above* the norm;
- *boundary mass*: the number of attachment points or external stubs;
- *rank currencies*: cycle rank, sparsity rank, and the target-rank of a test family;
- *entropy*: $\log_2$ of the number of admissible labelled configurations consistent with the branch state.

Deficiency and surplus are a matched pair: they watch the two directions in which the degree norm can fail. A budget containing one side of the pair must include a compensating mechanism for the other side; otherwise the adversary escapes the account by trading one failure direction for the other (the monotonicity audit of Section 3.13 exposes exactly this).

Every closing budget in the method then has the **normal form**

$$B(X) = \text{supply}(X) - \text{demand}(X) - \rho |X|,$$

where supply and demand are nonnegative combinations of currencies and ρ is a **rate**. Three properties are required and verified: *additivity* (Section 2.25), *monotonicity* under the proof's moves (Section 3.13), and *payability* — every unit of demand is assigned to a unit of supply by a canonical local rule, i.e., the budget is backed by a charging scheme in the sense of Section 2.18.

Computing rates backwards

The rate ρ is computed from two numbers on the branch at hand:

- the **demand rate** ρ_{dem} : the charge per unit size that the adversary is *provably forced* to pay on this branch (a lower bound from the charging scheme);
- the **supply rate** ρ_{sup} : the payment per unit size that the resources *provably provide* (an upper bound from packing, incidence, or enumeration counting).

If $\rho_{\text{dem}} > \rho_{\text{sup}}$, any ρ in the open interval $(\rho_{\text{sup}}, \rho_{\text{dem}})$ closes the branch. Choose the midpoint or the roundest number in the interval, and record the slack at both ends in the slack table (Section 4.3). Thresholds are obtained by solving supply = demand for the density or size parameter. The exact crossover value is the new quantitative branch invariant (Section 3.11, source 1), with the two sides of the crossover as its two branches.

If the interval is empty, the failure is informative: return to the inventory and either sharpen a supply term (a finer finite enumeration, a better extremal ratio over the label alphabet) or reroute part of the demand into a different account (a new tier, Section 2.27). The theorem is the *existence of the gap*; the numerical value of the rate is bookkeeping.

3.13 Monotonicity and design checklists

The monotonicity audit

For every move the proof can perform — deletion, replacement/compression, peeling (Section 2.26), charge transfer, handoff (Section 3.18) — record the budget’s response in a **moves $\times$ budgets table**. The table has one row per move and one column per budget, with each entry recording the sign of the change. Monotone accounting requires every available move to preserve the recorded deficit or decrease the adversary’s supply.

A violation identifies a budget-design defect. In the examples below it may reveal a missing surplus, boundary, or quotient correction. If no justified correction has the required sign, the budget must be rerouted or rejected. Thus a high-degree move may require a surplus term, a boundary-modifying move may require a stub correction, and a quotient move may require quotient-invariant demand or a route entry.

Budget separation and slack accounting

One unit of supply pays one demand, in exactly one account. The two standing separations are:

- **Structure versus entropy.** Dependence among tests is *structure* and is routed by rank forcing (Section 2.29); the certified-independent remainder is *entropy* and may be charged to the counting budget. Entropy is spent once, after rank forcing.
- **Competing demand families.** When two demand families draw on one supply, either prove their payers disjoint, or split the supply into explicit recorded fractions; the fractions enter the constant interlock of Section 4.3.

Maintain the **slack table**: every load-bearing inequality with its numerical margin, so that the binding constraint of the proof is recorded explicitly. A newly designed branch may spend only slack the table shows unspent; two branches independently consuming the same margin is the quantitative

analogue of double charging, and the table records this conflict.

The design checklists

These extend the audit checklist of Section 3.10 to the design stage. A proposal satisfying the relevant checklist is ready to serve as a branch or budget.

Invariant checklist (per proposed branch):

1. Which lemma’s hypothesis was this invariant extracted from?
2. What does the positive side pay, and into which account?
3. What does the negative side force, and to which route?
4. Is the invariant measurable from recorded branch-state data?
5. Is its position in the ladder after everything both of its sides need?
6. Does its failure side route differently from every existing branch?

Budget checklist (per proposed budget):

1. Which currencies appear, with which coefficients — and if the norm is two-sided, are both deficiency and surplus present?
2. Is the budget additive, with boundary corrections bounded per attachment point?
3. Was the rate computed backwards, with both endpoints of the interval and the resulting slack recorded?
4. Does the moves $\times$ budgets table show no adversarial increase under any available move?
5. Is the localization lemma stated for the budget’s admissibility class, and is that class closed under the local moves?
6. Is every demand paid from exactly one account, with entropy charged only after rank forcing?

3.14 Selection policy

Selection uses the inert data published by Section 2. The priority order is:

1. Consume the payload queue. A routed residual already carries a declared consumer trigger, so routing is forced except for recorded nondeterminism.
2. Among fresh proposals, run **check**-class tactics whose triggers are already recorded, such as CT1 hit checks, CT2 deletion tests, and CT9 bounded-fibre checks.

3. Run **synth**-class tactics whose required parameters already appear in the inventory.
4. Synthesize missing parameters using Section 3.16.
5. Use **loop**-class tactics and class-closing aggregate tactics after their measures, rates, and slack records are in place.

The invariant ladder refines this order inside a branch. Minimality consequences precede normalization dichotomies; normalization precedes rank splits; rank splits precede saturation splits; saturation precedes certificate splits; finite packaging comes last. The circularity rule is that a lemma may consume only invariants earlier on the ladder, except for a declared well-founded loop whose S-Meas obligation is recorded.

Granularity is part of policy. One branch should correspond to one routing principle. Merge branches when they share the same payload type, the same consumer trigger, and the same proof obligation; split them when a later argument reads different data from them.

CT	Policy skip or delay	Reason
CT3	Delay until the interface and context universe are explicit.	External types without declared contexts are disguised global arguments.
CT7	Delay in dense or unbounded-interface settings unless a finite context generator is known.	The exchange classifier is useful only when S-Equiv is locally decidable.
CT8	Delay until the label space and threshold are finite.	Pumping is a finite-state argument, not an informal periodicity claim.
CT11	Delay until additivity and admissibility closure are known.	Localization consumes a budget structure; it cannot manufacture additivity.
CT12 and CT13	Use after ordinary charge and type routes have been exhausted or queued.	Loops and tiers require measures, restored state, and slack discipline.

Figure 24 displays the selection workflow encoded by the priority order and delay table.

3.15 Nondeterminism inventory

Genuine strategic freedom is concentrated at a few nodes; below those nodes execution is forced by the payload type and the entry in Section 2.

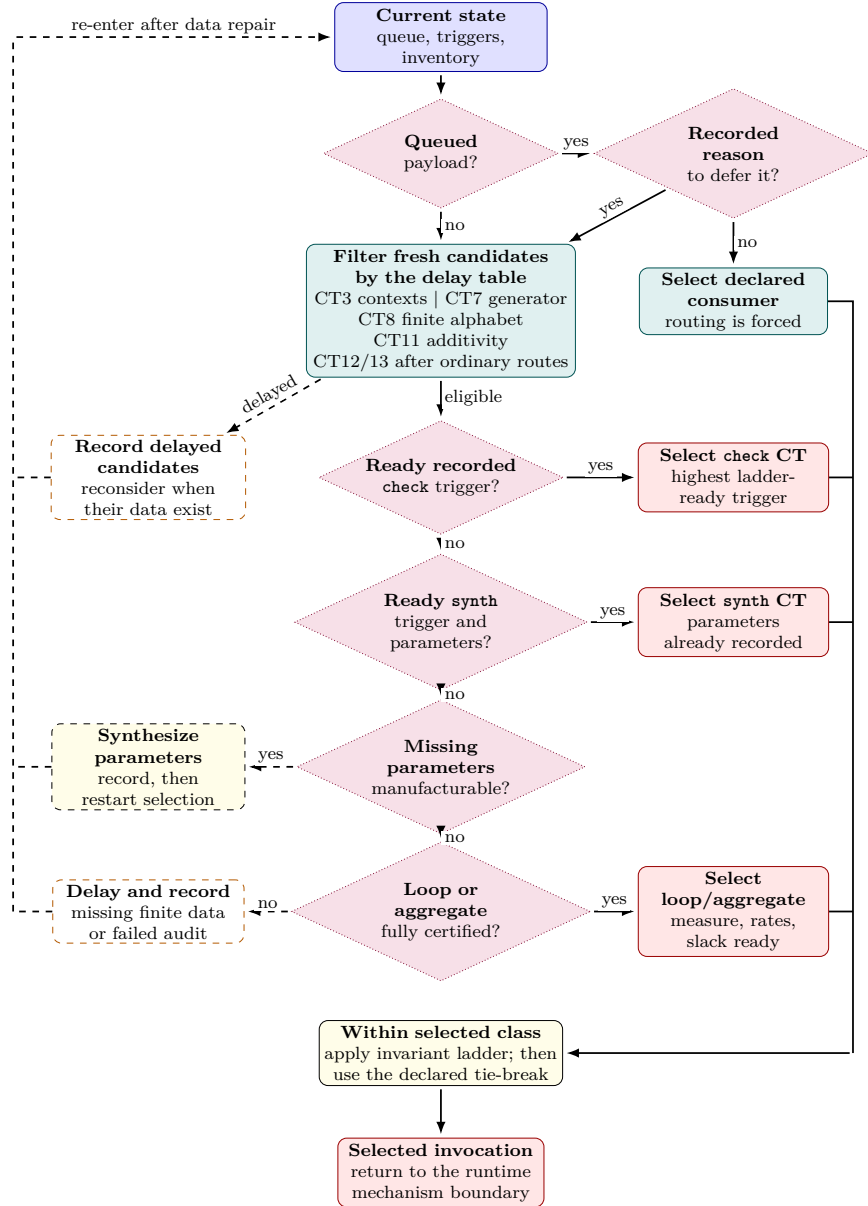


Figure 24: Strategy selection policy. A queued payload is consumed unless a reason to defer it is recorded. Fresh candidates first pass the CT-specific delay table; the first nonempty eligible priority class then supplies the invocation. Deferred candidates and parameter-repair tasks return to the top only after the missing data are recorded.

Node	Options	Deciding test
CT1 target-avoidance fan-out	CT2, CT3, CT4, CT5, CT6, or CT17 payloads.	Hypothesis extraction determines the available payload constructors; if several are certified, apply the selection priority, invariant ladder, and declared tie-break.
CT2 minimality route	deletion C2, replacement C2, separating-context $P_{2 \rightarrow 3}$, criticality-refinement $P_{2 \rightarrow 10}$, or response-defect $P_{2 \rightarrow 3}$.	Audit the interface, analyze deletion, certify a selected smaller replacement candidate, and classify the exhaustively uncompressible survivor.
CT5 local-to-global exit	C4 summation, charge ledger $P_{5 \rightarrow 4}$, localized deficit $P_{5 \rightarrow 11}$, or aggregate mass $P_{5 \rightarrow 14}$.	The certified local witness decides the route: payers give CT4, additive deficit gives CT11, and residual mass gives CT14.
CT7 neutral noncompressive branch	CT10 or CT16.	If the missing datum is local and finite, refine; if the support is the whole object with computable closed type, use exact types.
CT9 overloaded fibre extraction	Target hit, same-interface exchange, or long chain; any problem-specific matching or star is an intermediate configuration.	Not free once the homogeneous subfamily is chosen; the instantiation lemma maps the configuration to a declared CT9 output.
Overload label choice	largest class, extremal class, or predeclared canonical tie-break.	The default is largest class; any other choice must be part of the charge-map definition.
CT11 localized deficit route	target-test data $P_{11 \rightarrow 1}$, exchange data $P_{11 \rightarrow 7}$, admissibility gap $P_{11 \rightarrow 10}$, or carried mass $P_{11 \rightarrow 14}$.	After additivity and admissibility closure, the localized piece's certified structure determines the consumer.
CT13 fallback route	tier-one account $P_{13 \rightarrow 4}$, overlap fibre $P_{13 \rightarrow 9}$, or overlap mass $P_{13 \rightarrow 14}$.	The minimal obstruction decides the output: assignable charge, overfull overlap fibre, or residual overlap mass.

Node	Options	Deciding test
Aggregate versus pointwise treatment	CT14 aggregate closure, CT9 overload, or CT10 label refinement.	Select CT14 only when class mass, per-member capacity, and multiplicity records are available; use CT9 for a named overload and CT10 when the missing ingredient is a bounded finite capacity label. Once CT14 is admitted, its local audit returns $C4$, $P_{14 \rightarrow 9}$, $P_{14 \rightarrow 10}$, or the declared scope boundary.
CT16 exact-type escalation	C2 exact identification, proper-piece response $P_{16 \rightarrow 3}$, or distinct closed datum $P_{16 \rightarrow 10}$.	Escalate only when local support no longer separates the object and the closed type is computable.
Budget failure after empty rate interval	sharpen supply, split demand into a tier, or localize deficit.	The slack/interlock table identifies which inequality is binding.

This table is where the human mathematician or supervised model sits. The remaining work is delegated to admitted tactic invocations and their worksheets.

Figure 25 displays the main choice nodes in the nondeterminism inventory.

3.16 Parameter synthesis

Every Parameters slot in Section 2 points to one synthesis rule here. The rules are organized by parameter type. Choices of charging scale, activity threshold, finite alphabet, next missing invariant, rank notion, well-founded measure, and offset family are made here; the CT entries then consume the recorded data mechanically.

15.1 Resource inventory. Compile the available minimality tools, local test algebra, external black boxes, charging resources, finite label alphabets, type tables, and closed routes. Reject a proposal whose positive or negative branch cannot be paid from this inventory. Feeds: all triggers; especially CT2, CT4, CT5, CT11, CT15.

15.2 Invariants and labels. Use hypothesis extraction: the invariant is the hypothesis of the lemma one wants to apply, and the failure of that

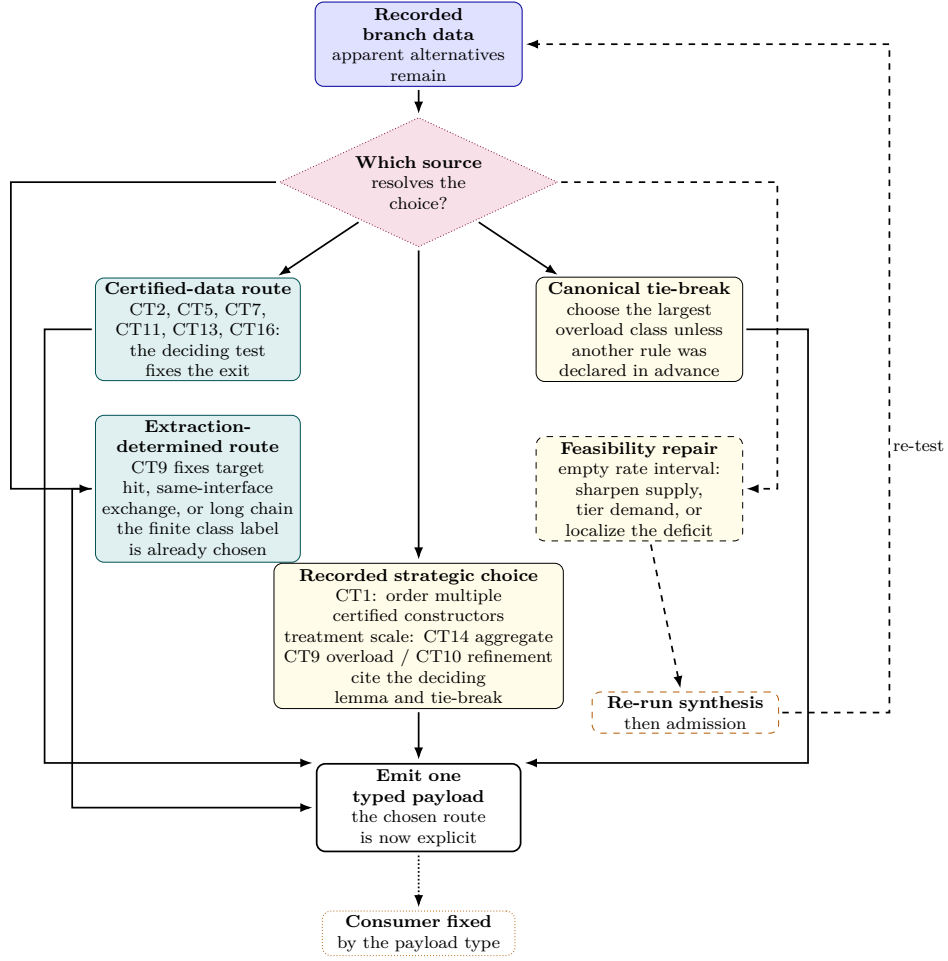


Figure 25: Strategy nondeterminism inventory. Certified residuals and extraction lemmas force most exits. Strategic freedom remains when CT1 certifies several constructors, when aggregate and pointwise treatments compete, or when a canonical label tie-break is required. Each such choice is recorded before one typed payload fixes the consumer.

hypothesis must already route to a CT payload. The seven mechanical sources are threshold equations, minimality consequences, type distinctions, label refinements, first failures, saturation conditions, and finite-table cutoffs. Feeds: label slots of CT8, CT9, CT10, CT13, CT17 and branch-state predicates for all CTs.

15.3 Types and generators. Produce certified finite context generators, exact external types, and closed whole-object types. The audit obligation is S-Equiv: the generator or type must decide all compatible contexts relevant to the target. Feeds: CT3, CT7, CT8, CT16.

15.4 Budgets and rates. Choose currencies, put the budget in normal form, compute the rate interval backwards, and record both endpoints and slack. Solved threshold equations become new invariants. Feeds: CT4, CT5, CT11, CT14, CT15.

15.5 Measures. A loop or cyclic handoff requires a well-founded measure that decreases on each pass after the branch state is restored. Feeds: CT12 and any cyclic combinator edge.

15.6 Synthesis audits. Run monotonicity of every move against every budget, separation of supply accounts, and slack-consumption checks before the tactic is selected. These audits feed the worksheet’s Artifacts and Open items fields.

Bijection claim. Every parameter slot in Section 2 must cite exactly one rule among 15.1–15.6, and every rule above lists its consuming CT slots. A slot with no rule is synthesis folklore; a rule with no consumer is unused bureaucracy.

Rule	Parameter family	Consuming slots	Inverse check
15.1	Resource inventory, CT1 witness inventory; CT2 admissibility class, minimality tools, replacement family.	CT1 witness inventory; CT2 size order, admissibility, replacements; CT4 payer resources; CT5 summation domain; CT11 admissibility class; CT15 independent coordinates.	Every entry using a proof resource cites the inventory before execution.
15.2	Invariants, finite labels, branch predicates, first-failure flags.	CT1 branch-state record; CT6 activity predicate and threshold flag; CT8 label alphabet; CT9 fibre label map; CT10 refined label alphabet; CT13 tier labels; CT17 survivor labels.	Every finite label or branch predicate is a recorded invariant or label source.

Rule	Parameter family	Consuming slots	Inverse check
15.3	Context generators, external types, exact closed types, response evaluators.	CT3 context universe/type evaluator; CT7 context generator and response evaluator; CT8 exact type equality test; CT16 closed-type evaluator.	Every context-equivalence or exact-type obligation has a generator/type rule.
15.4	Budgets, ledgers, capacities, rates, slack records.	CT4 canonical map/capacity/lower bound; CT5 local lower bound; CT11 budget; CT14 mass and capacity bounds; CT15 full-rank ledger; CT17 arithmetic threshold and residual-mass bound.	Every quantitative comparison reads a budget/rate/slack entry.
15.5	Well-founded measures for loops and cyclic routes.	CT12 peel measure; CT13 tier reconciliation loops; cyclic typed-handoff combinators.	Every repeated route carries S-Meas or is rejected.
15.6	Synthesis audits: monotonicity, separation, slack, restoration.	Audit hooks for CT4, CT5, CT11, CT12, CT13, CT14, CT15, and worksheet Artifact/Open-item fields.	Every synthesized parameter writes an audit row or a defect-register reject.

Figure 26 displays the parameter synthesis workflow.

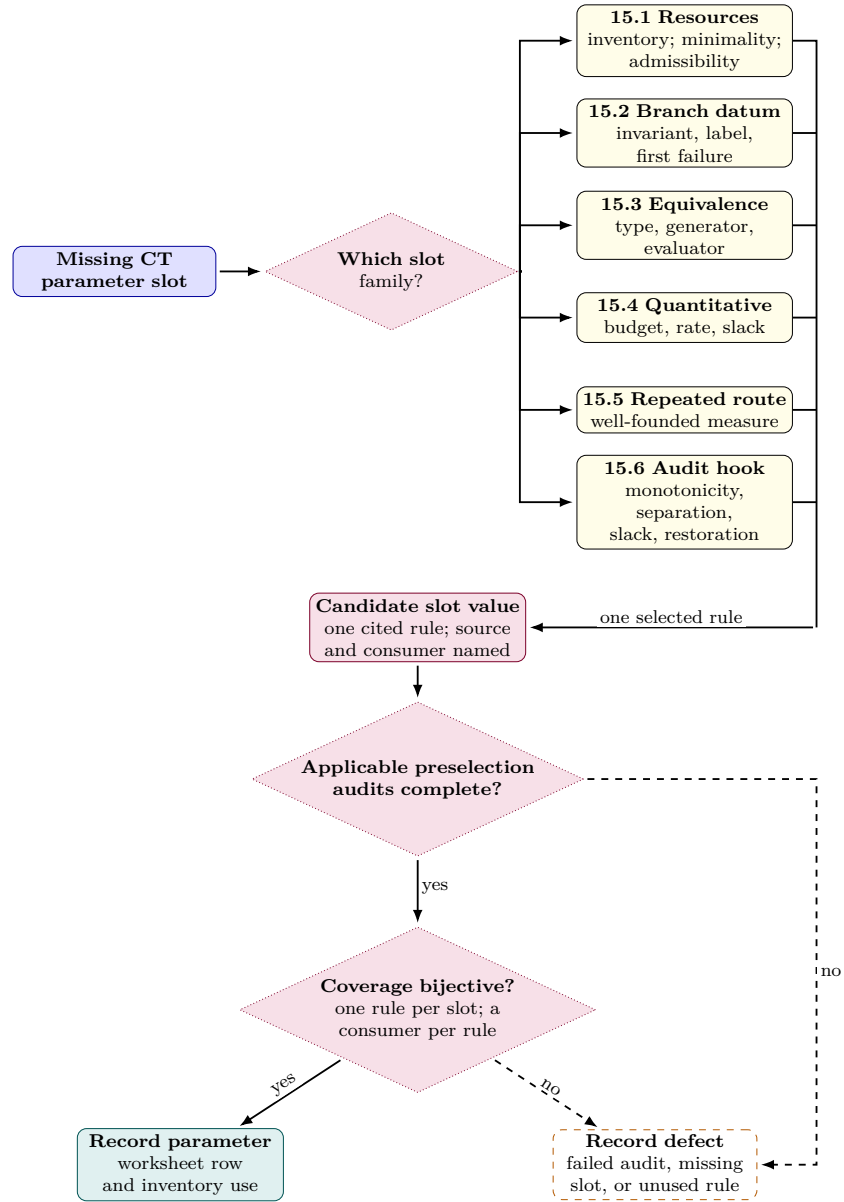


Figure 26: Strategy parameter synthesis. The slot type selects exactly one rule family among 15.1–15.6 for this filling pass. Outputs may become inputs to later passes—for example, a solved rate threshold becomes a recorded invariant—but each slot cites one rule. The resulting value then passes the applicable preselection audits and the slot–rule coverage check.

3.17 Combinators

Combinators organize tactic invocations but are not closure tactics, because they produce neither a certificate nor a smaller residual by themselves.

Combinator	Signature	Obligations	Formal counterpart
Sequencing / spine	ordered invocations $I_1, \dots, I_m$ with later triggers depending on earlier outputs	ladder order, no backward hypothesis consumption, S-Meas on cycles	proof script with dependency order
Typed handoff	payload $P_{i \rightarrow j}$ emitted by CT i and consumed by CT j	S-Rout at source, S-Trig at target, branch-table row	typed term passed between lemmas
Black-box isolation	external theorem plus constants and interface contract	cited theorem, constant interlock, trust-base register	imported axiom or theorem namespace

The admission test keeping a construct out of Section 2 is decisive: a CT entry must return C1–C5 or emit a legal typed payload. A cyclic route must additionally satisfy the measure requirement stated in its canonical entry. Otherwise the construct is a combinator or an assurance artifact, not a CT entry.

Figure 27 displays the combinator classification workflow.

3.18 Advanced closure patterns

The following patterns manage dependencies spanning several tactic invocations. They record sequencing, typed handoffs, or black-box imports together with the associated assurance artifacts.

Branch sequencing and derived normalizations

Estimates frequently hold under a normalization — a density, regularity, or budget hypothesis about G . The normalization is **derived**: the branch on which the normalization fails is exhausted *before* any estimate depending on it is invoked. The result is a **spine**: the surviving branch on which all normalizations have been either derived or made unnecessary, and along which the main quantitative estimates are valid.

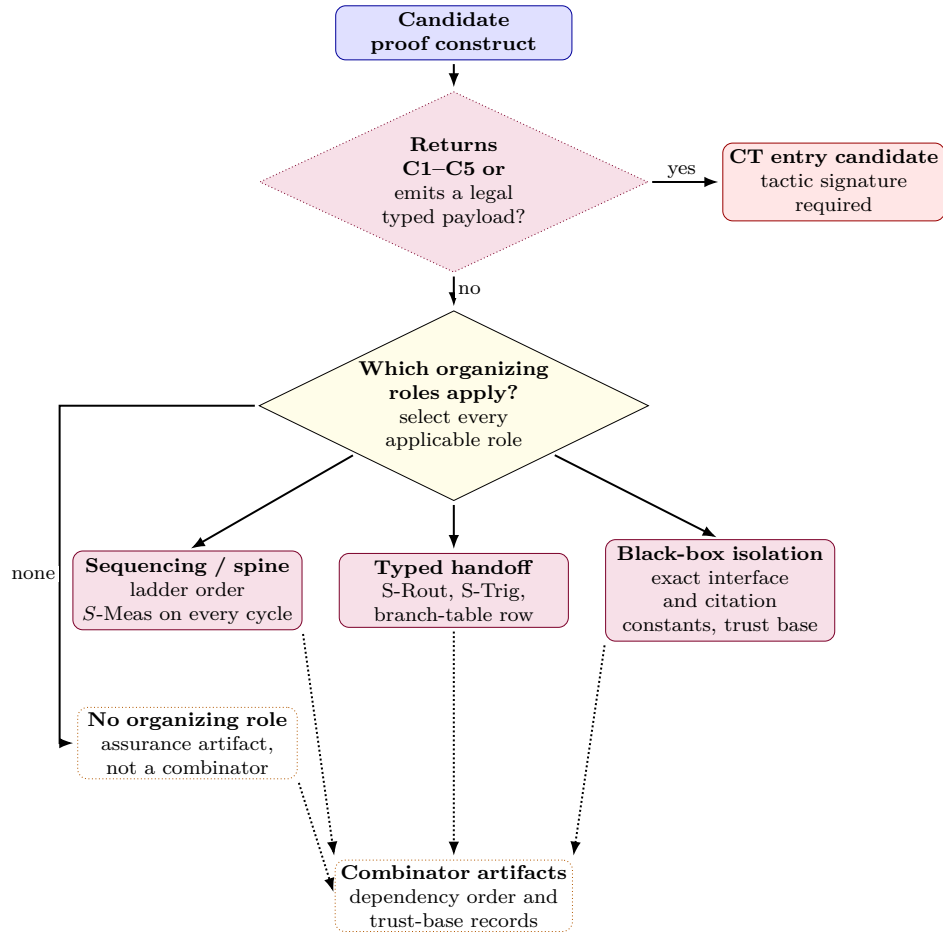


Figure 27: Strategy combinators. A construct is a CT candidate only when it returns C1–C5 or a legal typed payload; cyclic routes must also satisfy their recorded measure requirement. Otherwise the strategy selects every applicable organizing role. If no organizing role applies, the construct is an assurance artifact rather than a combinator.

The design rule is explicit sequencing: the proof declares the order in which conditional branches are discharged, and every estimate names the branch state on which it is valid. A normalization-failure branch must itself terminate in the standard taxonomy: a routed exit, a charged demand, or the derived normalization itself as its third alternative. The normalization-failure branch uses only estimates available before the normalization. The dependency order is part of the proof and is recorded in the artifacts of Section 4.4.

Typed handoffs between branches

Large branch systems require branches to delegate to one another. A **typed handoff** is an inter-branch interface with a declared data format. The source establishes that its output has the registered payload type and records the route; the receiving trigger must follow from that payload, and inherited branch data must remain available. The dependency artifacts (Section 4.4) record the handoff as a directed edge with its payload.

Handoffs may be cyclic at the diagram level (branch A hands to B, whose residual re-enters A’s saturation test). The declared measure then proves termination of the cycle as in Section 2.26; restored branch data must also match the state expected at re-entry. An invocation that uses another branch’s conclusion without the registered payload, trigger, or state transport remains an unresolved dependency.

Black-box isolation of external inputs

External theorems imported into the proof are **black boxes**. Each is stated once, with its exact interface (hypotheses consumed, conclusion supplied), and listed in a dedicated external-inputs register. The proof consumes a black box through its declared interface.

This serves three purposes: the trust base of the proof is enumerable at a glance; a strengthening or weakening of the external theorem can be impact-checked through the register; and external material remains cited at the point where the proof consumes it. The admission audit rejects an interface stronger than the cited theorem. The register entry cites the exact source statement.

In the Erdős–Gyárfás instantiation, the P_{13} -free theorem of Hegde–Sandeep–Shashank is the representative black box: it is used for precisely the hereditary-class conclusion it proves and remains isolated from unrelated branch closures [?].

3.19 Opening script and strategy narratives

An in-scope instantiation begins by admitting CT1 for the target predicate and local test family. If a test is realized, it halts at C1. On the target-avoiding branch, each certified datum proposes its corresponding payload: a proper piece to CT2, boundary response to CT3, charge ledger to CT4, local-site family to CT5, activity setup to CT6, or sparse offset data to CT17. If several constructors are certified, the selection policy and invariant ladder order the simultaneous proposals. Each choice is justified by the nondeterminism inventory, not by the CT1 entry.

A representative strategy thread reaches CT7 by

$$CT1 \rightarrow CT4 \rightarrow CT9 \rightarrow CT7.$$

The CT7 invocation then returns C1, C2, or C3, or one of the registered typed routes

$$\begin{aligned} P_{7 \rightarrow 3} &\rightarrow CT3, & P_{7 \rightarrow 12} &\rightarrow CT12, \\ P_{7 \rightarrow 10} &\rightarrow CT10, & P_{7 \rightarrow 16} &\rightarrow CT16. \end{aligned}$$

On this representative route, CT1 proposes the demand/payer payload only after a recorded local-density lemma supplies the demand family and its payers. CT4 is selected because the ledger is already synthesized; CT9 is forced when a fibre overload appears; CT7 is selected because the homogeneous fibre contains a bounded same-interface exchange. Once each invocation is admitted, its internal proof is the corresponding mechanism of Section 2.

Figure 28 displays the opening script and the representative strategy thread.

3.20 Filling the templates

There are two templates. The entry template is filled once per tactic, problem-independently, and belongs to Section 2. The instantiation worksheet is filled once per tactic invocation in a concrete proof and belongs to the artifacts. In worksheet templates, “parameter-synthesis rule” means one of the rules in Section 3.16.

```
Instance:      C Tk-#      Branch node: [n]
Input payload: <concrete objects>
               emitted by: CTj-# / opening script
Parameters:   slot (1) := concrete object
               synthesized by parameter-synthesis rule;
               inventory consumed: ...
               slot (2) := ...
```

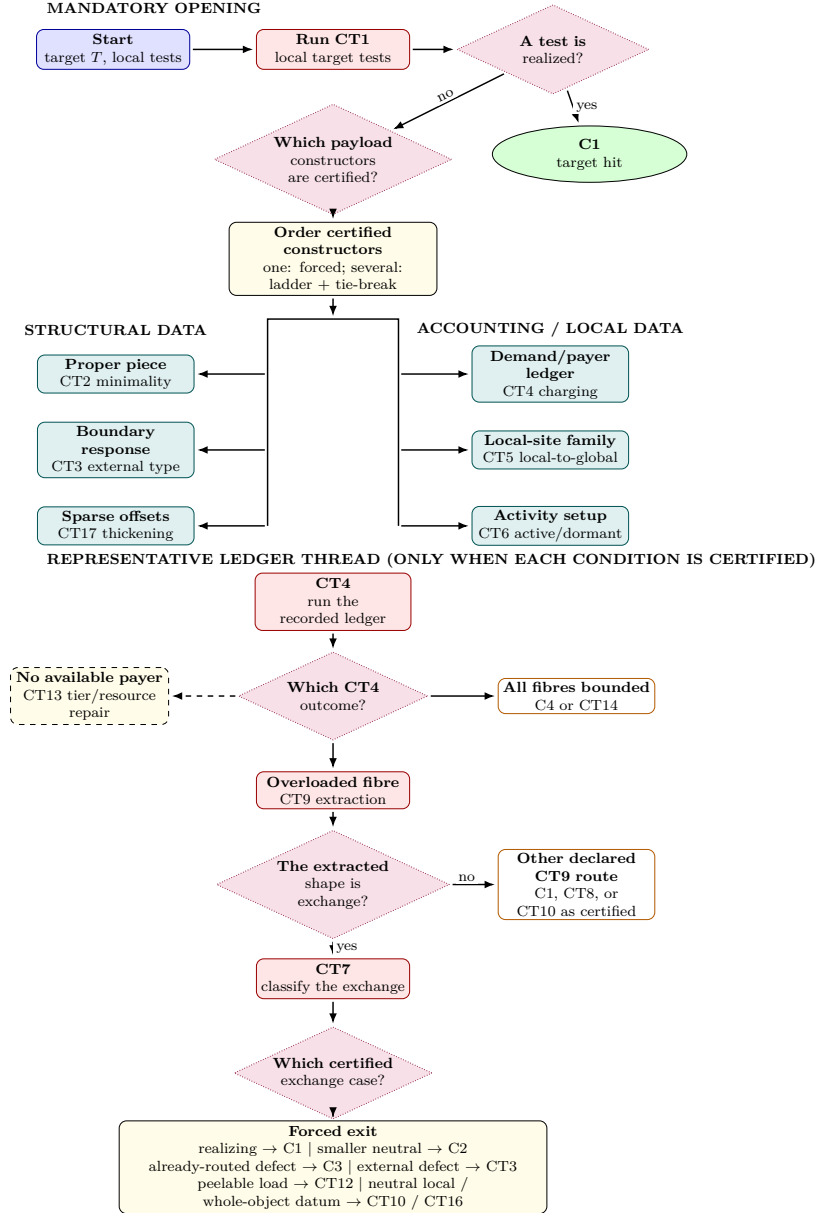


Figure 28: Strategy opening script. The upper flow first records which target-avoidance constructors are certified and uses the ladder only when several coexist. The lower flow follows CT4–CT9–CT7 only on the overloaded-exchange route; a missing payer goes to CT13, bounded fibres go to C4 or CT14, and the final panel lists every CT7 certificate or typed consumer.

```

Obligations:  S-Xxx<...> := Lemma/Prop label
               status: proved / pending
               ...
Outcome:      C1 | C2 | C3 | C4 | C5  | emitted <payload> -> CTm-#
Artifacts:    branch-table row #; invariant-register entries;
               ledger deposits;
               slack consumed (interlock line #)
Open items:   pending obligations -> defect-register hooks

```

The filling procedure is linear: take the typed input from the queue or from a certified opening branch state, fill parameter slots from Section 3.16, attach named lemmas to the obligations, record the certificate or emitted payload, and write the artifact rows. A worksheet with a pending obligation is not a partial proof; it is an open defect-register item naming the missing schema.

Figure 29 displays the worksheet filling workflow.

A new tactic entry is warranted at level-3 repair when a residual survives admission, no listed CT consumes it, and it is expressible in the current data language. Assign the next CT identifier and update the frozen-library range declaration. Draft the complete tactic entry, verify that every non-certificate output lands on an existing or newly added trigger, and update the master CT ledger, signature and obligation matrices, parameter-slot mapping, index card, payload table, and routing graphs. Add a toy or [EG] instantiation, apply the level-3 exit test, and close the defect-register item only if that test passes. A level-4 repair first extends the data language, then applies the same registration and exit tests to the new tactic family.

4 Assurance

Section 4 is the verification layer around the motivation, mechanism, and policy layers. It contains the certificate and branch-table discipline, the global checklists, proof-engineering artifacts, constant interlocks, failure semantics, the defect register, LLM operationalization, portability, and the scope limitations. It does not choose tactics and does not execute tactics; it audits that each admitted invocation, policy decision, and routed payload has been recorded.

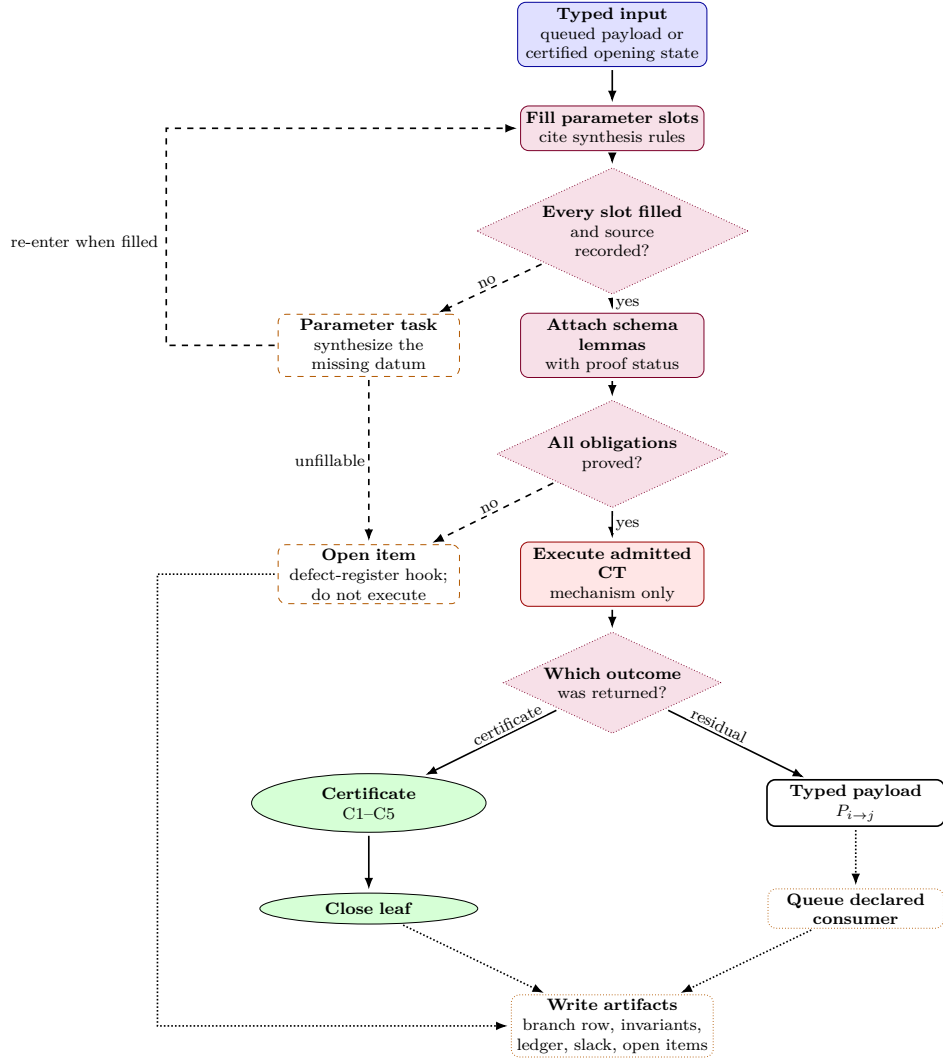


Figure 29: Strategy worksheet flow. Slot coverage and obligation completeness are separate pre-execution checks. Only a complete worksheet crosses the mechanism boundary; the returned outcome is then classified as a certificate or typed payload, while pending obligations stop at a defect-register item.

Assurance artifact	Verification role	Source
Branch table	One row per terminal branch or named residual; every row points to a worksheet or certificate.	Section 3.3
Worksheet list	One row per CT invocation, including pending obligations and emitted payloads.	Section 3.20
Dependency graph and table	Nodes, dichotomies, and lemma clusters; every leaf named.	Section 4.4
Invariant register and reverse index	Standing hypotheses and their consumers; supports blast-radius computation.	Section 4.4
Slack and constant interlock	Numerical constants, inequalities, margins, and finite computations.	Section 4.3
Resilience register	Reduction theorem that survives each hypothetical closure failure.	Section 4.5
Defect register	Structured record of discovered execution errors, design repairs, and pattern-level additions.	Section 4.6

4.1 Decoupling conformance checks

The following table defines release conformance for the mechanism/policy separation. M1–M6 and H3–H4 are normative release checks. H1–H2 are usability goals, not completeness theorems.

ID	Check	Passing condition
M1	Policy purity of entries	The CT ledger in Section 2 and entries contain no selection advice except the quoted forbidden-word list in the decoupling contract.
M2	Mechanism purity of strategy	Section 3 describes admission, selection, synthesis, and composition; it does not restate CT-internal procedures except in the worksheet template.
M3	Slot–rule bijection	Each parameter family in the CT ledger is covered by a rule in Section 3.16; each rule names its consuming tactics.
M4	Worksheet coverage	Each Appendix C trace row is either a reject row or has a worksheet-consistent tuple; CT2-1 and CT9-1 are fully displayed.

ID	Check	Passing condition
M5	Failure-classification compliance	Each scope-obstruction entry is a nonexistence statement; execution errors appear as audit hooks or defect-register rejects; design failures appear as routed payloads.
M6	Graph regeneration	The interlock graph is generated by the union of intended-output fields, and the repair graph by the union of recovery-output fields; the success sinks are exactly C1–C5, with C3 present.
H1	Mechanism usability goal	A reader can apply a CT entry from the vocabulary of Section 2 plus the CT entry itself without consulting Section 3.
H2	Policy usability goal	A reader can decide what to do next from Section 3 plus Appendix D, using only Block 1 and scheduler data from Section 2.
H3	Index-card coverage	Appendix D lists ID, name, tier, cost class, trigger, certificates, and residual edges for every CT.
H4	Worksheet auditability	The displayed worksheets identify input payloads, parameters, obligations, outcome, artifacts, and open items.

These checks are manuscript invariants. A contributor adding a CT, payload edge, worksheet field, or policy rule must use a declared extension namespace, record the library version and compatibility, and update every registry or derived view whose invariant changes.

4.2 LLM and formalization bridge

The formalization bridge of Section 1.4 extends to these artifacts as follows.

Artifact	Formal counterpart	AI-facing role
Instantiation worksheet	One proof-script step with named subgoals for each schema obligation.	Converts generated prose into statused obligations and defect hooks.
Selection policy	Tactic orchestration rule set, in the style of ordered tactic hints or an <code>aesop</code> -style rule base.	Restricts model freedom to admission, selection, and parameter synthesis.

Artifact	Formal counterpart	AI-facing role
Payload queue	Typed goal stack or worklist of unconsumed intermediate terms.	Prevents orphan residuals by forcing every emitted payload to be consumed or recorded.

4.3 Constant interlock and computational certificates

In a quantitative branch system, the numerical constants are load-bearing interfaces. The discipline has four parts. Every constant is *named*, displayed to full stated precision, and *established at a cited location*. Every finite enumeration behind a constant is reproduced by *published code*, so that the leaf is machine-checkable. The *interlock* records which inequality each constant must clear, and by what margin. The slack table identifies the binding constraint, namely the inequality with the least slack. A change to any constant can be impact-checked through the artifacts of Section 4.4. The proof records every constant at the precision used, so a constant used at higher precision than it was established, and unrecorded slack consumption — two branches independently “using up” the same margin.

4.4 Proof-engineering artifacts

A branch system beyond a handful of branches requires explicit audit artifacts, each of which mechanizes one clause of the audit checklist (Section 3.10):

- a **dependency graph** with uniquely numbered nodes — assertions, dichotomies, and terminal leaves — in which every dichotomy has both edges resolved and every leaf is a named closure or a named residual;
- a **dependency table** mapping node ranges to the lemma clusters that implement them;
- an **invariant register**: the numbered list of standing invariants (the accumulated branch-state facts), grouped by the stage at which each is established;
- **per-lemma requirement rows**: for each lemma, the invariants and prior results it consumes, so that every consumed hypothesis is recorded;
- a **reverse index** from each invariant to every lemma that consumes it — the impact-analysis table that answers “if this invariant weakens, what breaks?” by lookup.

These artifacts are the audit scaffold of the closure claim, and they support the AI-assisted workflow of Section 1.3 at scale: every one of the four failure

modes leaves a visible trace in at least one artifact.

4.5 Redundant closures and graceful degradation

Many long proofs behave conjunctively: the theorem is represented as the conjunction of its closing lemmas, so a false or incomplete lemma can create an unlocalized gap whose exact extent is difficult to determine without re-auditing the argument. The branch-closure architecture assigns a different semantics to failure. The central property is the **degradation invariant**:

At every stage of construction, audit, or repair, the document proves a true reduction theorem. A lemma failure changes *which* reduction theorem is proved; it never changes *whether* one is proved.

The failure-to-reduction transformation Suppose a lemma L closes branch B , whose residual class is $\mathcal{C}_B$, and suppose L is later found to be false or incomplete. Because the method forbids unnamed residuals (Section 3.3), $\mathcal{C}_B$ already exists as a defined object. It carries exact data: the branch state under which it arose, the baseline hypotheses, minimality, every standing invariant of the load path, the formal absence of all other exits, and the relevant minimality, carrier, and type data. Assume here that the branch partition is exhaustive, every other branch closure and every ancestor of the statement remain sound, and the reverse dependency data are complete. Under those assumptions, the surviving statement is:

Reduction theorem.

The main theorem holds unless a counterexample lies in $\mathcal{C}_B$.

The residual $\mathcal{C}_B$ inherits every invariant that the reverse index shows to be established upstream of L . This transformation is a consequence of the closure discipline: a residual that was named, typed, and routed while the proof was presented as complete is also a residual that can stand alone when a closure is retracted. The failed closure is replaced by a weaker theorem with an explicit exceptional class.

Declared dependency closure The damage caused by a failure is computable from the audit artifacts. The invariant register records which invariants L establishes; the reverse index maps each such invariant to the lemmas consuming it; the dependency graph gives the transitive closure.

This downstream set is the **declared dependency closure**, also called the blast radius, of L .

The architecture makes this set explicit, but does not imply that it is small. Branch states preserve upstream information along a load path; certificate dependencies are explicitly scoped rather than uniformly bounded-local; and typed handoffs (Section 3.18) give inter-branch dependencies declared payloads and edges. Terms such as “small” or “bounded” require a measured case-study result.

Redundant layers and slack absorption Two buffers may absorb a failure before it re-opens a branch. The first is **redundancy**: where the architecture provides arithmetically distinct closures for the same contradiction — an entropy cap, a rank squeeze, and a charge collision, for example — a branch re-opens only when every layer supporting that closure fails. A numerical repair in one layer preserves the formal content of the other layers.

The second buffer is **slack**. When the failure is quantitative, for instance because a constant is established with less margin than expected, the slack table (Section 3.13) determines whether recorded unspent margin covers the loss. In that case the repair is a re-computation rather than a redesign. The constant interlock classifies each quantitative failure as slack-absorbable, transferable to a redundant layer, or branch-reopening.

The repair protocol A re-opened branch is repaired with the same template that built the proof. Repair begins from the surviving verified ancestor state computed by dependency analysis: only upstream invariants independent of the failed result remain available, while the residual records where the difficulty lies. Repair proceeds through four levels.

Before choosing a level, open a defect-register row for the failed closure. Record the failed dependency-graph node and, when applicable, the schema, the residual class $\mathcal{C}_B$, the surviving reduction theorem, the upstream invariants inherited by $\mathcal{C}_B$, and the blast radius computed from the reverse index. Then classify the defect as an execution error (E), a repairable design failure (D), or a candidate scope obstruction. An omitted or incorrectly discharged finite-computation obligation is an execution error and starts at level 1. A schema or structural execution error starts at level 2. A verified numerical failure that emits a legal typed residual naming its binding term is a design failure and also starts at level 2. A legal design residual reaches level 3 only when its declared consumers and every existing pattern composition fail to

close it or advance its declared measure. A routing execution error requiring a new interface triggers a level-3 interface review, but is reclassified as (*D*) only after the payload-table entry and unique consumer are declared, the producer’s S-Rout obligation is discharged, and the consumer’s S-Trig obligation is recorded for discharge at admission. A candidate scope obstruction reaches level 4 and is classified as an intrinsic scope obstruction (*S*) only after the audit proves that the residual is not expressible in the current proof language. Repairs are attempted at the lowest applicable level and escalate only when that level’s exit test fails.

1. **Numerical repair.** This level applies when the definitions, branch partition, payload routes, and lemma statements remain valid and only a finite computation or quantitative value has failed.

Audit. Freeze the branch state and trace every affected number through the constant interlock, slack table, and reverse index. Check the value at the precision at which it is consumed, rather than at a rounded display precision. The audit distinguishes four cases.

- **Finite enumeration or computational certificate.** Reproduce the computation from its declared input domain. Verify the pruning rules, coverage of every terminal case, exact arithmetic, and the published certificate or checksum. Correct a code error, stale output, or transcription error and regenerate every dependent constant. The defect row records the checker identity and version, exact command, trusted inputs, and verification status. A correct computation with an incorrect domain or missing case is a structural defect and moves to level 2.
- **Constant or precision.** Re-derive the constant from the cited inequalities and compare its established precision with every use listed in the reverse index. Replace the value consistently in the constant interlock and all consuming inequalities. If the derivation itself uses a false hypothesis, incompatible currency, or omitted boundary term, move to level 2.
- **Rate interval.** Recompute ρ_{sup} and ρ_{dem} , including the strictness of both endpoints and every parameter on which they depend. When the interval remains nonempty, choose and record a new rate in its interior as in Section 3.12. An empty interval identifies a binding supply or demand term and moves to level 2.
- **Slack allocation.** List every consumer of the affected margin

and compare the list with the slack table. Spend only slack recorded as unassigned, or transfer the closure to an arithmetically independent redundant layer. A margin already spent by another branch, or two nominally redundant closures drawing on the same margin, is a level-2 accounting defect.

Exit test. Reverify the finite certificates and every affected inequality. Each must hold with its new recorded margin. Update the numerical fields in the constant interlock, slack table, invariant records, and dependent inequalities; the branch partition, lemma statements, payload signatures, and graph topology remain unchanged. Otherwise the repair escalates to level 2.

2. **Structural repair inside the pattern library.** Treat $\mathcal{C}_B$ as a fresh residual and apply Section 3.11 using the recorded workflow and existing tactic vocabulary. The repair may change invariants, labels, budgets, worksheets, or routes, but it uses only the existing proof language and closure taxonomy.

Audit and repair by failure type.

- **Failed hypothesis or stale branch state.** Use the per-lemma requirement row to find the first unavailable invariant, and use the reverse index to locate every consumer. Check measurability, ladder position, and whether the invariant was derived before it was used. Derive it from an earlier resource, weaken or restage the consuming estimate, and separately close the invariant’s failure branch by the both-sides test.
- **Ledger or budget defect.** Audit the canonical assignment rule, payer uniqueness, capacities, additivity, boundary corrections, moves $\times$ budgets table, and separation of currencies. Repair a missing compensating term or boundary correction; localize the deficit; split competing demands into recorded fractions; or use an existing tier, peel, or aggregate closure. A new tier is governed by Section 2.27, a peel by Section 2.26, and an aggregate bound by Section 2.28.
- **Label or type defect.** Check that the label alphabet is finite, measurable from the branch state, and fine enough to support every downstream dichotomy. Check that the context generator covers every permitted attachment and that exact-type comparisons use literal equality. Repair by refining the finite label,

enlarging the certified context generator, adjusting the threshold, or computing a sharper exact type. Label refinement follows Section 2.24.

- **Routing or schema defect.** Compare the worksheet input and output with the tactic signature, S-Rout, S-Trig, and the declared consumer's trigger. Discharge a missing schema obligation when the required datum is already present. When the route is already declared, construct the payload exactly as declared and verify that the producer output field, payload-table row, and consumer signature agree. Reverify the producer's S-Rout certificate, and record and verify the consumer's S-Trig obligation for discharge when that consumer is admitted; then regenerate the affected interlock and repair graphs. An untyped output, an output with no unique declared consumer, or a repair requiring a new producer field, payload-table entry, or consumer signature is an execution reject. Record it without emitting or queuing it, and send it to the level-3 interface review.
- **Progress or termination defect.** Verify S-Meas on every cyclic or recursive non-closing transition, and verify S-Rest when a transition restores branch state. An acyclic handoff requires S-Rout and S-Trig but need not decrease a measure. Repair a stationary cycle by splitting it at the next routing principle, refining its descriptor, or applying the existing peel, tier, or well-founded measure. A cycle with no decreasing measure cannot be admitted.
- **Black-box interface defect.** Compare the registered hypotheses and conclusion with the exact cited statement and with each consuming requirement row. Weaken the interface to the cited theorem and weaken or restage every consumer whose requirement exceeded it. Alternatively, supply a missing hypothesis from the branch state, branch on that hypothesis and close its failure side, or route the missing-hypothesis obligation to a declared producer. A stronger unconditional statement is not a repair inside the existing library.

Exit test. Re-run admission and selection, the invariant and budget checklists, the moves $\times$ budgets audit, affected worksheets, and S-Rout and S-Trig for each regenerated route. Every leaf must return C1–C5 or a typed payload to an existing consumer. Keep at level 2

any execution error correctable without an interface change. Send an interface-changing routing reject to level-3 review; it remains (*E*) until its payload-table row, unique consumer, producer’s S-Rout certificate, and consumer’s S-Trig obligation are recorded. Move a legal, expressible residual unmatched by the taxonomy to level 3. Record an inexpressible datum as a candidate scope obstruction and send it to the level-4 audit.

3. **Pattern-level repair.** This level applies when $\mathcal{C}_B$ is expressible using existing descriptors and schema obligations, but no pattern in Section 3.4 or Section 3.18 closes it.

Audit. Normalize the residual by recording its descriptor inventory, input payload, inherited invariants, live exits, available certificates, current typed consumers, and progress measure. Check the existing taxonomy after permissible relabelling, sequencing, typed handoff, and black-box composition. First distinguish legal recurrence from an execution error: a return with unchanged state and no decrease on a required measure is rejected, whereas a legally decreasing recurrence may indicate a missing pattern. The following signatures support a pattern-level repair: the same normalized routing defect recurring after the level-2 audit; a legal typed residual repeatedly returned by its declared consumers without closure; the same defect class recurring in two or more branches; a residual that repeatedly stops shrinking; a branch whose cases share a new routing principle; or a closure assembled from existing tactics but absent from the pattern library.

Repair by failure type. Merge branches whose failure sides have the same route, and split a nonshrinking branch at the first routing principle that distinguishes its outcomes. Repair a routing defect requiring a new interface by declaring one abstract producer–consumer contract. A legal residual returned without closure requires a pattern with an explicit decreasing measure. When a closure is assembled from existing tactics, record its sequencing and typed handoffs as a combinator and add its reusable closure statement as an advanced pattern. For every recurrent residual, state one abstract pattern whose input is the normalized residual signature. The pattern must declare its trigger, inherited data, closure certificate, non-closing typed outputs and their consumers, scope obstruction, well-founded measure, and artifact writes. Add the pattern to Section 3.18, instantiate it on $\mathcal{C}_B$, update every affected producer output field, payload-table entry, and consumer signature. If the repair adds a tactic, assign the next CT identifier and update

the frozen-library range declaration; register a complete entry with its signature, trigger, parameters, procedure, obligations, certificates, outputs, audit hooks, and scope obstruction, and update the master CT ledger, signature and obligation matrices, parameter-slot mapping, and index card. Discharge the producer's S-Rout obligation, and record and verify the consumer's S-Trig obligation for discharge when that consumer is admitted. Then regenerate the dependency, interlock, and repair graphs together with the branch table and worksheets. Once the payload-table entry, unique consumer, and S-Rout certificate are present, reclassify the routing reject from (E) to (D) and admit its payload; the consumer discharges S-Trig when it accepts that payload.

Exit test. Every outcome of the new pattern must close at C1–C5 or enter an existing consumer through a typed payload, and every loop must decrease the declared measure. If the pattern cannot be stated using the current descriptor types, the obstruction moves to level 4.

4. **Language extension.** This level applies to a candidate scope obstruction: the needed object may not be expressible by the current local tests, bounded interfaces, finite context generators, finite labels, additive budgets, or measurable loads.

Audit. First prove nonexpressibility in the current language by identifying the missing datum and showing that no existing descriptor records it. A missing lemma or untried pattern is not a language obstruction. Once nonexpressibility is established, classify the defect as an intrinsic scope obstruction (S) . Then determine whether the proposed datum admits a finitely encoded descriptor hierarchy, canonical extraction from the branch state, compatibility with the declared gluing or decomposition operations, finite certificate semantics, the applicable S-Pers and S-Equiv obligations, and a well-founded progress measure. These conditions decide whether the obstruction is repairable at level 4 or terminal. For cut and energy residuals, the witnessed set or Rayleigh function may be global, but its routed descriptor record must be finitely encoded: for example, by volume, boundary mass, conductance scale, side profiles, and recursive obligations.

Repair by failure type. For a missing local test or bounded interface, declare the test family, attachment data, and finite context generator. For a missing label or type, declare the finite label or finitely encoded descriptor record and its equality or refinement rule. For a missing budget or load, declare its currency, additivity domain, canonical extraction,

and measurable progress quantity. Every extension must specify its descriptor type and payload format; auxiliary finite certificate data; the existing terminal class among C1–C5 that validates each closing leaf; schema obligations; parameter-synthesis rules; producer and consumer tactics; and non-closing routes back to the existing interlock graph. Assign each new tactic the next CT identifier and update the frozen-library range declaration. Register every new producer or consumer as a complete CT entry with its signature, trigger, parameters, procedure, obligations, certificates, outputs, audit hooks, and scope obstruction. Update the master CT ledger, signature and obligation matrices, parameter-slot mapping, index card, tactic output fields, payload table, consumer signatures, branch table, worksheet fields, assurance checks, quick-reference card, and limitations. Discharge each producer’s S-Rout obligation, and record and verify each consumer’s S-Trig obligation for discharge at admission; then regenerate the interlock and repair graphs before using the new language.

Exit test. Re-run the level-2 closure procedure in the extended language. The extension is admitted only when every output closes or routes, every cyclic or recursive route decreases the declared measure, and S-Rest holds whenever a route restores branch state. If no finitely encoded descriptor hierarchy equipped with a well-founded measure exists, record a terminal scope obstruction and retain the surviving reduction theorem. Its defect-register row remains open with disposition **terminal-S**; it is not closed while that theorem remains conditional. Here terminal means terminal for the current language only; it is neither a closed proof leaf nor evidence of leaf-totality.

Every attempted repair updates the defect-register row before the proof is rerun. For each attempt, append its audit evidence, repair level, pre- and post-repair classification and disposition, exact edit, exit-test result, post-repair dependency set, and reduction theorem currently in force. The blast radius at discovery remains immutable. A defect is closed only after the applicable exit test passes; until then the row remains open and the current residual class, initially $\mathcal{C}_B$, remains among the exceptional classes named by the recorded reduction theorem.

At levels 3 and 4, the typed residual becomes either a reusable closure pattern or vocabulary in which later patterns can be stated.

The resilience register and the reduction lattice The **resilience register** makes the degradation invariant auditable. It has one row per intermediate branch, recording the branch state, the residual class published if its closure fails, the blast radius computed from the reverse index, the surviving reduction theorem, and the anticipated repair level. Maintaining the register is inexpensive because each column is read from artifacts already required by Section 4.4.

The register also gives the global structure of a proof in progress. The proved reduction theorems form a lattice ordered by inclusion of excluded residual classes. Verifying a closure moves the proof upward in the lattice; retracting a closure moves it downward by one computable step; the unconditional theorem is the top element. A referee who has checked k of N branches has established the reduction theorem corresponding to those k closures, unconditionally. The same containment governs the AI-assisted workflow of Section 1.3: a fabricated or incomplete step affects one node, is detected by the artifacts, and lowers the lattice position by a specified step.

Register integrity. Graceful degradation is a property of the maintained architecture. A stale reverse index reports a false blast radius, and an undeclared handoff introduces an untracked dependency. Both errors convert a mesh of reductions into a chain of implicit assumptions. The degradation invariant also requires exact residual statements: a vague claim with no corresponding element in the reduction lattice is not a surviving reduction theorem. Finally, redundant layers must be arithmetically independent. Two closures that consume the same slack margin or the same invariant form one layer with two names; the slack table and reverse index are the instruments that detect this collapse.

4.6 The defect register: recording discovered failures

The resilience register of Section 4.5 records what survives under hypothetical failures. Its temporal complement is the **defect register**: the structured record of every defect actually discovered during construction or audit, together with the way the document absorbed it. The resilience register is predictive; the defect register is evidentiary. It turns the assertion that a manuscript has been audited into a checkable object.

Each entry has a fixed schema.

1. **Identifier.** A stable ID, with defects grouped into lettered families by kind.

2. **Location.** The affected dependency-graph node or nodes and the affected lemma or lemmas. Nodes are used rather than page numbers, since pages drift and nodes are stable.
3. **Classification.** The formal class (E), (D), or (S), or the provisional status “candidate scope obstruction,” together with the concrete subtype and the failure mode of Section 1.3 or checklist item of Sections 3.10–3.11 that the defect violates. Typical subtypes include quantifier-scope collapse, phantom ledger entry, double-charged token, currency mismatch between accounts, and unrecorded exit dependence. For tactic invocations, the classification also records the CT instance and schema name, for example CT7-3/S-Equiv or CT12-1/S-Rest.
4. **Residual and surviving theorem.** The current residual class, initially $\mathcal{C}_B$, its inherited upstream invariants, and the exact reduction theorem that remains in force while the defect is open.
5. **Audit evidence.** The failed obligation, computation, dependency, or nonexpressibility statement supporting the classification, together with the residual’s normalized failure signature when level 3 or 4 is under consideration.
6. **Blast radius at discovery.** The downstream set computed from the reverse index as in Section 4.5, recorded before the repair is made and never overwritten by later repairs.
7. **Repair history.** An append-only sequence with one entry per attempt. Each entry records the repair level, pre- and post-repair classification and disposition, exact edit (including any invariant added, weakened, or re-staged), exit-test result, resulting downstream dependency set, descendant residual, and reduction theorem then in force. A later attempt never overwrites an earlier one or the blast radius at discovery.
8. **Status.** Open or closed. An open defect records the reduction theorem currently in force, which is the degradation invariant applied to the audit history. A terminal scope obstruction remains open with disposition **terminal-S**; it is not a closed defect.

Three disciplines make the register load-bearing. First, every repair cites its defect ID, so the proof’s edit history is navigable by defect rather than by line number. Second, open defects are explicit entries: a register listing only closed defects is incomplete, while a complete register states the current lattice position of Section 4.5. Third, recurrence triggers pattern or language review: two or more defects in the same class signal that the pattern library is missing a closure shape, or that the current vocabulary lacks the descriptor in which the closure should be stated. The appropriate repair is then a new

Section 3.18 pattern or a level-4 language extension. In this sense the defect register supplies empirical input to the method: it records which repairs recur often enough to deserve names or new data types.

The register also closes a loop with the AI-assisted workflow of Section 1.3. Each defect classification names the failure mode that produced it, so the register becomes a calibration record for machine assistance: it records where the assistance failed and whether the proof artifacts caught the failure at the intended layer.

Scope-obstruction escalation rule. Every non-closing branch is classified before it is allowed to terminate. An execution error (E) means that a named obligation is unfilled or incorrectly discharged, including a computational certificate; the register records the CT instance, the schema or certificate, and the missing or invalid derivation, and the repair is to discharge that obligation correctly. A repairable design failure (D) means that the failed datum is a typed residual; the register records the emitted payload and follows the corresponding output edge in the interlock graph. An intrinsic scope obstruction (S) means that the needed bounded object is not expressible in the current proof language: no local test family, no bounded interface, no finite context generator, no finite label space, no additive budget, or no measurable load. The defect register then decides whether the obstruction admits a finitely encoded proof-language extension and a well-founded progress measure. If it does, the repair is level 4; if it does not, the obstruction is terminal and routes to the limitations section Section 4.9. Inside an admissible proof language, the framework has unfilled or incorrectly discharged obligations, legal routed design residuals, and routing execution rejects, not terminal scope obstructions.

Register-location invariant. A register keyed to prose locations decays with every revision; node-based locations are stable. A register maintained only at release time records the repairs but not the blast radii at discovery, which is the evidence needed to validate the bounded-blast-radius claim of Section 4.5.

4.7 Repair taxonomy and common failure shapes

The repair protocol above separates failures by the contract that breaks. Every failure in this manual has the same core shape: a declared resource is missing, stale, misrouted, or not expressible in the current language. The difference is which declaration fails first. Some failures are local computational

mistakes; some are structural mismatches inside the current pattern library; some are legal residuals that need a different pattern; and some exceed the current language entirely.

Failure family	What breaks	First repair move	Prevention rule
Execution error (E), level 1	A finite computation, constant, precision, or certificate is wrong or stale.	Recompute at the consumed precision, refresh all dependent values, and update the constant interlock, slack table, or certificate row.	Record the source computation, the exact precision, and every consumer before the value is used.
Structural execution error (E), level 2	A hypothesis, branch state, route, or schema obligation is missing, outdated, or inconsistent.	Restage the missing invariant, repair the route table, or discharge the schema against the current branch state.	Keep the dependency graph and reverse index current, and verify S-Rout and S-Trig at every handoff.
Repairable design failure (D), level 2	A legal typed residual has appeared, but the current tactic library has not yet consumed it.	Send the residual to an existing consumer by a declared payload, or move the residual to CT4, CT10, CT13, CT14, CT15, or CT17 as appropriate.	Declare the output field, payload tuple, and unique consumer at the moment the residual is introduced.

Failure family	What breaks	First repair move	Prevention rule
Pattern gap (D), level 3	The residual is legal and expressible, but no existing pattern closes it.	Abstract the recurring residual into a reusable pattern, then register the new combinator or CT entry and regenerate the dependency graphs.	Watch for repeated residual signatures, repeated nondecrease, or repeated routing defects across branches.
Language gap (S), level 4	The needed object cannot be expressed with the current finite interfaces, labels, or descriptors.	Extend the language with a finitely encoded descriptor hierarchy and a well-founded progress measure, or record the obstruction as terminal.	Check expressibility before proof search begins: if the needed datum has no declared descriptor, no local repair will finish the job.

Three common invariants run through the table. First, the defect register is append-only: the blast radius at discovery is part of the evidence, not a detail to be reconstructed later. Second, every repair is local to the failure class: numerical repairs do not change the interface; structural repairs do not change the meaning of the theorem; pattern repairs add a new reusable shape; language repairs add a new descriptor language. Third, prevention is mostly bookkeeping discipline. Keep the declared consumer fixed, keep the payload signature explicit, record the exact measure or precision at which the object is consumed, and rerun the relevant audit checks before the object is admitted downstream.

For framework design, the useful summary is short. If the defect is numerical, tighten the arithmetic and update dependent constants. If the defect is structural, restore the missing hypothesis or route. If the defect repeats,

factor it into a pattern. If the needed object is inexpressible, the framework needs a language extension rather than a local fix. That separation is what keeps the repair ladder finite and the taxonomy reusable.

4.8 Portability

The method is designed for classes of objects in which there is a target predicate; local modifications can be compared against outside contexts; minimality yields a replacement principle; branch states admit finite labellings; and global constraints are expressible as charging schemes.

Candidate domains include extremal graph theory, Ramsey-type problems, colouring and list-colouring, forbidden-substructure problems, minors and immersions, additive-combinatorial configurations, matroid representability obstructions, and constraint satisfaction and finite model theory. The portability claim is strongest in settings where the comparison literature already has traction: local reducibility, finite interfaces, representative replacement, and charging/discharging all need local definitions compatible with the target predicate [?????]. This manual demonstrates the graph setting; each other domain requires its own verified interfaces, types, replacement rules, and capacity arguments.

One common base-language pipeline is:

$$\begin{aligned} &\text{minimal counterexample} \Rightarrow \text{local tests} \Rightarrow \text{branch states} \\ &\quad \Rightarrow \text{charging scheme} \\ &\Rightarrow \text{overloaded label class} \Rightarrow \text{bounded exchange} \\ &\quad \Rightarrow \text{hit / defect / compression.} \end{aligned}$$

Direct target, minimality, and finite-exhaustion branches may bypass the charging, overload, or exchange stages.

Equivalently,

Every way of avoiding the target costs a scarce resource,
violates an accumulated exclusion,
or creates a repeatable local operation.

Scarce resources are charged by charging schemes, exclusions are spent by branch-state dichotomies, and repeatable local operations are closed by hit,

defect, or compression. This is the core mechanism.

4.9 Limitations

The method has a precise scope condition and a nontrivial cost. Both are part of the framework. This section records where the machinery applies and what it charges when it does.

The scope condition: locally testable targets

The base tactic library consumes bounded-radius, bounded-interface data: local tests, external types, finite labels, charging schemes, and the exchange trichotomy. This gives the base proof-language scope condition.

Base proof-language scope condition. The target must admit a local test family (Section 2.15) whose bounded-interface types have finite Myhill–Nerode index under gluing. In graph-sequence language, the target must be visible to local Benjamini–Schramm data rather than only to a global invariant.

Substructure targets satisfy this condition by construction. A cycle, minor, colouring obstruction, or forbidden configuration has a bounded witness, so its presence is a bounded-radius event once the relevant interface is recorded.

The **spectral gap** is the canonical witness that the base proof language is proper. A d -regular expander and a d -regular graph separated by a sparse cut can have identical bounded-radius statistics up to $o(1)$; both may look locally tree-like, while their global expansion behaviour differs. No family of bounded forbidden-pattern tests is therefore equivalent to expansion. The obstruction is not a bounded pattern but a global bottleneck: a set S with

$$\Phi(S) = \frac{|\partial S|}{\text{vol}(S)}$$

small, or equivalently, up to Cheeger-type inequalities, a low-energy Rayleigh witness for the second Laplacian eigenvalue [?].

This observation does not put spectral-gap arguments outside the framework. It identifies the descriptor missing from the base proof language. The witness S may contain half the graph, but the residual needed by the proof is a finitely encoded descriptor record: volume, boundary mass, conductance scale, side profiles, boundary interface, inherited branch constraints, and the recursive obligations on the two sides. Once that record is part of the proof

language, the spectral failure has the same form as the other residuals in the document: a typed payload with declared routes.

Proof-language scope condition. A target is admissible in a proof language when every branch admits certificates with finitely encoded descriptor records and a well-founded progress measure. The base proof language uses bounded-radius local descriptors; proof-language extensions may use global witnesses whose descriptor records are finitely encoded and whose recursive obligations are measurable.

The standard expansion toolkit motivates an interface of this shape. The cut-matching game gives a dichotomy: either the matching player constructs an expansion certificate, or the cut player emits a sparse-cut payload [?]. Expander decomposition iterates the same dichotomy: certify an expanding core or extract a cut and recurse on the sides, using volume as the well-founded measure and charging boundary edges across the recursion [?]. Cheeger’s inequality supplies the S-Equiv bridge between the analytic spectral statement and the combinatorial cut certificate. In the vocabulary of this document, these results supply mathematical ingredients; the citations do not by themselves discharge the descriptor, routing, or trigger obligations. The following is therefore a proposed, unregistered extension sketch rather than an admitted tactic family.

Extension sketch: CT-Cut/CT-Energy. The input is a branch state $\mathfrak{B}$, a graph G , and a target expansion scale γ . The proposed tactic would first record the conductance or Rayleigh currency. It would then prove an equivalence lemma converting failure of the gap into either a cut descriptor

$$\langle S, \text{vol}(S), |\partial S|, \Phi(S), \partial S, \text{side profiles} \rangle$$

or an energy descriptor that can be rounded to such a cut. A positive outcome would close by a flow or expander certificate. A negative outcome would emit a typed sparse-cut residual whose routes are: bounded separator, recursive decomposition on the two sides, expander-core handoff, flow dual certificate, or contradiction with an additive boundary ledger. The soundness obligations are ordinary schema obligations: S-Def for the descriptor, S-Equiv for Cheeger/Rayleigh conversion, S-Rout and S-Trig for the emitted cut payload, S-Comp for boundary ledgers, and S-Meas for the volume decrease in the recursion.

This sketch leaves one asymmetry visible. Decomposition-grade spectral statements use sparse cuts and flow certificates as their governing objects.

Other spectral results may instead be imported as black boxes when an exact theorem interface is registered through Section 3.18. Moment-accessible spectral shadows, such as closed-walk counts, girth, and bulk spectral-measure statements, may also serve as currencies when they are expressed through recorded counts.

An effective certificate hierarchy with a well-founded progress measure is a sufficient admissibility condition for this framework, not a proved necessary boundary. Obstacles include undecidable or independent targets, nonconstructive existence arguments, recursion without a known finite compression rule, and algebraic objects for which no typed residual or exact black-box interface is available. Such cases may require a different proof-language extension or trust boundary; the present manual makes no general impossibility claim. Dense-host problems with linear-size interfaces are not automatically excluded, but they require their own language extension: regularity, container, or similar global summaries must become first-class descriptors before the branch-closure machinery can operate.

This is also the intrinsic content of the failure classification. Inside a fixed proof language, a failed dashed box is either an execution error caught by a worksheet obligation or a repairable design failure emitted as a typed residual. A scope obstruction is terminal only when no finitely encoded proof-language extension equipped with a well-founded progress measure is available. Otherwise the obstruction names a level-4 language extension.

The cost: volume

Where the method applies, its main cost is **volume**. The framework deliberately trades concentrated insight for explicit accounting. A proof that a specialist might compress into a short global estimate becomes, in this architecture, a branch system with named residuals, ledgers, invariant registers, and closure tables. This expansion is intentional: the compressed estimate is often where the errors described in Section 1.3 occur, and the additional volume records the checks that detect them. The cost is nevertheless real. The referee must check more text, more artifacts, and more cross-references, and the proof may contain no single decisive paragraph whose verification settles the matter.

Three features contain the cost. First, the volume is **locally checkable**: each obligation is local, standard, and independently checkable. Refereeing can be parallelized across branches, and the artifacts of Section 4.4 make checking

a matter of lookup rather than reconstruction. Second, the degradation invariant of Section 4.5 gives partial refereeing a formal output. A referee who verifies k of N branches has established the corresponding reduction theorem, rather than a diffuse confidence judgment about an all-or-nothing argument. Third, the repetitive volume is **mechanizable**. Because every branch terminates in one of the fixed certificate types and every closure pattern has a formal counterpart (Section 1.4), the repetitive part of the proof is the part most amenable to formalization: finite leaves become verified computations, peeling loops become well-founded recursions, branch tables become checked exhaustive case splits, and numerical interlocks become interval-arithmetic obligations.

The intended division of labour assigns the repetitive volume to formalization and the architectural decisions to the human referee: the choice of tests, the invariant ladder, and the budget design. Until such tooling is routine, the limitation remains: the method converts implicit risk into explicit length, and that length is part of the cost of the framework.

A Glossary

Standard terms (matching, star, pigeonhole, fibre, minor, immersion, and the usual counting/entropy/density arguments) are used in their ordinary sense and omitted here. For each method-specific term, the closest standard analogue is indicated where one exists.

Term	Definition	Closest standard analogue
Active / dormant (hot/cold)	A local test is <i>active</i> at a site if it fires there and pays an explicit charge; <i>dormant</i> otherwise.	Case split in density arguments
Certificate alphabet	The five terminal return types C1–C5: direct target contradiction, minimality contradiction, routed defect account, bounded-capacity contradiction, and finite verified impossibility.	Return types
Closure property	The library property that every non-certificate CT output is a typed payload accepted by another CT trigger, while the only terminal successes are C1–C5.	Totality of a case split

Term	Definition	Closest standard analogue
Combinator	A policy-level organizer of tactic invocations, such as sequencing, typed handoff, or black-box isolation; it is not a closure tactic because it does not itself return C1–C5 or a smaller residual.	Proof-script composition
Cost class	Inert scheduler datum attached to a CT entry: check , synth , or loop . Policy uses the datum; the entry does not choose itself.	Instruction latency class
Proposed cut/energy descriptor	A proposed payload for the unregistered extension sketch, summarizing a global spectral or expansion witness by volume, witness boundary mass, conductance scale, side profiles, boundary interface, inherited constraints, and recursive obligations.	Sparse-cut certificate; Rayleigh quotient
Aggregate closure	Closing a residual class by proving its total deficit-carrying capacity is smaller than the deficit the branch state forces it to carry (Section 2.28).	Mass/measure bounds
Attachment point	A designated boundary vertex or element through which a piece attaches to the rest of G .	Terminal (boundaried graphs)
Barrier argument	A global argument showing that a separating structure controls all local tests crossing it, so the global external type excludes the target. Must be audited as a global step.	Separator/barrier arguments
Black box	An external theorem imported through a declared interface and listed in the external-inputs register (Section 3.18).	Cited external result
Blast radius	The transitively closed downstream set of a failed lemma, computed from the invariant register and reverse index (Section 4.5).	Dependency cone
Both-sides test	The admissibility criterion for a branch invariant: its positive outcome must feed a quantitative account and its negative outcome must force routable structure (Section 3.11).	—

Term	Definition	Closest standard analogue
Boundaried piece (<i>piece</i>)	A subobject $(X, \partial X, \text{interface data})$ analyzed together with its controlled interface; its size is bounded only when the invoking tactic requires that hypothesis, and it is <i>proper</i> if $X \neq G$.	Boundaried graph; protrusion
Branch state	The accumulated logical context of a case: baseline hypotheses, minimality, certified exclusions on the current branch path, and the current residual condition.	Case with recorded hypotheses
Budget	A quantitative account in normal form supply – demand $-\rho X $, assembled from standard currencies and required to be additive, monotone under the proof's moves, and payable by a charging scheme (Section 3.12).	Potential function
Carrier	An auxiliary bounded substructure canonically associated to a piece, carrying interface and target data between distant pieces; charges may be routed to it. Carrier quotient: identification of pieces sharing the same carrier data.	Auxiliary/gadget structure
Cell	An inclusion-minimal piece of the decomposition induced by the current branch state; the smallest unit bearing a finite label.	Part/cell of a decomposition
Chain	A maximal sequence of bounded-width pieces glued consecutively along their boundaries.	Bounded-width path decomposition
Charge transfer	Canonical relocation of a demand from a saturated or ineligible payer to a designated high-degree neighbour.	Discharging rule
Charged	A demand is charged when the charging scheme assigns it a payer.	Charging arguments
Charging scheme	A bounded-multiplicity assignment $\pi : \mathcal{D} \rightarrow \mathcal{P}$ from demands to payers, with a canonical (deterministic) assignment rule and a capacity inequality $ \mathcal{D} \leq C \mathcal{P} $.	Charging/ discharging method
Compression	Replacement of a piece by a strictly smaller piece of identical external type; contradicts minimality.	Protrusion replacement (kernelization)

Term	Definition	Closest standard analogue
Currency	A standard local quantity from which budgets are assembled: size, deficiency, surplus, boundary mass, rank currencies, entropy (Section 3.12).	—
Defect (<i>target-defective</i>)	Two pieces sharing a coarse candidate label but distinguished by some outside context. If a context separates pieces assigned the same alleged exact external type, the type definition or its S-Equiv proof has failed. A certified defect enters the designated defect account.	—
Defect register	The structured record of every defect discovered during audit: identifier, node location, classification, residual and surviving theorem, audit evidence, immutable blast radius at discovery, append-only repair history, and status (Section 4.6).	Errata / issue tracker
Deficit-carrying capacity	The total charge a residual class can absorb, counted per member with its capacity bound; the quantity bounded above in an aggregate closure.	—
Degradation invariant	The guarantee that the document always proves a true reduction theorem: a lemma failure changes which reduction theorem is proved, not whether one is proved (Section 4.5).	—
Delocalization	The distinguishing structure escapes the local cell into a larger piece; handled by replacement if proper, or by an audited global argument if not.	—
Demand	An event that must be accounted for by a charging scheme (bad window, blocked pair, missing attachment point, route entry, saturated payer, first-failure witness).	Charged object
Derived normalization	A density/regularity/budget hypothesis whose failure branch is exhausted before any estimate depending on the hypothesis is invoked; see <i>spine</i> (Section 3.18).	Reduction to a normalized case

Term	Definition	Closest standard analogue
Exit / exit flag	An <i>exit</i> is a named closure route of the branch taxonomy (hit, defect, compression, delocalization, overload, auxiliary); an <i>exit flag</i> records whether it remains available in the current branch state.	—
Explicit charge	A contribution to a demand count, bounded from below, paid by an active local test.	Charge (discharging)
External type $\tau(X)$	The complete record of target-relevant answers a piece can exhibit to all compatible outside contexts; equal types = indistinguishable pieces. Computed in <i>exact closed form</i> for the whole-object piece (Section 2.30).	Myhill–Nerode-style context equivalence; type (logic)
First failure / first-failure witness	The minimal scale or index at which a monitored property fails, together with the bounded local data witnessing it. Later failures are not counted unless they create new witnesses.	Minimal counterexample
Hypothesis extraction	The rule that branch invariants are obtained as the hypotheses of the lemmas one wishes to apply, so that every failure branch is generated mechanically and obligated in advance to close from the resource inventory (Section 3.11).	—
Instantiation worksheet	Per-invocation artifact $\text{CT}k\text{-}\#$ recording the input payload, concrete parameters, schema obligations, outcome, artifact writes, and open defect hooks.	Proof obligation record
Invariant register	The numbered list of standing branch-state invariants, grouped by the stage at which each is established (Section 4.4).	—
Label	The finite descriptor of a demand’s combinatorial role in a charging scheme; the pigeonhole is applied over labels.	Color/type in Ramsey arguments

Term	Definition	Closest standard analogue
Language extension	A level-4 repair that adds finitely encoded descriptor records or hierarchies, payload formats, auxiliary finite certificate data, schema obligations, and complete CT entries, and declares which existing C1–C5 class validates each closing leaf before re-running the ordinary closure discipline.	Extension of a formal vocabulary / tactic language
Local degrees of freedom	The independent bounded modifications available at a piece; few degrees of freedom force rigidity.	Degrees of freedom
Local exchange (<i>exchange</i>)	A bounded local operation, presented as two representatives with the same interface, changing a target-relevant quantity by a bounded increment. Classified by the exchange trichotomy : <i>realizing</i> (some context realizes the target), <i>distinguishing</i> (some context separates the representatives), <i>neutral</i> (no context separates them; if one is smaller, compress). Zero-increment exchanges enter a finite same-interface table.	Exchange/switching argument
Local obstruction	A bounded structure explaining why a local test fails at a site; the canonical payer for that failure.	Local obstruction
Local test	A member of a family $\{T_i\}$ of locally checkable conditions whose disjunction is equivalent to the target predicate; <i>realized</i> if some configuration satisfies it.	Local certificate/witness
Lemma schema	A reusable obligation shape such as S-Def, S-Equiv, S-Dich, S-Rout, or S-Trig; a worksheet instantiates it by a named lemma, proposition, theorem, computation, or pending defect row.	Proof obligation schema
Localization lemma	The generic consequence of budget additivity: an aggregate budget deficit yields a connected admissible piece in deficit, converting a global numerical shortfall into a bounded local object (Section 2.25).	First-moment / averaging argument

Term	Definition	Closest standard analogue
Mechanism layer	The tactic reference manual: problem-independent CT entries with signatures, triggers, parameters, procedures, obligations, certificates, outputs, audit hooks, and scope obstructions.	Primitive tactic library
Minimal counterexample	The adversarial object, minimal in a lexicographic size parameter; source of deletion-criticality and replacement tools.	Minimal counterexample
Monochromatic subfamily	A large subfamily of demands all bearing the same label, extracted from an overloaded charging scheme by pigeonhole.	Monochromatic class (Ramsey)
Outside context	Any completion of $G \setminus X$ compatible with the interface data of the piece X .	Gluing context
Overload	The failure of a capacity inequality, $ \mathcal{D} > C \mathcal{P} $; yields a homogeneous substructure rather than an immediate contradiction.	Overloaded charging
Payer	A scarce structure that absorbs charges in a charging scheme; saturated once it has absorbed its capacity.	Payer (charging arguments)
Payload tuple	A typed message $\langle x_1, \dots, x_m \rangle$ emitted by one CT entry and accepted by another; every component is a vocabulary data type and the edge is named $P_{i \rightarrow j}$.	Typed intermediate result
Failure classification	The closure-tactic convention separating — execution errors, repairable design failures, and intrinsic scope obstructions relative to the current proof language. Scope obstructions are terminal only when no finitely encoded proof-language extension with a well-founded progress measure exists.	
Policy layer	The strategy manual: admission, selection, parameter synthesis, nondeterminism inventory, combinators, and worksheet filling. It decides which tactic to invoke and with what parameters.	Tactic orchestration / scheduler

Term	Definition	Closest standard analogue
Signature	The entry field listing the input payload type and all certificate or payload outputs of a CT entry.	Type signature
Proof-language scope condition	The requirement that a target admit a certificate hierarchy with finitely encoded descriptor records and a well-founded progress measure in an explicitly declared proof language; the base proof language uses bounded-radius local tests.	Certified decomposition hierarchy
Tactic	A mechanism entry CTk : a rigid procedure that consumes a trigger payload, discharges fixed schemas, and returns C1–C5 or typed residual payloads.	Primitive proof tactic
Trigger	The syntactic payload-format condition that permits a CT entry to run on the recorded branch state. It is not a selection rule.	Precondition
Verification layer	The assurance system: worksheets, branch tables, dependency graphs, invariant registers, slack tables, resilience registers, and defect registers.	Proof audit layer
Peeling loop	An exit that removes one unit of routed load per pass and returns to the saturation test; closed by a well-founded termination measure plus the per-pass peel lemma (Section 2.26).	Descent / well-founded recursion
Rank forcing	The accounting layer that routes every dependence among local tests through defect/compression/delocalization, so that only independent tests are charged into the budget (Section 2.29).	Linear-independence reduction
Rate	The coefficient ρ of the size term in a budget, computed backwards as any value in the open interval between the provable supply rate and the forced demand rate; thresholds are obtained by solving supply = demand (Section 3.12).	—

Term	Definition	Closest standard analogue
Reduction lattice	The partial order of reduction theorems, — ordered by inclusion of excluded residual classes; verifying a closure moves the current position up, and retracting a closure moves it down by one computable step (Section 4.5).	
Residual	The <i>residual condition</i> is the formula — recorded in the branch state; the <i>residual class</i> is its semantic solution class; and a <i>residual payload</i> is the explicit value sent to a declared consumer. A fully decorated residual packages these data with the piece, branch state, certified absent exits, and relevant minimality, carrier, or type data; it is terminal only after a valid C1–C5 result.	
Selection policy	The priority order that consumes queued payloads, then check-class tactics, then already-synthesized synth-class tactics, then new parameter synthesis, and finally loop or class-closing tactics.	Scheduler policy
Resilience register	A record, per intermediate branch, of — exactly which reduction theorem survives if that branch’s closure fails (Section 4.5).	
Resource inventory	The living list of the proof’s standing — resources — minimality tools, target algebra, black boxes, charging schemes, label alphabet, closed routes — each with its price list; proposed branches unpayable from the inventory are rejected before analysis (Section 3.11).	
Reverse index	The map from each invariant to every lemma consuming it; the impact-analysis artifact (Section 4.4).	Cross-reference index
Routed	A residual payload is routed when its — registered type identifies its producer and unique consumer, the source certifies the route, the payload entails the consumer trigger, and inherited state is transported. A route entry records that handoff.	

Term	Definition	Closest standard analogue
Slack table	The record of every load-bearing inequality with its numerical margin; identifies the binding constraint and prevents two branches from spending the same margin (Section 3.13 and Section 4.3).	—
Scope condition	The requirement that the target admit finite, routable certificates in the current proof language. In the base proof language this means a local test family whose bounded-interface types have finite Myhill–Nerode index under gluing; proof-language extensions may use finitely encoded descriptor records of global witnesses (Section 4.9).	Local testability / certificate hierarchy
Spine	The surviving branch on which all conditional normalizations have been derived or discharged, and along which the main quantitative estimates are valid (Section 3.18).	Normalized main case
Target predicate T	The property whose existence the theorem asserts.	—
Target thickening	Conversion of a sparse arithmetic target into translates $x + \Delta$ via a bounded family of local offsets; no consecutiveness is assumed unless it is supplied by the arithmetic lemma, and avoidance becomes a strong finite/arithmetic constraint (Section 2.31).	Smearing/shifting arguments
Tiered charging	A conditional charging scheme whose availability failure produces a canonical minimal obstruction, itself charged in a designated fallback scheme (Section 2.27).	Two-level charging
Typed handoff	An inter-branch delegation with a declared payload format: the sender proves its residual carries the package, and the receiver’s opening hypothesis is exactly that package (Section 3.18).	Interface contract

Term	Definition	Closest standard analogue
Window	A bounded-size region of G , fixed by the branch’s local structure, in which the local tests are evaluated; <i>bad</i> if the required tests fail there.	Sliding window

B Instantiation map: the Erdős–Gyárfás power-of-two-cycle instantiation

This appendix maps each section of the present document to the part of the Erdős–Gyárfás power-of-two-cycle instantiation — *A Structural Exhaustion Proof of the Erdős–Gyárfás Power-of-Two Cycle Problem: An LLM-assisted minimal-counterexample analysis* (henceforth [EG]) — that instantiates it. [EG] addresses the Erdős–Gyárfás power-of-two cycle problem and was written first; the present document is its generalization. The claim made precise by this table is that every load-bearing move in [EG] is an instance of a named pattern here, so that [EG] serves as a full-scale, rigorous instantiation of the method, and conversely the method inherits from [EG] a worked demonstration that its patterns compose at scale.

Section references to [EG] use its section numbers and titles. The hot/cold quantitative case ledger for the P_{13} entropy interface is now part of [EG] §7 and Figure 11, with the first-failure extraction and finite same-interface table recorded as named proof objects.

B.1 Section-by-section correspondence

This document	Pattern	Instantiation in [EG]
Section 2.1 Scope; branch states	Minimal-counterexample routing; cumulative branch states	[EG] §0 (Architecture): the eleven-step load path over a lexicographically minimal G ; “every residual retains the standing invariants accumulated along the load path.” The surviving cold-branch state (def:surviving-cold-branch) is a literal branch state in the sense of Section 1.1.
Section 1.3 AI-assisted proof rationale	Failure-mode countermeasures	Operationalized by [EG] §1’s referee-facing artifacts (see Section 4.4 row below): fixed certificate lists, named residuals, per-lemma requirement rows, and the reverse index are the mechanical detectors for failure modes 1–3.
Section 2.15 Target predicate and local tests	Local test family equivalent to the target	[EG] §2 (Target cycles as Mersenne returns): the power-of-two-cycle predicate is replaced by edge-rooted Mersenne returns, $R_e(G) \cap \mathcal{M} = \emptyset$ for every oriented edge (lem:return-equivalence) — the local test algebra.

This document	Pattern	Instantiation in [EG]
Section 2.16 Minimal counterexample	Deletion criticality and replacement tools	[EG] §3 (Minimality and replacement): lexicographic minimality in (V , E) ; edge-deletion criticality; no proper internal 3-core; independence of $V_{\geq 4}$.
Section 2.17 Boundaried pieces and external types	Interfaces and Myhill–Nerode-style types	[EG] §3: boundaried atoms with boundary degree profiles, degree-profile fibres, context-universality; [EG] §10: exact response profiles (def:exact-response-profile).
Section 2.18 / Section 2.23 Charging schemes; overcount $\Rightarrow$ closed	Canonical assignment, bounded multiplicity, overload exhaustion	[EG] §3: canonical blocker ledger (def:canonical-blocker-ledger) with its no-overcount identity; capacity-token ledger with token supply and no-overcount lemmas; the exact high-load alternative and the sharp surplus overload audit.
Section 3.4 Direct-hit branch	Immediate target closure	[EG] §14: Type A exits (1)–(3) — Mersenne return, power-of-two theta, P_{13} label collision (def:typeA-saturated-exits).

This document	Pattern	Instantiation in [EG]
Section 3.4 Defect branch	Context-distinguished quotient, routed to the defect account	[EG] §14: exit (4), the target-defective quotient and its peeling charge; the defect route inside the rank branch ([EG] §10).
Section 3.4 Compression branch	Smaller piece of identical type	[EG] §3: replacement lemma (lem:replacement) and hereditary target-uncompressibility (cor:uncompressible); exit (5) in [EG] §14; case G3 of the cold-branch trichotomy and the neutral rows of the same-interface cold table.
Section 3.4 Delocalization branch	Escape to a larger piece; audited global case	[EG] §10: proper-support smearing closure; whole-graph case via lem:no-silent-global-smearing; exit (6) in [EG] §14.
Section 3.4 Overload and monochromatic-family branches	Pigeonhole over labels; matching/star/chain extraction	[EG] §3: same-token patterns, matching-star accounting (lem:same-token-matching-star), role-homogeneous extraction over the window-incidence / remainder-surplus / primitive-carrier token classes.

This document	Pattern	Instantiation in [EG]
Section 3.4 Terminal-residual branch	Exact data closes the residual	[EG] §14: the terminal two-carrier route-8 package (def:typeA- terminal-two-carrier) with carrier-deletion witnesses; the no-go theorem (thm:typeA- two-carrier-nogo) derives exit (4) from the witnesses the residual must carry, contradicting the exit-(4) absence that defines a true route-8 residual.
Section 3.5 Branch-closure algorithm	Steps 1–9	[EG] §1: the twelve-step outline and the eleven-part proof-dependency diagram implement the algorithm at node level; Step 9’s “no unnamed residuals” is [EG]’s explicit residual ledgers.
Section 2.19 Local-versus-global rationale	Global bounds as outputs of local charging	[EG]’s phase structure: entropy caps and the net-charge inequality $\Delta_{\text{net}}(R) < 1/4$ are derived only after window, stub, and wedge charging ([EG] §§7–9, §13).

This document	Pattern	Instantiation in [EG]
Section 2.20 Active/dormant dichotomy; quantitative form	Hot/cold split; linear cold mass from linear hot failure	[EG] §7 and Figure 11: hot windows pay the c_{hot} slot tax; hot failure forces $C \geq (\theta - \theta_{\text{win}})n - o(n)$ cold windows; the 15-stub count converts cold mass into branch excess $b \geq 13C - o(n)$.
Section 2.21 Exchange trichotomy; zero-increment table	Realizing / distinguishing / neutral; finite same-interface table	[EG] §7: bounded switch/theta germs with nonzero increment δ , cases G1/G2/G3; zero-increment switches and short exceptional self-returns are assigned to the finite same-interface cold table (def:cold-same- interface-table, lem:cold-same- interface-table). Also the germ routing inside the sparse branch ([EG] §3).

This document	Pattern	Instantiation in [EG]
Section 2.22 Finite-state pumping	Repeated exact types on long chains	[EG] §14: trace-basin profiles in the route-8 analysis; [EG] §7: cold corridor first failures (def:cold-corridor-first- failure, lem:cold-corridor-first- failure) and the periodic-carrier case of the increment arithmetic (orbit avoids the smear $\Rightarrow$ periodic carrier $\Rightarrow$ visible/invisible dichotomy).
Section 3.3 Closure criteria; branch table	Fixed certificate list; exhaustive tables	[EG] §15: thm:branch-kill ("entropy-admissible closure outside explicit residuals") is the branch table as a theorem; the cold-branch residual ledger (tab:cold-branch- ledger) is a literal instance of the table format, with a quantitative-constraint column.

This document	Pattern	Instantiation in [EG]
Section 3.7 Bookkeeping vs. new theorems	Spend existing resources; no new global inputs	[EG]’s single-black-box discipline (only thm:p13free is external); the cold-branch ledger spends the existing resources explicitly: branch-excess half-edges, first-failure corridors, bounded-overlap extraction, target defect, handoff, and replacement.
Section 3.8 Finite labels; first failure; compression	Label design; multi-scale failure; compression invariant	[EG] §6: the 399-label P_{13} algebra and curvature enumeration (lem:curv-enum); first-failure selection inside the entropy interface; uncompressibility as the standing contradiction lever (invariant 8 of the constraint ledger).

This document	Pattern	Instantiation in [EG]
Section 3.9 Template steps A–J	End-to-end pipeline	Steps A–B $\leftrightarrow$ [EG] §§2–3; C $\leftrightarrow$ [EG] §§3, 10; D $\leftrightarrow$ [EG] §§3, 8–9; E $\leftrightarrow$ [EG] §§7, 11–12 including the hot/cold interface; F–G $\leftrightarrow$ [EG] §14 and the cold first-failure/same-interface lemmas; H $\leftrightarrow$ [EG] §3 token machinery; I $\leftrightarrow$ [EG] §14 trace basins; J $\leftrightarrow$ [EG] appendix (finite curvature enumeration code).
Section 3.10 Audit checklist	Mechanical branch audit	[EG] §1: per-lemma invariant requirements with stage legend; the constraint ledger’s numbered invariants answer checklist items 1–6 per lemma.

This document	Pattern	Instantiation in [EG]
Section 3.11 (invariants) Hypothesis extraction; ladder; granularity	Mechanical invariant design	[EG]’s invariant ladder is the constraint ledger (blocks I–VII), each invariant established at a declared stage and consumed downstream through the reverse index. The mechanical sources of Section 3.11 all appear: estimate hypotheses (surplus above/below $C_{\text{sp}}\sqrt{n}$; remainder entropy above/below $\frac{1}{10}\log_2 n$); ledger saturation (receiver load $L(w) \geq 4q(w)$); scheme availability (the B2 disjointness ledger); rank (full curvature rank vs. rank drop); certificates (certificate labelling present/absent at fan centers); label thresholds (the certificate-marked fan-degree cap).

This document	Pattern	Instantiation in [EG]
Section 3.11 (budgets) Currencies; backward rates; localization; separation	Mechanical budget design	[EG]'s budget stack instantiates the currency list of Section 3.12: cycle rank β, sparsity rank λ, surplus $\sigma(G) = 2m - 3n$, positive deficiency ∂_+, wedge supply W_2, forced curvature cost $c_\Omega W_2$, remainder entropy $\eta(R)$, and the net charge $\mathcal{N}(X) = \partial_+(X) - \sigma(X) - \frac{1}{4} V(X)$ in exact normal form (deficiency and surplus as the matched pair of Section 3.12). The rate $\frac{1}{4}$ is backward-computed against $\tau_{\text{win}} < \frac{1}{4}$, with the solved crossovers $\theta < 1/73$ and $\theta < 1/78$ as quantitative branch invariants (Section 3.12). Localization is prop:negative-net-charge (a global deficit forces a connected admissible X with $\mathcal{N}(X) < 0$); budget separation is the two-budget routing (prop:two-budget) with entropy charged only after rank forcing.

B.2 Closure-tactic instantiation map

ID	Tactic	Trigger in [EG]	Principal closure / intended residuals	Recovery residuals
CT1	Local target tests	Mersenne returns, power-of-two theta tests, and P_{13} -label collisions.	Direct power-of-two-cycle hit, or $P_{1 \rightarrow 2}$, $P_{1 \rightarrow 3}$, $P_{1 \rightarrow 4}$, $P_{1 \rightarrow 5}$, $P_{1 \rightarrow 6}$, $P_{1 \rightarrow 17}$.	—
CT2	Minimality and replacement	Lexicographic minimality; admissible boundaried atoms and replacement candidates.	Edge-deletion criticality, no proper internal 3-core, or replacement compression.	$P_{2 \rightarrow 3}$, $P_{2 \rightarrow 10}$
CT3	External-type compression	Boundaried atoms with degree profiles, context-universality, and exact response profiles.	Valid replacement, accepted routed-defect account, finite type table, $P_{3 \rightarrow 7}$, $P_{3 \rightarrow 12}$, or $P_{3 \rightarrow 8}$.	—
CT4	Charging schemes	Canonical blocker ledger, capacity-token ledger, and receiver-load accounts.	Capacity contradiction, $P_{4 \rightarrow 9}$, $P_{4 \rightarrow 13}$ or $P_{4 \rightarrow 14}$.	—
CT5	Local-to-global bookkeeping	Window, stub, wedge, and net-charge local accounts before global entropy or surplus estimates are invoked.	$P_{5 \rightarrow C4}$ closes; $P_{5 \rightarrow 4}$, $P_{5 \rightarrow 11}$, and $P_{5 \rightarrow 14}$ route onward.	—
CT6	Active/dormant dichotomy	Hot/cold interface in [EG] §7; hot failure forces linear cold mass.	Direct first-failure hit, $P_{6 \rightarrow 4}$, $P_{6 \rightarrow 3}$, $P_{6 \rightarrow 7}$, or $P_{6 \rightarrow 9}$.	$P_{6 \rightarrow 10}$
CT7	Exchange trichotomy	Switch and theta germs with bounded increment δ ; zero-increment same-interface rows.	Realizing germ, neutral replacement, routed defect account, $P_{7 \rightarrow 3}$, or $P_{7 \rightarrow 12}$.	$P_{7 \rightarrow 10}$, $P_{7 \rightarrow 16}$
CT8	Finite-state pumping	Trace-basin profiles, cold corridors, periodic carriers, and same-interface tables.	Compression, finite exhaustion, $P_{8 \rightarrow 10}$ $P_{8 \rightarrow 3}$, or $P_{8 \rightarrow 7}$.	—
CT9	Overload exhaustion	Same-token patterns, role-homogeneous extraction, matching-star and chain overloads.	Target hit, bounded return $P_{9 \rightarrow 4}$, $P_{9 \rightarrow 7}$, or $P_{9 \rightarrow 8}$.	$P_{9 \rightarrow 10}$
CT10	Default refinement	Label-class refinement when a surviving class exposes a missing datum.	Finite class closure, $P_{10 \rightarrow 3}$, $P_{10 \rightarrow 7}$, or $P_{10 \rightarrow 15}$.	—
CT11	Localization	Negative net-charge localization and admissible connected pieces.	$P_{11 \rightarrow 1}$, $P_{11 \rightarrow 7}$, or $P_{11 \rightarrow 14}$.	$P_{11 \rightarrow 10}$
CT12	Peeling loop	Exit-(4) target-defect peeling and routed-load depletion.	Well-founded discharge by capacity.	$P_{12 \rightarrow 4}$, $P_{12 \rightarrow 13}$
CT13	Tiered charging	Type B incidence and disjointness ledgers; minimal overlap obstruction.	Reconciled tier capacity, $P_{13 \rightarrow 4}$, $P_{13 \rightarrow 9}$, or $P_{13 \rightarrow 14}$.	—
CT14	Aggregate closure	Type B bridge residuals and large-budget route-8-only aggregate estimates.	Linear deficit against sublinear capacity.	$P_{14 \rightarrow 9}$, $P_{14 \rightarrow 10}$
CT15	Rank forcing	Curvature rank forcing before entropy charging.	Rank-budget contradiction, $P_{15 \rightarrow 3}$, $P_{15 \rightarrow 7}$, $P_{15 \rightarrow 16}$, or $P_{15 \rightarrow 4}$.	—
CT16	Whole-object exact types	Whole-graph exact response profiles in the no-silent-global-smearing branch.	Literal whole-object identification, $P_{16 \rightarrow 3}$, or $P_{16 \rightarrow 10}$.	—
CT17	Target thickening	Window offsets $\{0, \dots, 12\}$, doubling-orbit arithmetic, and completion classes.	Target block hit, finite survivor exhaustion, $P_{17 \rightarrow 3}$, $P_{17 \rightarrow 10}$ $P_{17 \rightarrow 8}$, or $P_{17 \rightarrow 14}$.	—

B.3 EG proof-flow charts

Figure 30 gives a one-page aggregate view of the proof architecture of [EG]. Figures 31–36 then expand the same proof synopsis CT by CT. Unlike Fig. 3, these are not routing graphs on the tactic library: they follow the actual proof load path. In the detailed panels, each blue node is one tactic application to one payload; the same CT label appears several times precisely

when the proof uses that tactic again with different branch data. The final node of each detailed panel is repeated as the first node of the next panel. In these EG proof-flow diagrams, a solid transition is an intended execution of the admitted proof script, including a non-closing residual, a branch handoff, a rank-drop route, or a peeling loop return. A dashed transition is reserved for an audit-discovered design repair. The displayed EG proof flow has no dashed repair transition: rejected proposals and later repairs belong to the defect register, not to the proof path.

B.4 Advanced patterns in [EG] keyed to Section 3.18

Pattern (this document)	Instantiation in [EG]
Section 2.26 Peeling loops	Exit-(4) peeling: the target-defect route peels exactly one routed load and returns to the saturated-receiver test with the residual load L_4 (diagram nodes $[102] \rightarrow [89]$); termination measure = the routed load.
Section 2.27 Tiered charging	The Type B ledgers: the hybrid B1 incidence payment (tier 1) is conditional on the B2 disjointness ledger; B2 failure produces the <i>minimal Type B overlap obstruction</i> , charged to the fan-mass scheme (tier 2) at its assigned surplus units.
Section 2.28 Aggregate closure	prop:typeB-bridge-sublinear + thm:large-budget-route8-only: Type B bridge residuals are handled aggregately; their total negative net charge is bounded by the global surplus budget $\sigma(G) = O(\sqrt{n})$, hence $o(R)$, while the branch state forces a linear deficit — so the class is empty on the surviving route.

Pattern (this document)	Instantiation in [EG]
Section 3.18 Branch sequencing; derived normalizations; spine	The near-cubic spine (def:near-cubic-spine) is derived by prop:nonnear-cubic-sharp-overload- routing, which exhausts the sparse surplus-token branch before any normalized skeleton budget $B_{\text{skel}} = \frac{3}{2}n \log_2 n + o(n \log n)$ is invoked; every later estimate names the spine branch on which it is valid.
Section 2.29 Rank forcing	[EG] §10: raw curvature tests pass through rank forcing before entropy charging; a rank drop is routed to defect/compression/delocalization (lem:curvature-dependence-routing), and only the full-rank branch (lem:full-rank) pays the forced cost $c_\Omega W_2(R)$ into the two-budget routing of [EG] §11.
Section 2.30 Whole-object piece; exact closed types	lem:no-silent-global-smearing with the exact response profiles of [EG] §10: at $Z = G$ identifications require literal equality of closed profiles.
Section 3.18 Typed handoffs	Exit (7): the Type A branch hands <i>decorated Type B fan data</i> to the Type B branch (nodes $[108] \rightarrow [65]$, typed input [66]); the Type B opening hypothesis is exactly that decorated package. The sparse branch’s “ordinary decorated Type B fan data” route is a second instance.
Section 3.18 Black-box isolation	The Hegde–Sandeep–Shashank P_{13} -free theorem (thm:p13free) is the single load-bearing external input, stated once in [EG] §5 and listed under “External inputs (black boxes)” in the constraint ledger.

Pattern (this document)	Instantiation in [EG]
Section 2.31 Target thickening	[EG] §7: the window offset set $\{0, \dots, 12\}$ smears each self-return length ℓ into the block $[\ell, \ell + 12]$; only 10 of the first 39 lengths dodge $\{4, 8, 16, 32\}$; at large increments the doubling-orbit criterion $\text{ord}_\delta(2) > \delta - 13$ forces a hit, and the surviving small-orbit cases become periodic carriers (G2/G3).
Section 4.3 Constant interlock; computational certificates	[EG] §1: the standing constants table $(c_\Omega, c_{13}, \theta_{\text{win}}, \theta_{\text{high}}, \omega, K, \tau_{\text{win}}, \Theta(n))$, each established at a cited place and reproduced by the code of the finite-curvature-enumeration appendix; the thresholds $\tau(\theta) < 1/4$ and $\tau(\theta) < 3/13$ ($\Leftrightarrow \theta < 1/73$, $\theta < 1/78$) are the recorded interlock of the large-budget layer and the hot/cold branch.
Section 4.4 Proof-engineering artifacts	[EG] §1 in full: the eleven-part numbered dependency diagram (nodes [1]–[157]); the detailed dependency table (node ranges $\rightarrow$ lemma clusters); the constraint ledger of numbered invariants in blocks I–VII; the per-lemma invariant requirements with stage legend; and the invariant $\rightarrow$ lemma reverse index for impact analysis.

Pattern (this document)	Instantiation in [EG]
Section 4.5 Redundant closures; resilience	[EG] §0, “Redundancy and checkability”: the entropy cap, the curvature-rank squeeze, and the net-charge collision are arithmetically distinct closures; the resilience appendix (“Proof resilience: residual counterexamples under lemma failure”) is the resilience register. Its rows instantiate the degradation invariant: a failed branch closure yields the named reduction theorem excluding precisely the corresponding residual class, with the cold-branch residual serving as the public model for this reduction semantics during incremental verification.
Section 4.6 Defect register	[EG]’s audit history is the model instance: named defect families are recorded by affected nodes, blast radius, append-only repair history, and status, and recurring classes are promoted into the Section 3.18 pattern library when they expose a reusable closure shape.

B.5 Reading order

For a reader new to both documents, Appendix C is the natural starting point: it gives the full pipeline on a short theorem, including two audited failed proposals. The suggested sequence is then Section 1 and the opening vocabulary of Section 2.1 → [EG] §0 (Architecture) → Section 2 with the CT inventory and interlock diagram open → [EG] §1 (overview, diagrams, and ledgers) → Sections 3.1–3.20 for policy → Sections 4.3–4.9 for assurance. The CT instantiation map in Appendix B.2 should be read beside the corresponding [EG] chapters; the advanced-pattern map in Appendix B.4 gives the section-level crosswalk. The cold-branch quantitative case ledger in [EG] §7 is recommended as the first full branch to verify end-to-end:

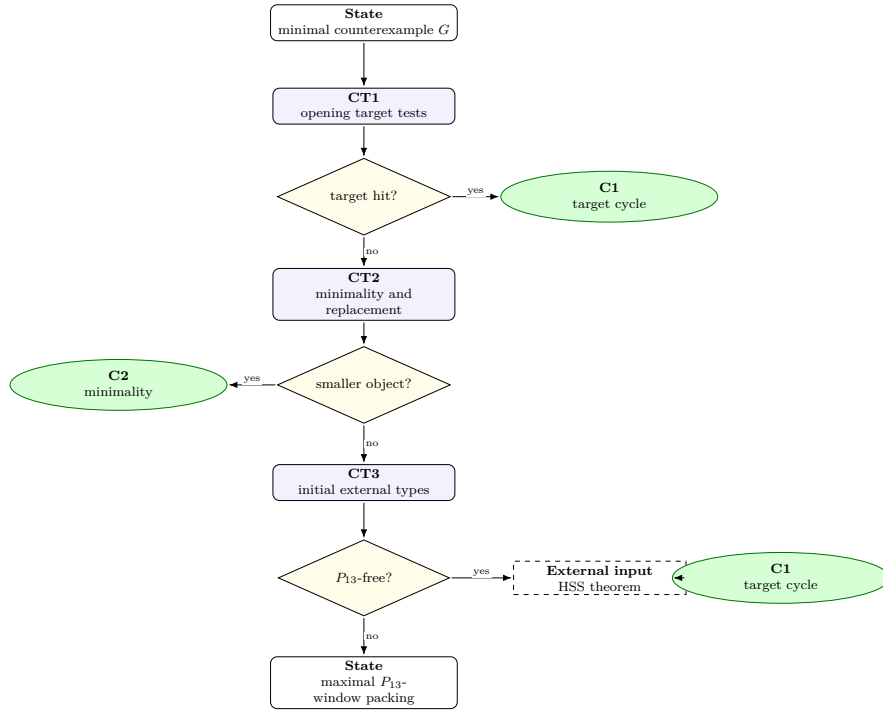


Figure 31: EG high-level CT proof-flow chart I. The first detailed panel starts with the minimal counterexample and ends at the maximal P_{13} -window packing, which is repeated as the first node of Fig. 32.

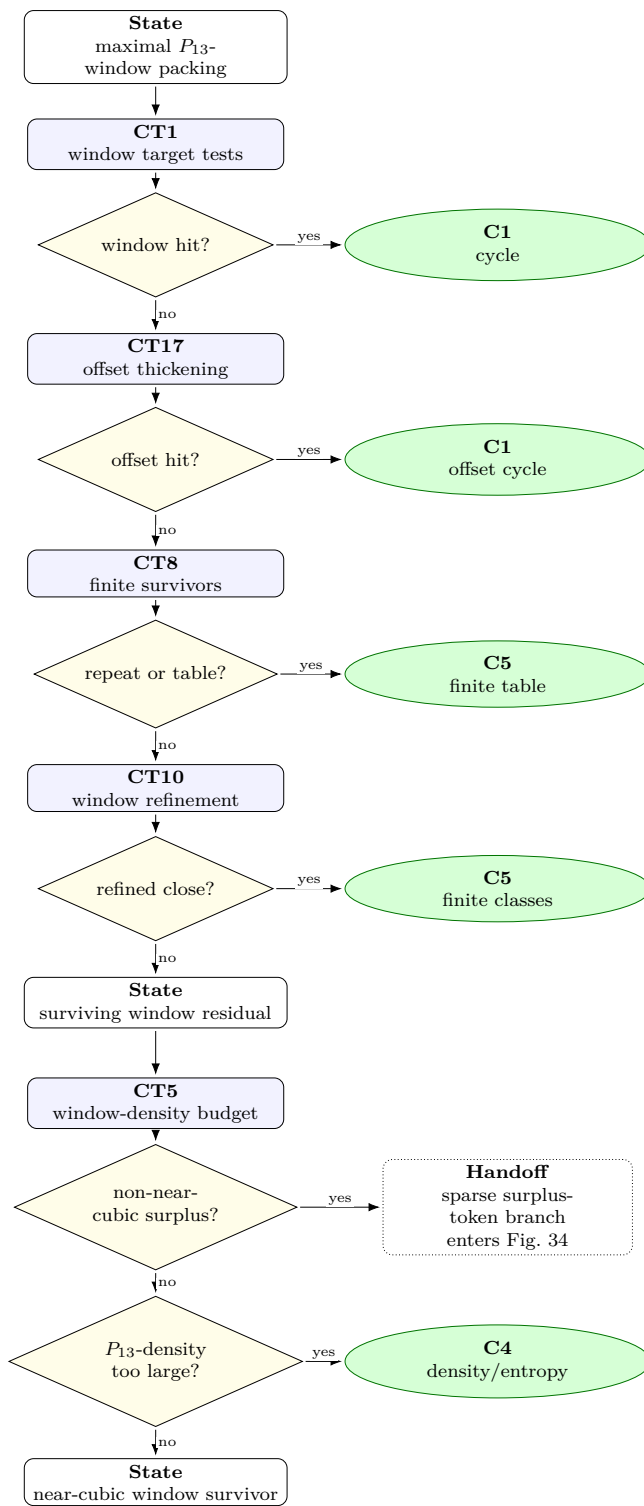


Figure 32: EG high-level CT proof-flow chart II. The second detailed panel starts with the maximal P_{13} -window packing, performs the window target, offset, finite-survivor, refinement, and density-budget tests, and ends at the near-cubic window survivor. The non-near-cubic surplus branch is not terminal; its typed handoff enters Fig. 34.

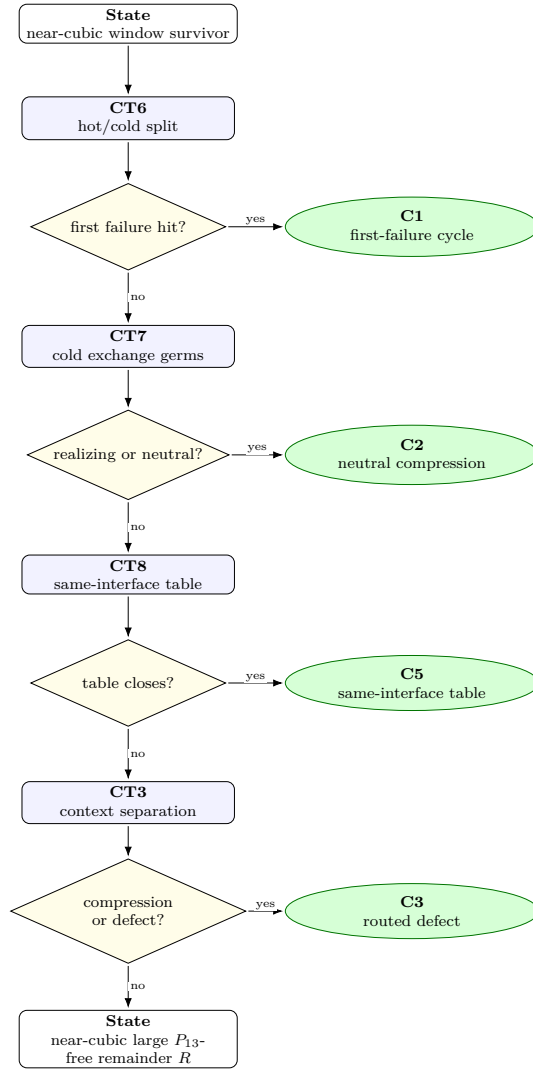


Figure 33: EG high-level CT proof-flow chart III. The third detailed panel expands the hot/cold window interface. Its surviving branch is the near-cubic large P_{13} -free remainder R , repeated as the first node of Fig. 34.

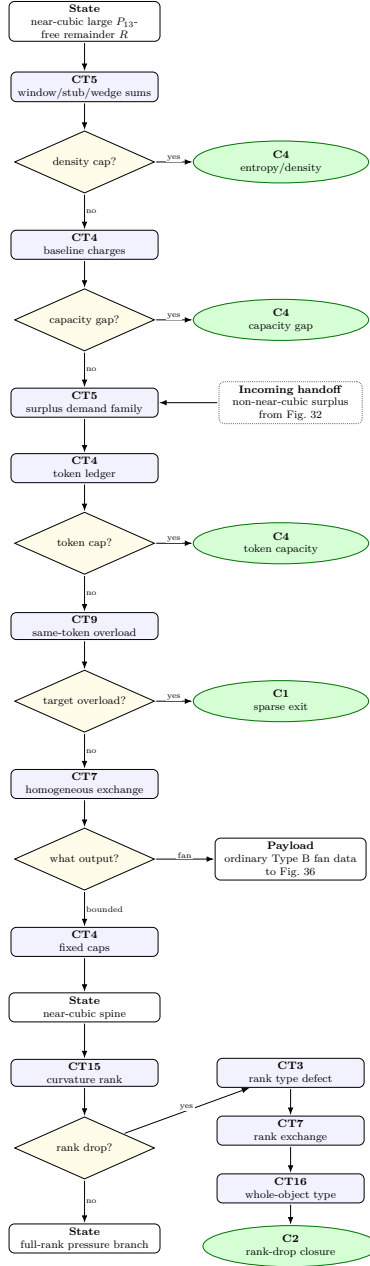


Figure 34: EG high-level CT proof-flow chart IV. The fourth detailed panel starts from the near-cubic large P_{13} -free remainder, incorporates the non-near-cubic surplus handoff from Fig. 32, runs the surplus-token branch, and ends at the full-rank pressure branch. Ordinary Type B fan data is emitted as a typed payload for Fig. 36.

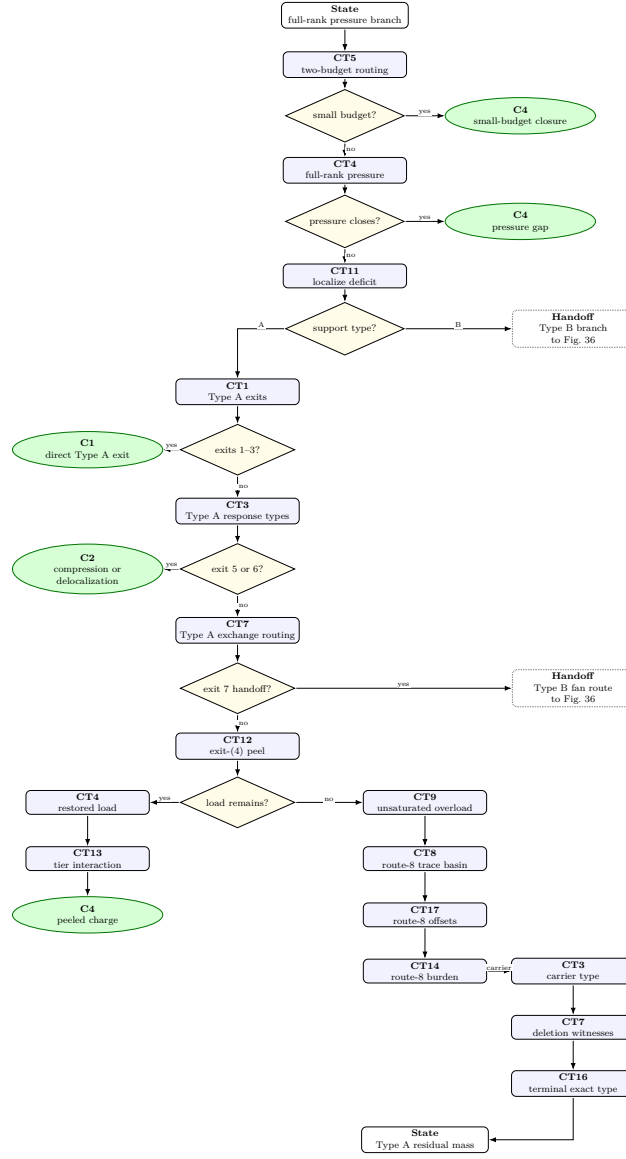


Figure 35: EG high-level CT proof-flow chart V. The fifth detailed panel starts from the full-rank pressure branch, localizes the deficit, and expands the Type A route. Type B handoffs are passed forward to Fig. 36; the surviving Type A output is the residual-mass state repeated in Fig. 36.

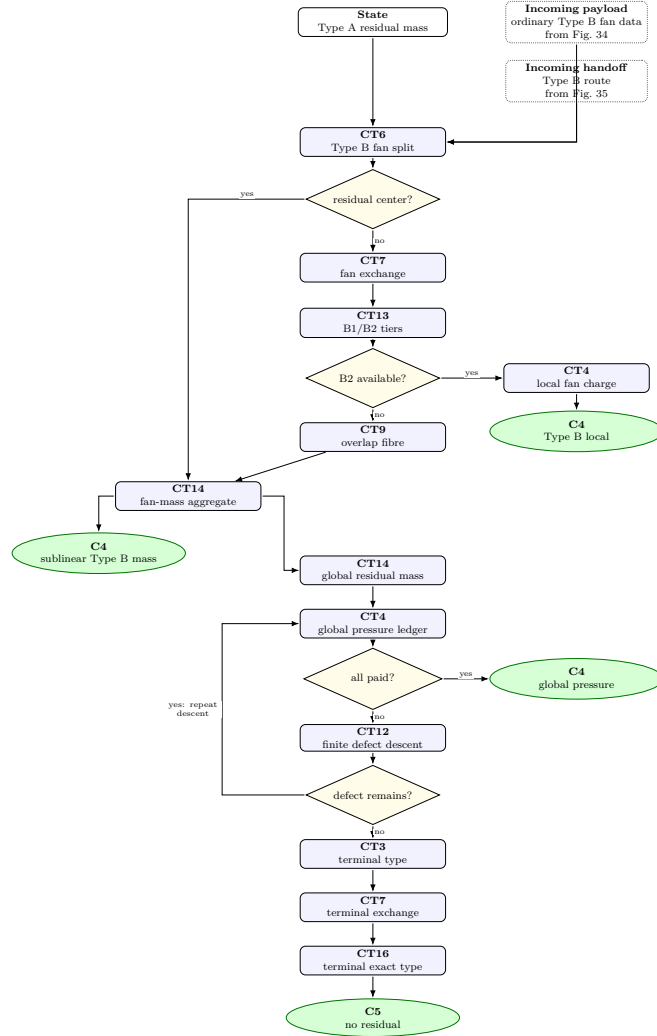


Figure 36: EG high-level CT proof-flow chart VI. The sixth detailed panel joins the Type A residual mass with the Type B fan and handoff payloads, closes the Type B mass, and runs the final pressure and finite-descent closure.

it is short, fully quantitative, and exercises the active/dormant dichotomy (Section 2.20), the exchange trichotomy (Section 2.21), target thickening (Section 2.31), and the exhaustive-table closure format (Section 3.3) in a single self-contained unit.

C A small worked instantiation

This appendix runs the full pipeline on a theorem short enough to verify in one sitting. Its purpose is mechanical transparency rather than mathematical depth. Every main artifact of the method appears in miniature, and two deliberately failed proposals are included to show the audits rejecting them. The target set here is the even numbers, a single residue class modulo 2; it is the degenerate shadow of the sparse target $\{2^k\}$ in the Erdős–Gyárfás instantiation.

The proof trace below records the same argument in the tactic-invocation format used by the library. The later subsections supply the mathematical details behind each row.

Step	Tactic	Trigger data	Parameters	Outcome	Emitted
1	CT1	Target predicate “even cycle” and parity-theta tests.	Local test family $T_{x,y}$.	Test equivalence.	Target-avoiding state.
2	CT2	Heavy edge uv with $d(u), d(v) \geq 4$.	Lexicographic size order (V , E) .	C2.	Invariant I1.
3	CT12, rejected	Proposed peel $\mu = V_{\geq 4} $ by deleting a high-degree incident edge.	Candidate loop move.	S-Rest fails: the move can break $\delta \geq 3$.	No tactic output; I1 remains.
4	CT6	Maximum path P and endpoint v .	Measure $ P $.	Residual branch.	I2: $N(v) \subseteq P$.
5	Global shortcut, rejected	Cycle-space parity argument.	None.	Certificate check fails: no realized local test.	No C-certificate.
6	CT9	Three endpoint-neighbour positions and two parity labels.	Capacity 1 per parity class.	Overload.	Same-parity pair.
7	CT1	Parity theta at the monochromatic pair.	Two internally disjoint paths.	C1.	Even cycle.

In the worksheets below, “parameter-synthesis rule” refers to Section 3.16.

Worksheet CT2-1.

Instance: CT2-1 Branch node: [2]
 Input payload: <minimal G; heavy edge uv with $d(u), d(v) \geq 4$ >
 emitted by: opening script after CT1-1
 Parameters: slot (1) size order := $(|V|, |E|)$
 synthesized by parameter-synthesis rule
 slot (2) deletion move := delete uv
 inventory item: minimality
 Obligations: S-Def<piece uv, deletion> := Lemma C.2, proved
 S-Dich<delta preserved> := degree check, proved
 S-Comp<smaller object> := lexicographic order, proved
 Outcome: C2, deletion-minimality variant
 Artifacts: branch-table row 2; invariant I1:
 every edge has a degree-3 endpoint
 Open items: none

Worksheet CT9-1.

Instance: CT9-1 Branch node: [6]
 Input payload: <demands $\{p_1, p_2, p_3\}$;
 payers = parity classes; capacity $C = 1$ >
 emitted by: CT6-1
 Parameters: slot (1) label map := position mod 2
 synthesized by parameter-synthesis rule
 slot (2) extraction target := pair
 synthesized by parameter-synthesis rule
 Obligations: S-Dich<3 demands, 2 classes> := pigeonhole, proved
 S-Rout<pair payload> := positions distinct, proved
 S-Trig<pair $\rightarrow$ CT1> := Lemma C.6, proved
 Outcome: emitted <parity theta at (x_i, x_j) > $\rightarrow$ CT1-2
 Artifacts: branch-table row 6; no slack consumed
 Open items: none

Reject rows. Step 3 is an (E)-reject for CT12-1/S-Rest: the proposed edge deletion does not restore the standing minimum-degree branch state, so no CT12 output is produced. Step 5 is a certificate reject: the cycle-space parity argument produces an even-cardinality cycle-space element rather than a realized local test, so it is not C1 and emits no payload. Both rejections remain in the defect register rather than in the proof flow.

Theorem C. *Every finite graph with minimum degree at least 3 contains a cycle of even length.*

The bound is sharp: odd cycles C_{2k+1} have minimum degree 2 and no even cycle. This sharpness certificate reappears in Appendix C.6 as the solved

crossover of the rate computation.

C.1 Target predicate and local tests

The target predicate is

$$T(G) = 1 \iff G \text{ contains an even cycle.}$$

For distinct vertices x and y , the **parity theta** $T_{x,y}$ is realized when G contains two internally disjoint x - y paths whose lengths have the same parity.

Lemma C.1 (test equivalence). $T(G)$ holds if and only if some parity theta is realized.

Proof. Two internally disjoint x - y paths of lengths p and q with $p \equiv q \pmod{2}$ form a cycle of length $p + q$, which is even. Conversely, an even cycle split at any two of its vertices gives two internally disjoint paths with $p + q$ even, hence $p \equiv q \pmod{2}$. $\square$

Target thickening in the degenerate case. The target values $2\mathbb{Z}$ form one residue class, so the offset machinery of Section 2.31 collapses: the offset family $\{0,1\}$ already covers all residues, and the large-scale arithmetic criterion is just parity. For the target $\{2^k\}$, the same machinery does not collapse; Section 2.31 is needed in full.

C.2 Minimal counterexample and resource inventory

Assume a counterexample and choose G minimal in the lexicographic order $(|V|, |E|)$. The resource inventory is small.

Resource	Price list
Minimality, by deletion	Any smaller subgraph with $\delta \geq 3$ contradicts minimality, since target-freeness is subgraph-closed
Parity algebra	Arithmetic modulo 2; supplies the two-element label alphabet
Extremal selection	A maximum-length path exists, and an endpoint has no off-path neighbour

Resource	Price list
Local tests	Realizing any parity theta closes the proof
Pigeonhole	Two labels; any three demands overload the label classes

There are no black boxes, carriers, or tiers. The branches below are admitted or rejected against this explicit inventory.

C.3 A failed peeling proposal and the recorded invariant

A natural first proposal is a derived normalization: reduce to the cubic case. The proposed peeling loop uses the measure $\mu = |V_{\geq 4}|$ and deletes an edge incident to a vertex of degree at least 4. The moves $\times$ budgets audit of Section 3.13 gives the following table.

Move variant	$\delta \geq 3$ preserved	Target-free preserved	μ change	Available?
Delete uv , with $d(u) \geq 4$ and $d(v) \geq 4$	yes	yes, as a subgraph	≤ -1	yes
Delete uv , with $d(u) \geq 4$ and $d(v) = 3$	no, since v drops to 2	yes	—	no

The first row is available, and minimality therefore forbids its precondition in G : if an edge had both endpoints of degree at least 4, deletion would produce a smaller counterexample. The second row is unavailable because it fails to preserve the baseline hypothesis. The loop can terminate with μ positive, for example at a degree-4 vertex all of whose neighbours are cubic. The proposed peel lemma fails to re-establish the branch state after each pass, as warned in Section 2.26, so cubicity remains unproved.

The audit nevertheless yields a valid invariant:

I1 (deletion criticality). In G , every edge has an endpoint of degree exactly 3.

The invariant register records I1. In this worked example, no downstream branch consumes it. The granularity rule of Section 3.11 permits this: I1 is a free criticality consequence, but it supports no subsequent branch. In the Erdős–Gyárfás problem, the analogous near-cubic statement is a substantive theorem requiring its own machinery, which is why Section 3.18 requires normalizations to be derived before they are used.

C.4 The branch invariant by hypothesis extraction

The leaf we want is a parity pigeonhole among several chords into one path. Hypothesis extraction (Section 3.11) identifies the needed hypothesis: some vertex has three neighbours on a common path that cannot be extended past it. Extremal selection supplies that hypothesis. Fix a maximum-length path

$$P = x_0x_1 \cdots x_\ell$$

and let $v = x_\ell$.

Let P_1 be the branch invariant “every neighbour of v lies on P .” The both-sides test of Section 3.11 has all four blanks filled:

P_1 pays the positions of at least three neighbours of v
on P into the parity-pigeonhole account;

$\neg P_1$ forces a path longer than P
, routed to the extremal-choice contradiction.

The $\neg P_1$ branch closes immediately: a neighbour $w \notin P$ extends P to a longer path. On the surviving branch, v has three neighbours $x_{p_1}, x_{p_2}, x_{p_3}$ with

$$0 \leq p_1 < p_2 < p_3 \leq \ell - 1.$$

C.5 A failed global-estimate shortcut

Before the leaf, consider the following candidate shortcut: the cycle space has dimension at least 2; choose two independent cycles; in a counterexample both are odd; their symmetric difference has even cardinality, hence gives an even cycle. The certificate check of Section 3.3 rejects this argument. The

object produced is an even-cardinality cycle-space element rather than a realized local test. An element of the cycle space may be an edge-disjoint union of cycles, and two odd cycles can sum to two odd cycles.

The admissible hit certificate is a realized parity theta, which requires the internal-disjointness data that the global argument does not construct. Repairing the shortcut requires controlling how the two cycles intersect; that repair reconstructs the local analysis encoded by the theta test. The global estimate has therefore displaced the local work rather than closing the branch.

C.6 The leaf: overload, monochromatic pair, direct hit

The demands are the three positions p_1, p_2, p_3 . The labels are residues modulo 2. The payers are the two residue classes, each with capacity 1.

Overload. Since $|\mathcal{D}| = 3$ and $C|\mathcal{P}| = 2$, the label classes are overloaded. The pigeonhole principle extracts a monochromatic pair $p_i \equiv p_j \pmod{2}$, with $p_i < p_j$.

Direct hit. There are two internally disjoint x_{p_i} - x_{p_j} paths. One is the subpath of P , of length $p_j - p_i$, which is even and at least 2. The other is the path through v , of length 2. Since $v = x_\ell$ and $p_j \leq \ell - 1$, v is not internal to the subpath of P . The parity theta $T_{x_{p_i}, x_{p_j}}$ is realized, giving an even cycle of length $p_j - p_i + 2 \geq 4$. This contradicts the counterexample assumption. $\square$

The rate computation of Section 3.12 appears here in miniature. The forced demand rate is $\rho_{\text{dem}} = \delta \geq 3$, the number of endpoint neighbours forced onto P . The supply rate is $\rho_{\text{sup}} = 2$, the two residue classes at capacity 1. The branch closes precisely when $\rho_{\text{dem}} > \rho_{\text{sup}}$. Solving supply = demand gives the crossover $\delta = 2$, so the hypothesis $\delta \geq 3$ is the smallest integer strictly above the crossover. The odd cycle C_{2k+1} is the sharpness certificate at $\delta = 2$. The recorded slack is one demand.

C.7 The closing artifacts

The branch table has the format of Section 3.3.

Branch	Residual condition	Demand	Payer	Bounded multiplicity	Closure
Heavy edge	an edge has both endpoints of degree at least 4	edge	minimality	n/a	contradiction; yields I1
Rejected loop	cubicity via $\mu = V_{\geq 4} $	—	—	availability fails	rejected by audit
Off-path neighbour	$w \in N(v) \setminus P$	longer path	extremal measure $ P $	n/a	contradiction
Parity overload	p_1, p_2, p_3 modulo 2	three positions	two residue classes, $C = 1$	$3 > 2$	monochromatic pair gives direct hit

No terminal residual survives.

The invariant register is:

- I0: $\delta \geq 3$ and minimality, as baseline assumptions.
- I1: every edge has a degree-3 endpoint, established in Appendix C.3 and consumed by no later branch.
- I2: all neighbours of the maximum-path endpoint lie on the path, established in Appendix C.4 and consumed in Appendix C.6.

C.8 Scope of the example

This instantiation contains no charging scheme with multiplicity greater than 1, no tiers, no typed handoffs, no aggregate closure, and no nontrivial exact types. Every local exchange is immediately realizing, so the distinguishing and neutral cases of the exchange trichotomy do not fire. This is a notation and workflow tutorial, not evidence of end-to-end framework completeness. The Erdős–Gyárfás candidate proof and its live defect dossier are the full-scale review target.

D Library index and quick-reference card

This appendix is the compact index of the tactic library in Section 2. The controlling descriptions remain the inventory in Section 2, the tactic diagrams, and the certificate alphabet.

ID	Name	Tier	Cost	Trigger, short form	Certificates	Intended residuals	Recovery residuals
CT1	Local target tests	O	check	Local test family and witness spaces recorded.	C1	$P_{1 \rightarrow 2}, P_{1 \rightarrow 3}, P_{1 \rightarrow 4}, P_{1 \rightarrow 5}, P_{1 \rightarrow 6}, P_{1 \rightarrow 17}$	—

ID	Name	Tier	Cost	Trigger, short form	Certificates	Intended residuals	Recovery residuals
CT2	Minimality and O replacement	O	check	Proper reducible or replaceable piece in a minimal counterexample.	C2	—	$P_{2 \rightarrow 3}, P_{2 \rightarrow 10}$
CT3	External-type compression	R	synth	Certified interface and finite response/type data.	C2, C3, C5	$P_{3 \rightarrow 7}, P_{3 \rightarrow 8}, P_{3 \rightarrow 12}$	—
CT4	Charging schemes	E	synth	Demand set, payer set, and canonical assignment rule recorded.	C4	$P_{4 \rightarrow 9}, P_{4 \rightarrow 14}$	$P_{4 \rightarrow 13}$
CT5	Local-to-global bookkeeping	E	synth	Linear local-site family with local witnesses and payers.	C4	$P_{5 \rightarrow 4}, P_{5 \rightarrow 11}, P_{5 \rightarrow 14}$	—
CT6	Active/dormant dichotomy	E	synth	Activity threshold and first-failure witness data recorded.	C1	$P_{6 \rightarrow 3}, P_{6 \rightarrow 4}, P_{6 \rightarrow 7}, P_{6 \rightarrow 9}$	$P_{6 \rightarrow 10}$
CT7	Exchange trichotomy	R	synth	Bounded exchange pair on a certified common interface.	C1, C2, C3	$P_{7 \rightarrow 3}, P_{7 \rightarrow 12}$	$P_{7 \rightarrow 10}, P_{7 \rightarrow 16}$
CT8	Finite-state pumping	R	synth	Bounded sequence with finite labels or exact types.	C2, C5	$P_{8 \rightarrow 3}, P_{8 \rightarrow 7}$	$P_{8 \rightarrow 10}$
CT9	Overload exhaustion	R	check	Payer fibre exceeds its certified capacity.	C1	$P_{9 \rightarrow 4}, P_{9 \rightarrow 7}, P_{9 \rightarrow 8}$	$P_{9 \rightarrow 10}$
CT10	Default refinement	R	synth	Residual class exposes a finite missing datum.	C5	$P_{10 \rightarrow 3}, P_{10 \rightarrow 7}, P_{10 \rightarrow 15}$	—
CT11	Localization	R	synth	Additive global deficit with admissible decomposition.	—	$P_{11 \rightarrow 1}, P_{11 \rightarrow 7}, P_{11 \rightarrow 14}$	$P_{11 \rightarrow 10}$
CT12	Peeling loop	L	loop	Routed defect/load account with well-founded peel measure.	C4	—	$P_{12 \rightarrow 4}, P_{12 \rightarrow 13}$
CT13	Tiered charging	L	loop	First-tier availability or named obstruction data.	C4	$P_{13 \rightarrow 4}, P_{13 \rightarrow 9}, P_{13 \rightarrow 14}$	—
CT14	Aggregate closure	L	synth	Residual class has forced mass and a capacity record.	C4	—	$P_{14 \rightarrow 9}, P_{14 \rightarrow 10}$
CT15	Rank forcing	E	synth	Target-relative rank datum controls independent coordinates.	C4	$P_{15 \rightarrow 3}, P_{15 \rightarrow 4}, P_{15 \rightarrow 7}, P_{15 \rightarrow 16}$	—
CT16	Whole-object exact types	R	synth	Delocalized closed object with computable exact type.	C2	$P_{16 \rightarrow 3}, P_{16 \rightarrow 10}$	—
CT17	Target thickening	R	synth	Bounded offset family against sparse target set.	C1, C5	$P_{17 \rightarrow 8}, P_{17 \rightarrow 14}$	$P_{17 \rightarrow 3}, P_{17 \rightarrow 10}$

Certificate alphabet. C1 is direct target contradiction. C2 is minimality contradiction. C3 is entry in an already-routed defect account. C4 is bounded-capacity contradiction. C5 is finite verified impossibility.

Combinators. Typed handoff, branch sequencing or derived spine, and black-box isolation belong to the strategy layer of Section 3: they organize tactic applications rather than closing a branch or emitting a smaller residual by themselves.

Selection priority. Consume queued payloads first. Then run **check**-class tactics whose triggers are already recorded. Then run **synth**-class tactics whose parameters already sit in the inventory. Then synthesize

missing parameters. Use **loop**-class and class-closing tactics only after their measures, rates, and slack records are in place.

The loop. Maintain the current branch state, choose a tactic whose syntactic trigger is present, discharge the displayed obligations, record the certificate or named residual, and repeat until every leaf is one of C1–C5.

Forbidden closing phrases. “This structure should be impossible”; “this seems periodic”; “this should be rare”; “this resembles a known theorem”; “this cannot happen generically.” Each phrase is a prompt for stratification, not a closure.

The both-sides blanks. Before creating a branch, fill both lines:

P pays _____ into account _____; $\neg P$ forces _____ routed to _____.

The seven invariant sources. Estimate hypotheses; ledger saturation; scheme availability; rank; certificates; first failure; label thresholds. The rule is: invariants are the hypotheses of the lemmas one wishes to apply.

Budget recipe. Use the normal form

$$B = \text{supply} - \text{demand} - \rho|X|$$

over the standard currencies: size, deficiency, surplus, boundary mass, rank, and entropy. Deficiency and surplus travel as a pair. Compute ρ_{sup} from provable supply and ρ_{dem} from forced demand; choose $\rho \in (\rho_{\text{sup}}, \rho_{\text{dem}})$; record slack at both ends; and obtain thresholds by solving $\text{supply} = \text{demand}$. An empty interval calls for a sharper supply term or a new tier, not a forced value of ρ .

The audit rule. Maintain a moves $\times$ budgets table. No available move may increase the adversary’s supply, erase a recorded deficit, or break a standing hypothesis. Every violation identifies a missing compensating term.

The repair ladder. Level 1 is numerical: recompute or re-spend slack. Level 2 is structural: apply Section 3.11 to the re-opened residual. Level 3 is pattern-level: name the new pattern, add it to Section 3.18, and instantiate it. Level 4 extends the proof language and then adds the tactic family that consumes the new payloads. A persistent residual is a description of where the difficulty is concentrated. At every level, classify the defect, record the

audit evidence, make the repair, rerun the applicable checks, and append the attempt to the defect register. Close the row only after its exit test passes; a terminal scope obstruction remains open with disposition **terminal-S**.

The artifacts. The normative artifact inventory and source locations are given in the Assurance table at the start of Section 4.